

Design Patterns: A Complete Guide

A structured and beginner-friendly reference to the three major categories of software design patterns — Creational, Structural, and Behavioral.

Design patterns are reusable solutions to commonly occurring problems in software design. They are not code — they are blueprints or templates that guide how to structure your code to solve a problem efficiently and elegantly.

Table of Contents

- [1. Creational Design Patterns](#)
 - [Singleton](#)
 - [Factory Method](#)
 - [Abstract Factory](#)
 - [Builder](#)
 - [Prototype](#)
 - [2. Structural Design Patterns](#)
 - [Adapter](#)
 - [Bridge](#)
 - [Composite](#)
 - [Decorator](#)
 - [Facade](#)
 - [Flyweight](#)
 - [Proxy](#)
 - [3. Behavioral Design Patterns](#)
 - [Chain of Responsibility](#)
 - [Command](#)
 - [Iterator](#)
 - [Mediator](#)
 - [Memento](#)
 - [Observer](#)
 - [State](#)
 - [Strategy](#)
 - [Template Method](#)
 - [Visitor](#)
-

1. Creational Design Patterns

Overview

Creational patterns deal with **how objects are created**. They abstract the instantiation process, making a system independent of how its objects are created, composed, and represented. These patterns give you more flexibility in deciding which objects need to be created for a given situation.

Singleton

Description: Ensures that a class has only **one instance** throughout the lifetime of an application and provides a global access point to that instance.

- **Why it is used:** To control access to shared resources such as a configuration manager, logger, or database connection — where having multiple instances would cause conflicts or waste resources.
 - **Where it is commonly used:** Logging systems, application configuration managers, thread pools, caching systems.
-

Code Example (CPP)

```
/*
 * SINGLETON DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Singleton pattern ensures a class has only one instance and provides
 * a global point of access to that instance.
 *
 * WHY DOES IT EXIST?
 * - Some resources should only exist once (database connections, logging
 * services, configuration managers)
 * - Prevents multiple instances from causing conflicts or inconsistent
 * state
 * - Provides controlled access to the sole instance
 *
 * WHEN TO USE IT?
 * - When exactly one instance of a class is needed to coordinate actions
 * across the system
 * - When the sole instance should be extensible by subclassing, and
 * clients should be able to use an extended instance without modifying their
 * code
 * - Common use cases: database connection pools, logging services,
 * configuration managers, thread pools, caches
 *
 * PROS:
 * - Controlled access to the sole instance
 * - Reduced namespace pollution (compared to global variables)
 * - Can be subclassed
 * - Can be lazy-loaded (created only when first needed)
 *
 * CONS:
 * - Violates Single Responsibility Principle (handles both its business
 * logic and instance creation)
 * - Can hide dependencies (making code harder to test)
 * - Global state can lead to unexpected behavior in multi-threaded
 * environments
 * - Difficult to unit test due to global state
 *
 * REAL-WORLD ANALOGY:
 * Think of a government - there's only one president or prime minister at
```

a time. Everyone who needs to interact with the head of state goes through the same single instance. You can't have two presidents simultaneously making conflicting decisions.

```

*/

#include <iostream>
#include <memory>
#include <mutex>

// Thread-safe Singleton class using Meyer's Singleton pattern
class DatabaseConnection {
private:
    // Private constructor to prevent instantiation from outside
    DatabaseConnection() {
        std::cout << "DatabaseConnection: Initializing connection..." <<
std::endl;
        // Simulate connection setup
        isConnected = true;
    }

    // Private destructor to prevent deletion from outside
    ~DatabaseConnection() {
        std::cout << "DatabaseConnection: Closing connection..." <<
std::endl;
        isConnected = false;
    }

    // Delete copy constructor and assignment operator
    DatabaseConnection(const DatabaseConnection&) = delete;
    DatabaseConnection& operator=(const DatabaseConnection&) = delete;

    bool isConnected;
    int queryCount = 0;

public:
    // Static method to get the single instance
    // This is thread-safe in C++11 and later (Meyer's Singleton)
    static DatabaseConnection& getInstance() {
        // Static local variable is created only once, on first call
        // This is thread-safe in C++11 and later
        static DatabaseConnection instance;
        return instance;
    }

    // Business logic methods
    void executeQuery(const std::string& query) {
        if (isConnected) {
            queryCount++;
            std::cout << "Executing query #" << queryCount << ": " << query
<< std::endl;
        } else {
            std::cout << "Error: Not connected to database" << std::endl;
        }
    }
}

```

```

    int getQueryCount() const {
        return queryCount;
    }

    bool getConnectionStatus() const {
        return isConnected;
    }
};

// Alternative implementation using std::call_once for explicit thread
// safety
// This is useful when you need more control over initialization
class Logger {
private:
    static std::unique_ptr<Logger> instance;
    static std::once_flag initFlag;

    Logger() {
        std::cout << "Logger: Initializing logger..." << std::endl;
    }

    ~Logger() {
        std::cout << "Logger: Destroying logger..." << std::endl;
    }

    // Delete copy constructor and assignment operator
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;

    int logCount = 0;

public:
    static Logger& getInstance() {
        std::call_once(initFlag, []() {
            instance.reset(new Logger());
        });
        return *instance;
    }

    void log(const std::string& message) {
        logCount++;
        std::cout << "[LOG #" << logCount << "] " << message << std::endl;
    }

    int getLogCount() const {
        return logCount;
    }
};

std::unique_ptr<Logger> Logger::instance = nullptr;
std::once_flag Logger::initFlag;

// Example 3: Double-Checked Locking (Explicit Serialized Access)

```

```
// This pattern was common before C++11. It uses a mutex to ensure
// that only one thread creates the instance.
class SerializedSingleton {
private:
    static SerializedSingleton* instance;
    static std::mutex mutex;

    SerializedSingleton() {
        std::cout << "SerializedSingleton: Instance created." << std::endl;
    }

public:
    static SerializedSingleton* getInstance() {
        // First check (no locking for performance)
        if (instance == nullptr) {
            // Locking for thread safety
            std::lock_guard<std::mutex> lock(mutex);
            // Second check (to prevent race condition)
            if (instance == nullptr) {
                instance = new SerializedSingleton();
            }
        }
        return instance;
    }

    void doWork() {
        std::cout << "SerializedSingleton is working..." << std::endl;
    }
};

SerializedSingleton* SerializedSingleton::instance = nullptr;
std::mutex SerializedSingleton::mutex;

// Example 4: Bypassing Singleton
// In C++, the most common way to bypass a Singleton is using 'friend'
// classes/functions
// or pointer manipulation (hacking private access).
class BypassableSingleton {
private:
    static BypassableSingleton* instance;
    BypassableSingleton() {
        std::cout << "BypassableSingleton: Instance created." << std::endl;
    }

public:
    static BypassableSingleton* getInstance() {
        if (!instance) instance = new BypassableSingleton();
        return instance;
    }

    // This 'friend' class can access the private constructor!
    friend class SingletonHacker;
};
```

```

BypassableSingleton* BypassableSingleton::instance = nullptr;

class SingletonHacker {
public:
    static BypassableSingleton* createNewInstance() {
        // This works because SingletonHacker is a friend of
        BypassableSingleton
        return new BypassableSingleton();
    }
};

int main() {
    std::cout << "=== Singleton Pattern Demo ===" << std::endl <<
    std::endl;

    // Using Meyer's Singleton (DatabaseConnection)
    std::cout << "--- DatabaseConnection Example ---" << std::endl;

    // Get the singleton instance
    DatabaseConnection& db1 = DatabaseConnection::getInstance();
    db1.executeQuery("SELECT * FROM users");

    // Get the same instance again
    DatabaseConnection& db2 = DatabaseConnection::getInstance();
    db2.executeQuery("SELECT * FROM products");

    // Verify they are the same instance
    std::cout << "Same instance? " << (&db1 == &db2 ? "Yes" : "No") <<
    std::endl;
    std::cout << "Total queries: " << db1.getQueryCount() << std::endl <<
    std::endl;

    // Using std::call_once Singleton (Logger)
    std::cout << "--- Logger Example ---" << std::endl;

    Logger& logger1 = Logger::getInstance();
    logger1.log("Application started");

    Logger& logger2 = Logger::getInstance();
    logger2.log("User logged in");
    logger2.log("Processing request");

    std::cout << "Same instance? " << (&logger1 == &logger2 ? "Yes" : "No")
    << std::endl;
    std::cout << "Total logs: " << logger1.getLogCount() << std::endl <<
    std::endl;

    // Using Double-Checked Locking
    std::cout << "--- SerializedSingleton Example ---" << std::endl;
    SerializedSingleton* s1 = SerializedSingleton::getInstance();
    s1->doWork();
    SerializedSingleton* s2 = SerializedSingleton::getInstance();
    std::cout << "Same instance? " << (s1 == s2 ? "Yes" : "No") <<
    std::endl << std::endl;

```

```

// Bypassing Singleton
std::cout << "--- Bypassing Singleton Example ---" << std::endl;
BypassableSingleton* b1 = BypassableSingleton::getInstance();

// Bypass using a 'friend' class
BypassableSingleton* b2 = SingletonHacker::createNewInstance();

std::cout << "Same instance? " << (b1 == b2 ? "Yes" : "No (Bypass
Successful!)" << std::endl;

std::cout << "Ways to bypass in C++:" << std::endl;
std::cout << "1. Friend classes/functions (as shown above)." <<
std::endl;
std::cout << "2. Pointer hacking (casting or layout assumptions)." <<
std::endl;
std::cout << "3. Serialization (manually rebuilding state into a new
object)." << std::endl;

return 0;
}

```

Factory Method

Description: Defines an interface for creating an object, but **lets subclasses decide** which class to instantiate. The parent class defers object creation to its child classes.

- **Why it is used:** To promote loose coupling by eliminating the need to bind application-specific classes into your code. The creation logic is centralized and easy to extend.
- **Where it is commonly used:** UI frameworks (creating different buttons for different OS), plugin systems, document generators.

Code Example (CPP)

```

/*
 * FACTORY METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Factory Method pattern defines an interface for creating an object
 * but lets subclasses decide which class to instantiate.
 * It delegates the instantiation logic to child classes.
 *
 * WHY DOES IT EXIST?
 * - Decouples the creation of objects from their usage
 * - Allows a class to defer instantiation to subclasses
 * - Provides a way to extend the creation logic without modifying existing
 * code (Open/Closed Principle)
 * - Eliminates the need to bind application-specific classes into your
 * code
 */

```

```

* WHEN TO USE IT?
* - When a class can't anticipate the class of objects it must create
* - When a class wants its subclasses to specify the objects it creates
* - When classes delegate responsibility to one of several helper
subclasses, and you want to localize the knowledge of which helper subclass
is the delegate
* - Common use cases: document creation, UI components, payment
processing, logging frameworks
*
* PROS:
* - Avoids tight coupling between the creator and the concrete products
* - Follows Open/Closed Principle (easy to introduce new products without
changing existing code)
* - Single Responsibility Principle (product creation code is in one
place)
* - Can be combined with other patterns (e.g., Singleton, Prototype)
*
* CONS:
* - Can make the code more complex by introducing many subclasses
* - May require creating a parallel inheritance hierarchy for creators and
products
* - Can lead to excessive use of inheritance
*
* REAL-WORLD ANALOGY:
* Think of a car manufacturing plant. The plant (factory) knows how to
build cars, but it doesn't know what specific type of car to build until an
order comes in. The plant has a method to build cars, but the specific car
model (sedan, SUV, truck) is determined by the order or by specialized
assembly lines (subclasses).
*/

```

```

#include <iostream>
#include <memory>
#include <string>

// Product interface - defines the common interface for all products
class Document {
public:
    virtual ~Document() = default;

    // Common operation that all documents must implement
    virtual void open() = 0;
    virtual void close() = 0;
    virtual void save() = 0;

    // Type identification
    virtual std::string getType() const = 0;
};

// Concrete Product 1: Text Document
class TextDocument : public Document {
public:
    void open() override {
        std::cout << "Opening Text Document in text editor..." <<

```

```
std::endl;
}

void close() override {
    std::cout << "Closing Text Document..." << std::endl;
}

void save() override {
    std::cout << "Saving Text Document as .txt file..." << std::endl;
}

std::string getType() const override {
    return "Text Document";
}
};

// Concrete Product 2: PDF Document
class PDFDocument : public Document {
public:
    void open() override {
        std::cout << "Opening PDF Document in PDF viewer..." << std::endl;
    }

    void close() override {
        std::cout << "Closing PDF Document..." << std::endl;
    }

    void save() override {
        std::cout << "Saving PDF Document as .pdf file..." << std::endl;
    }

    std::string getType() const override {
        return "PDF Document";
    }
};

// Concrete Product 3: HTML Document
class HTMLDocument : public Document {
public:
    void open() override {
        std::cout << "Opening HTML Document in web browser..." <<
std::endl;
    }

    void close() override {
        std::cout << "Closing HTML Document..." << std::endl;
    }

    void save() override {
        std::cout << "Saving HTML Document as .html file..." << std::endl;
    }

    std::string getType() const override {
        return "HTML Document";
    }
};
```

```

    }
};

// Creator abstract class - declares the factory method
class Application {
protected:
    // Factory method - to be implemented by subclasses
    // This is the core of the pattern: subclasses decide what to create
    virtual std::unique_ptr<Document> createDocument() = 0;

public:
    virtual ~Application() = default;

    // Business logic that uses the product
    // This method doesn't need to know which specific document is created
    void newDocument() {
        // Use the factory method to create a document
        auto doc = createDocument();
        std::cout << "Created: " << doc->getType() << std::endl;
        doc->open();
        doc->save();
        doc->close();
    }
};

// Concrete Creator 1: Text Editor Application
class TextEditor : public Application {
protected:
    std::unique_ptr<Document> createDocument() override {
        return std::make_unique<TextDocument>();
    }
};

// Concrete Creator 2: PDF Viewer Application
class PDFViewer : public Application {
protected:
    std::unique_ptr<Document> createDocument() override {
        return std::make_unique<PDFDocument>();
    }
};

// Concrete Creator 3: Web Browser Application
class WebBrowser : public Application {
protected:
    std::unique_ptr<Document> createDocument() override {
        return std::make_unique<HTMLDocument>();
    }
};

// Alternative implementation: Parameterized Factory Method
// This allows a single creator to create multiple types of products
class DocumentFactory {
public:
    enum class DocumentType {

```

```

        TEXT,
        PDF,
        HTML
    };

    static std::unique_ptr<Document> createDocument(DocumentType type) {
        switch (type) {
            case DocumentType::TEXT:
                return std::make_unique<TextDocument>();
            case DocumentType::PDF:
                return std::make_unique<PDFDocument>();
            case DocumentType::HTML:
                return std::make_unique<HTMLDocument>();
            default:
                throw std::invalid_argument("Unknown document type");
        }
    }
};

int main() {
    std::cout << "=== Factory Method Pattern Demo ===" << std::endl <<
    std::endl;

    // Using inheritance-based Factory Method
    std::cout << "--- Inheritance-based Factory Method ---" << std::endl;

    std::unique_ptr<Application> textEditor = std::make_unique<TextEditor>
    ();
    textEditor->newDocument();
    std::cout << std::endl;

    std::unique_ptr<Application> pdfViewer = std::make_unique<PDFViewer>();
    pdfViewer->newDocument();
    std::cout << std::endl;

    std::unique_ptr<Application> webBrowser = std::make_unique<WebBrowser>
    ();
    webBrowser->newDocument();
    std::cout << std::endl;

    // Using parameterized Factory Method
    std::cout << "--- Parameterized Factory Method ---" << std::endl;

    auto textDoc =
    DocumentFactory::createDocument(DocumentFactory::DocumentType::TEXT);
    textDoc->open();
    std::cout << std::endl;

    auto pdfDoc =
    DocumentFactory::createDocument(DocumentFactory::DocumentType::PDF);
    pdfDoc->open();
    std::cout << std::endl;

    auto htmlDoc =

```

```

DocumentFactory::createDocument (DocumentFactory::DocumentType::HTML) ;
    htmlDoc->open() ;

    return 0;
}

```

Abstract Factory

Description: Provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

- **Why it is used:** To ensure that a set of related objects are used together without mixing incompatible combinations. It's like a factory of factories.
- **Where it is commonly used:** Cross-platform UI toolkits (e.g., Windows vs. macOS components), database drivers, theme systems.

Code Example (CPP)

```

/*
 * ABSTRACT FACTORY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Abstract Factory pattern provides an interface for creating families
of related or dependent objects without specifying their concrete classes.
 * It's like a factory of factories.
 *
 * WHY DOES IT EXIST?
 * - When you need to create families of related objects (e.g., UI
components for different operating systems)
 * - When you want to ensure that created objects are compatible with each
other
 * - When you want to provide a library of products without exposing their
implementation details
 * - When you need to support multiple themes or platforms
 *
 * WHEN TO USE IT?
 * - When a system should be independent of how its products are created,
composed, and represented
 * - When a system should be configured with one of multiple families of
products
 * - When the family of related product objects is designed to be used
together, and you need to enforce this constraint
 * - When you want to provide a class library of products, and you want to
reveal just their interfaces, not their implementations
 * - Common use cases: cross-platform UI components, database connectors
for different DBMS, game development (different themes)
 *
 * PROS:
 * - Ensures that products from the same family are compatible
 * - Hides the concrete classes from the client code

```

```

* - Easy to introduce new product families without modifying existing code
(Open/Closed Principle)
* - Promotes consistency among products
*
* CONS:
* - Can be complex to implement due to many interfaces and classes
* - Adding new products to existing families requires modifying all
factory interfaces
* - Can lead to excessive abstraction and indirection
* - May be overkill for simple scenarios
*
* REAL-WORLD ANALOGY:
* Think of a furniture factory that produces different styles of furniture
(Modern, Victorian, Rustic). Each style has its own family of products
(chairs, tables, sofas). If you order a Modern chair, you want it to match
with a Modern table. The abstract factory ensures that all furniture pieces
you order belong to the same style family.
*/

```

```

#include <iostream>
#include <memory>
#include <string>

```

```

// Abstract Product A: Button interface
class Button {
public:
    virtual ~Button() = default;
    virtual void click() = 0;
    virtual void render() = 0;
    virtual std::string getName() const = 0;
};

```

```

// Abstract Product B: Checkbox interface
class Checkbox {
public:
    virtual ~Checkbox() = default;
    virtual void check() = 0;
    virtual void render() = 0;
    virtual std::string getName() const = 0;
};

```

```

// Concrete Product A1: Windows Button
class WindowsButton : public Button {
public:
    void click() override {
        std::cout << "Windows button clicked!" << std::endl;
    }

    void render() override {
        std::cout << "Rendering Windows-style button..." << std::endl;
    }

    std::string getName() const override {
        return "Windows Button";
    }
}

```

```
    }  
};  
  
// Concrete Product B1: Windows Checkbox  
class WindowsCheckbox : public Checkbox {  
public:  
    void check() override {  
        std::cout << "Windows checkbox checked!" << std::endl;  
    }  
  
    void render() override {  
        std::cout << "Rendering Windows-style checkbox..." << std::endl;  
    }  
  
    std::string getName() const override {  
        return "Windows Checkbox";  
    }  
};  
  
// Concrete Product A2: Mac Button  
class MacButton : public Button {  
public:  
    void click() override {  
        std::cout << "Mac button clicked!" << std::endl;  
    }  
  
    void render() override {  
        std::cout << "Rendering Mac-style button..." << std::endl;  
    }  
  
    std::string getName() const override {  
        return "Mac Button";  
    }  
};  
  
// Concrete Product B2: Mac Checkbox  
class MacCheckbox : public Checkbox {  
public:  
    void check() override {  
        std::cout << "Mac checkbox checked!" << std::endl;  
    }  
  
    void render() override {  
        std::cout << "Rendering Mac-style checkbox..." << std::endl;  
    }  
  
    std::string getName() const override {  
        return "Mac Checkbox";  
    }  
};  
  
// Abstract Factory interface - declares methods for creating each abstract  
product  
class GUIFactory {
```

```

public:
    virtual ~GUIFactory() = default;
    virtual std::unique_ptr<Button> createButton() = 0;
    virtual std::unique_ptr<Checkbox> createCheckbox() = 0;
};

// Concrete Factory 1: Windows Factory
// Creates Windows-specific UI components
class WindowsFactory : public GUIFactory {
public:
    std::unique_ptr<Button> createButton() override {
        return std::make_unique<WindowsButton>();
    }

    std::unique_ptr<Checkbox> createCheckbox() override {
        return std::make_unique<WindowsCheckbox>();
    }
};

// Concrete Factory 2: Mac Factory
// Creates Mac-specific UI components
class MacFactory : public GUIFactory {
public:
    std::unique_ptr<Button> createButton() override {
        return std::make_unique<MacButton>();
    }

    std::unique_ptr<Checkbox> createCheckbox() override {
        return std::make_unique<MacCheckbox>();
    }
};

// Client class that uses the abstract factory
// The client doesn't know which concrete factory is being used
class Application {
private:
    std::unique_ptr<Button> button;
    std::unique_ptr<Checkbox> checkbox;

public:
    // Constructor takes a factory and creates UI components
    Application(std::unique_ptr<GUIFactory> factory) {
        button = factory->createButton();
        checkbox = factory->createCheckbox();
    }

    // Business logic that uses the products
    void renderUI() {
        std::cout << "Rendering UI components..." << std::endl;
        std::cout << "Button: " << button->getName() << std::endl;
        std::cout << "Checkbox: " << checkbox->getName() << std::endl;

        button->render();
        checkbox->render();
    }
};

```

```

    }

    void interact() {
        std::cout << "\nUser interaction..." << std::endl;
        button->click();
        checkbox->check();
    }
};

// Another example: Database Abstract Factory
// Shows how Abstract Factory can be used for database connections

// Abstract Product: Database Connection
class DatabaseConnection {
public:
    virtual ~DatabaseConnection() = default;
    virtual void connect() = 0;
    virtual void disconnect() = 0;
    virtual void executeQuery(const std::string& query) = 0;
};

// Abstract Product: Database Command
class DatabaseCommand {
public:
    virtual ~DatabaseCommand() = default;
    virtual void execute() = 0;
};

// Concrete Products: MySQL
class MySQLConnection : public DatabaseConnection {
public:
    void connect() override {
        std::cout << "Connecting to MySQL database..." << std::endl;
    }

    void disconnect() override {
        std::cout << "Disconnecting from MySQL database..." << std::endl;
    }

    void executeQuery(const std::string& query) override {
        std::cout << "Executing MySQL query: " << query << std::endl;
    }
};

class MySQLCommand : public DatabaseCommand {
private:
    std::string command;

public:
    MySQLCommand(const std::string& cmd) : command(cmd) {}

    void execute() override {
        std::cout << "Executing MySQL command: " << command << std::endl;
    }
}

```

```

};

// Concrete Products: PostgreSQL
class PostgreSQLConnection : public DatabaseConnection {
public:
    void connect() override {
        std::cout << "Connecting to PostgreSQL database..." << std::endl;
    }

    void disconnect() override {
        std::cout << "Disconnecting from PostgreSQL database..." <<
std::endl;
    }

    void executeQuery(const std::string& query) override {
        std::cout << "Executing PostgreSQL query: " << query << std::endl;
    }
};

class PostgreSQLCommand : public DatabaseCommand {
private:
    std::string command;

public:
    PostgreSQLCommand(const std::string& cmd) : command(cmd) {}

    void execute() override {
        std::cout << "Executing PostgreSQL command: " << command <<
std::endl;
    }
};

// Abstract Factory for Database
class DatabaseFactory {
public:
    virtual ~DatabaseFactory() = default;
    virtual std::unique_ptr<DatabaseConnection> createConnection() = 0;
    virtual std::unique_ptr<DatabaseCommand> createCommand(const
std::string& cmd) = 0;
};

// Concrete Factory: MySQL
class MySQLFactory : public DatabaseFactory {
public:
    std::unique_ptr<DatabaseConnection> createConnection() override {
        return std::make_unique<MySQLConnection>();
    }

    std::unique_ptr<DatabaseCommand> createCommand(const std::string& cmd)
override {
        return std::make_unique<MySQLCommand>(cmd);
    }
};

```

```
// Concrete Factory: PostgreSQL
class PostgreSQLFactory : public DatabaseFactory {
public:
    std::unique_ptr<DatabaseConnection> createConnection() override {
        return std::make_unique<PostgreSQLConnection>();
    }

    std::unique_ptr<DatabaseCommand> createCommand(const std::string& cmd)
override {
        return std::make_unique<PostgreSQLCommand>(cmd);
    }
};

int main() {
    std::cout << "=== Abstract Factory Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: GUI Components
    std::cout << "--- GUI Components Example ---" << std::endl;

    // Create Windows UI
    std::cout << "\nCreating Windows UI:" << std::endl;
    auto windowsFactory = std::make_unique<WindowsFactory>();
    Application windowsApp(std::move(windowsFactory));
    windowsApp.renderUI();
    windowsApp.interact();

    // Create Mac UI
    std::cout << "\nCreating Mac UI:" << std::endl;
    auto macFactory = std::make_unique<MacFactory>();
    Application macApp(std::move(macFactory));
    macApp.renderUI();
    macApp.interact();

    // Example 2: Database Connections
    std::cout << "\n\n--- Database Connections Example ---" << std::endl;

    // MySQL
    std::cout << "\nUsing MySQL:" << std::endl;
    auto mysqlFactory = std::make_unique<MySQLFactory>();
    auto mysqlConnection = mysqlFactory->createConnection();
    auto mysqlCommand = mysqlFactory->createCommand("CREATE TABLE users (id
INT)");

    mysqlConnection->connect();
    mysqlConnection->executeQuery("SELECT * FROM users");
    mysqlCommand->execute();
    mysqlConnection->disconnect();

    // PostgreSQL
    std::cout << "\nUsing PostgreSQL:" << std::endl;
    auto postgresFactory = std::make_unique<PostgreSQLFactory>();
    auto postgresConnection = postgresFactory->createConnection();
    auto postgresCommand = postgresFactory->createCommand("CREATE TABLE
```

```
products (id INT)");

    postgresConnection->connect();
    postgresConnection->executeQuery("SELECT * FROM products");
    postgresCommand->execute();
    postgresConnection->disconnect();

    return 0;
}
```

Builder

Description: Separates the **construction of a complex object** from its representation, allowing the same construction process to create different representations.

- **Why it is used:** To construct objects step-by-step, especially when an object requires many configuration options or when different representations of a product are needed.
- **Where it is commonly used:** Building complex query objects, constructing HTML/XML documents, configuring network requests.

Code Example (CPP)

```
/*
 * BUILDER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Builder pattern separates the construction of a complex object from
its representation, allowing the same construction process to create
different representations.
 * It constructs complex objects step by step.
 *
 * WHY DOES IT EXIST?
 * - When you need to create complex objects with many optional parameters
 * - When you want to separate the construction logic from the object's
representation
 * - When you need to create objects in a step-by-step fashion
 * - When you want to provide different representations for the same
construction process
 *
 * WHEN TO USE IT?
 * - When the algorithm for creating a complex object should be independent
of the parts that make up the object and how they're assembled
 * - When the construction process must allow different representations for
the object that's constructed
 * - When you want to construct objects with many optional parameters
without using a telescoping constructor
 * - Common use cases: building complex documents, SQL query builders,
HTML/XML generators, configuration objects
 *
 * PROS:
```

```

* - Allows you to construct complex objects step by step
* - Provides fine-grained control over the construction process
* - Can reuse the same construction process to create different
representations
* - Single Responsibility Principle (complex construction code is isolated
from business logic)
* - More readable than telescoping constructors
*
* CONS:
* - Increases overall code complexity
* - Requires creating a separate Builder class for each product
* - May not be necessary for simple objects
* - Can lead to code duplication if not carefully designed
*
* REAL-WORLD ANALOGY:
* Think of ordering a custom pizza. You don't just say "give me a pizza" -
you specify the crust, sauce, cheese, and toppings step by step. The pizza
builder (chef) takes your specifications and constructs the pizza. You can
use the same process to create different types of pizzas (vegetarian, meat
lover's, etc.) by choosing different options at each step.
*/

```

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>

// Product class - the complex object being built
class House {
private:
    std::string foundation;
    std::string structure;
    std::string roof;
    std::string interior;
    bool hasGarage;
    bool hasGarden;
    bool hasSwimmingPool;

public:
    House() : hasGarage(false), hasGarden(false), hasSwimmingPool(false) {}

    // Setter methods for builder to use
    void setFoundation(const std::string& f) { foundation = f; }
    void setStructure(const std::string& s) { structure = s; }
    void setRoof(const std::string& r) { roof = r; }
    void setInterior(const std::string& i) { interior = i; }
    void setGarage(bool g) { hasGarage = g; }
    void setGarden(bool g) { hasGarden = g; }
    void setSwimmingPool(bool s) { hasSwimmingPool = s; }

    // Display the house details
    void display() const {
        std::cout << "=== House Details ===" << std::endl;
        std::cout << "Foundation: " << foundation << std::endl;
    }
};

```

```

        std::cout << "Structure: " << structure << std::endl;
        std::cout << "Roof: " << roof << std::endl;
        std::cout << "Interior: " << interior << std::endl;
        std::cout << "Garage: " << (hasGarage ? "Yes" : "No") << std::endl;
        std::cout << "Garden: " << (hasGarden ? "Yes" : "No") << std::endl;
        std::cout << "Swimming Pool: " << (hasSwimmingPool ? "Yes" : "No")
<< std::endl;
        std::cout << "======" << std::endl;
    }
};

// Builder interface - defines the steps to build the product
class HouseBuilder {
public:
    virtual ~HouseBuilder() = default;

    virtual void buildFoundation() = 0;
    virtual void buildStructure() = 0;
    virtual void buildRoof() = 0;
    virtual void buildInterior() = 0;
    virtual void buildGarage() = 0;
    virtual void buildGarden() = 0;
    virtual void buildSwimmingPool() = 0;

    virtual House getResult() = 0;
};

// Concrete Builder 1: Simple House Builder
class SimpleHouseBuilder : public HouseBuilder {
private:
    House house;

public:
    void buildFoundation() override {
        house.setFoundation("Concrete foundation");
    }

    void buildStructure() override {
        house.setStructure("Wood frame structure");
    }

    void buildRoof() override {
        house.setRoof("Asphalt shingle roof");
    }

    void buildInterior() override {
        house.setInterior("Basic interior with drywall");
    }

    void buildGarage() override {
        house.setGarage(false);
    }

    void buildGarden() override {

```

```
        house.setGarden(false);
    }

    void buildSwimmingPool() override {
        house.setSwimmingPool(false);
    }

    House getResult() override {
        return house;
    }
};

// Concrete Builder 2: Luxury House Builder
class LuxuryHouseBuilder : public HouseBuilder {
private:
    House house;

public:
    void buildFoundation() override {
        house.setFoundation("Reinforced concrete foundation with
basement");
    }

    void buildStructure() override {
        house.setStructure("Steel frame structure with brick exterior");
    }

    void buildRoof() override {
        house.setRoof("Spanish tile roof with solar panels");
    }

    void buildInterior() override {
        house.setInterior("Luxury interior with marble floors and custom
cabinetry");
    }

    void buildGarage() override {
        house.setGarage(true);
    }

    void buildGarden() override {
        house.setGarden(true);
    }

    void buildSwimmingPool() override {
        house.setSwimmingPool(true);
    }

    House getResult() override {
        return house;
    }
};

// Director class - orchestrates the construction process
```

```
// The director knows the construction algorithm but doesn't know the
details
class ConstructionDirector {
private:
    HouseBuilder* builder;

public:
    void setBuilder(HouseBuilder* b) {
        builder = b;
    }

    // Construct a basic house
    void buildBasicHouse() {
        builder->buildFoundation();
        builder->buildStructure();
        builder->buildRoof();
        builder->buildInterior();
    }

    // Construct a full-featured house
    void buildFullHouse() {
        builder->buildFoundation();
        builder->buildStructure();
        builder->buildRoof();
        builder->buildInterior();
        builder->buildGarage();
        builder->buildGarden();
        builder->buildSwimmingPool();
    }

    // Construct a custom house
    void buildCustomHouse(bool withGarage, bool withGarden, bool withPool)
    {
        builder->buildFoundation();
        builder->buildStructure();
        builder->buildRoof();
        builder->buildInterior();
        if (withGarage) builder->buildGarage();
        if (withGarden) builder->buildGarden();
        if (withPool) builder->buildSwimmingPool();
    }
};

// Alternative: Fluent Builder pattern (method chaining)
// This is a common variation where the builder returns itself for chaining
class Computer {
private:
    std::string cpu;
    std::string gpu;
    int ram;
    int storage;
    bool hasWifi;
    bool hasBluetooth;
```

```
public:
    Computer() : ram(8), storage(256), hasWifi(false), hasBluetooth(false)
    {}

    void setCpu(const std::string& c) { cpu = c; }
    void setGpu(const std::string& g) { gpu = g; }
    void setRam(int r) { ram = r; }
    void setStorage(int s) { storage = s; }
    void setWifi(bool w) { hasWifi = w; }
    void setBluetooth(bool b) { hasBluetooth = b; }

    void display() const {
        std::cout << "=== Computer Specs ===" << std::endl;
        std::cout << "CPU: " << cpu << std::endl;
        std::cout << "GPU: " << gpu << std::endl;
        std::cout << "RAM: " << ram << " GB" << std::endl;
        std::cout << "Storage: " << storage << " GB" << std::endl;
        std::cout << "WiFi: " << (hasWifi ? "Yes" : "No") << std::endl;
        std::cout << "Bluetooth: " << (hasBluetooth ? "Yes" : "No") <<
std::endl;
        std::cout << "===== " << std::endl;
    }
};

class ComputerBuilder {
private:
    Computer computer;

public:
    ComputerBuilder& setCpu(const std::string& cpu) {
        computer.setCpu(cpu);
        return *this;
    }

    ComputerBuilder& setGpu(const std::string& gpu) {
        computer.setGpu(gpu);
        return *this;
    }

    ComputerBuilder& setRam(int ram) {
        computer.setRam(ram);
        return *this;
    }

    ComputerBuilder& setStorage(int storage) {
        computer.setStorage(storage);
        return *this;
    }

    ComputerBuilder& addWifi() {
        computer.setWifi(true);
        return *this;
    }
}
```

```

    ComputerBuilder& addBluetooth() {
        computer.setBluetooth(true);
        return *this;
    }

    Computer build() {
        return computer;
    }
};

int main() {
    std::cout << "=== Builder Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Classic Builder with Director
    std::cout << "--- Classic Builder with Director ---" << std::endl;

    ConstructionDirector director;

    // Build a simple house
    std::cout << "\nBuilding Simple House:" << std::endl;
    SimpleHouseBuilder simpleBuilder;
    director.setBuilder(&simpleBuilder);
    director.buildBasicHouse();
    House simpleHouse = simpleBuilder.getResult();
    simpleHouse.display();

    // Build a luxury house
    std::cout << "\nBuilding Luxury House:" << std::endl;
    LuxuryHouseBuilder luxuryBuilder;
    director.setBuilder(&luxuryBuilder);
    director.buildFullHouse();
    House luxuryHouse = luxuryBuilder.getResult();
    luxuryHouse.display();

    // Build a custom house
    std::cout << "\nBuilding Custom House (with garage and garden, no
pool):" << std::endl;
    SimpleHouseBuilder customBuilder;
    director.setBuilder(&customBuilder);
    director.buildCustomHouse(true, true, false);
    House customHouse = customBuilder.getResult();
    customHouse.display();

    // Example 2: Fluent Builder (method chaining)
    std::cout << "\n--- Fluent Builder (Method Chaining) ---" << std::endl;

    std::cout << "\nBuilding Gaming Computer:" << std::endl;
    Computer gamingComputer = ComputerBuilder()
        .setCpu("Intel Core i9-13900K")
        .setGpu("NVIDIA RTX 4090")
        .setRam(32)
        .setStorage(1000)
        .addWifi()
        .addBluetooth()

```

```

        .build();
    gamingComputer.display();

    std::cout << "\nBuilding Office Computer:" << std::endl;
    Computer officeComputer = ComputerBuilder()
        .setCpu("Intel Core i5-13400")
        .setGpu("Integrated Graphics")
        .setRam(16)
        .setStorage(512)
        .addWifi()
        .build();
    officeComputer.display();

    return 0;
}

```

Prototype

Description: Creates new objects by **copying (cloning) an existing object**, rather than creating from scratch.

- **Why it is used:** When object creation is expensive or complex, cloning an existing object is faster and simpler. It avoids the overhead of re-initialization.
- **Where it is commonly used:** Game development (duplicating characters or levels), graphic editors (duplicating shapes), document templates.

Code Example (CPP)

```

/*
 * PROTOTYPE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Prototype pattern specifies the kinds of objects to create using a
 * prototypical instance, and creates new objects by copying this prototype.
 * It allows you to create new objects by cloning existing ones instead of
 * creating them from scratch.
 *
 * WHY DOES IT EXIST?
 * - When creating an object is expensive or complex
 * - When you want to avoid creating subclasses just to create objects
 * - When you want to hide the concrete classes from the client
 * - When you need to create objects at runtime that are similar to
 * existing objects
 *
 * WHEN TO USE IT?
 * - When the classes to instantiate are specified at runtime
 * - When you want to avoid building a class hierarchy of factories that
 * parallels the class hierarchy of products
 * - When instances of a class can have one of only a few different
 * combinations of state
 * - When creating an object is expensive (e.g., requires database access,

```

```

network I/O, or complex initialization)
* - Common use cases: game objects, document templates, graphic shapes,
configuration objects
*
* PROS:
* - Allows you to clone objects without coupling to their concrete classes
* - Can avoid repetitive initialization code
* - Can be more efficient than creating objects from scratch
* - Lets you add or remove products at runtime
* - Works well with complex object graphs
*
* CONS:
* - Cloning complex objects with circular references can be tricky
* - Shallow vs. deep copying must be carefully considered
* - May be difficult to implement if the object's internal state is not
easily accessible
* - Can violate encapsulation if the clone method needs access to private
fields
*
* REAL-WORLD ANALOGY:
* Think of a cell division in biology. A cell (prototype) divides to
create an identical copy of itself. The new cell starts with the same DNA
and characteristics as the original, but can then evolve independently.
Similarly, in software, you can clone an object and then modify it as
needed.
*/

#include <iostream>
#include <string>
#include <memory>
#include <unordered_map>

// Prototype interface - declares the clone method
class Prototype {
public:
    virtual ~Prototype() = default;
    virtual std::unique_ptr<Prototype> clone() const = 0;
    virtual void display() const = 0;
    virtual std::string getType() const = 0;
};

// Concrete Prototype 1: Rectangle
class Rectangle : public Prototype {
private:
    int width;
    int height;
    std::string color;

public:
    Rectangle(int w, int h, const std::string& c)
        : width(w), height(h), color(c) {}

    // Copy constructor used for cloning
    Rectangle(const Rectangle& other)

```

```

        : width(other.width), height(other.height), color(other.color) {}

std::unique_ptr<Prototype> clone() const override {
    return std::make_unique<Rectangle>(*this); // Use copy constructor
}

void display() const override {
    std::cout << "Rectangle: " << width << "x" << height
                << ", Color: " << color << std::endl;
}

std::string getType() const override {
    return "Rectangle";
}

// Methods to modify the cloned object
void setWidth(int w) { width = w; }
void setHeight(int h) { height = h; }
void setColor(const std::string& c) { color = c; }
};

// Concrete Prototype 2: Circle
class Circle : public Prototype {
private:
    int radius;
    std::string color;

public:
    Circle(int r, const std::string& c) : radius(r), color(c) {}

    // Copy constructor used for cloning
    Circle(const Circle& other) : radius(other.radius), color(other.color)
    {}

    std::unique_ptr<Prototype> clone() const override {
        return std::make_unique<Circle>(*this); // Use copy constructor
    }

    void display() const override {
        std::cout << "Circle: Radius " << radius
                    << ", Color: " << color << std::endl;
    }

    std::string getType() const override {
        return "Circle";
    }

    void setRadius(int r) { radius = r; }
    void setColor(const std::string& c) { color = c; }
};

// Prototype Registry - manages a collection of prototypes
class PrototypeRegistry {
private:

```

```

        std::unordered_map<std::string, std::unique_ptr<Prototype>> prototypes;

public:
    void registerPrototype(const std::string& key,
        std::unique_ptr<Prototype> prototype) {
        prototypes[key] = std::move(prototype);
    }

    std::unique_ptr<Prototype> clone(const std::string& key) {
        auto it = prototypes.find(key);
        if (it != prototypes.end()) {
            return it->second->clone();
        }
        return nullptr;
    }
};

// Example: Complex object with deep copy
class Employee : public Prototype {
private:
    std::string name;
    std::string department;
    int salary;
    std::unique_ptr<std::string[]> skills; // Dynamic array
    int skillCount;

public:
    Employee(const std::string& n, const std::string& d, int s)
        : name(n), department(d), salary(s), skillCount(0) {}

    // Deep copy constructor
    Employee(const Employee& other)
        : name(other.name), department(other.department),
        salary(other.salary),
        skillCount(other.skillCount) {
        if (other.skillCount > 0) {
            skills = std::make_unique<std::string[]>(other.skillCount);
            for (int i = 0; i < other.skillCount; i++) {
                skills[i] = other.skills[i];
            }
        }
    }

    std::unique_ptr<Prototype> clone() const override {
        return std::make_unique<Employee>(*this);
    }

    void display() const override {
        std::cout << "Employee: " << name << ", Dept: " << department
            << ", Salary: $" << salary << std::endl;
        std::cout << "Skills: ";
        for (int i = 0; i < skillCount; i++) {
            std::cout << skills[i];
            if (i < skillCount - 1) std::cout << ", ";
        }
    }
};

```

```

    }
    std::cout << std::endl;
}

std::string getType() const override {
    return "Employee";
}

void addSkill(const std::string& skill) {
    auto newSkills = std::make_unique<std::string[]>(skillCount + 1);
    for (int i = 0; i < skillCount; i++) {
        newSkills[i] = skills[i];
    }
    newSkills[skillCount] = skill;
    skills = std::move(newSkills);
    skillCount++;
}

void setName(const std::string& n) { name = n; }
void setSalary(int s) { salary = s; }
};

// Example: Document with shallow vs deep copy
class Document : public Prototype {
private:
    std::string title;
    std::string* content; // Pointer to demonstrate shallow vs deep copy

public:
    Document(const std::string& t, const std::string& c)
        : title(t), content(new std::string(c)) {}

    // Deep copy constructor
    Document(const Document& other)
        : title(other.title), content(new std::string(*other.content)) {}

    ~Document() {
        delete content;
    }

    std::unique_ptr<Prototype> clone() const override {
        return std::make_unique<Document>(*this);
    }

    void display() const override {
        std::cout << "Document: " << title << std::endl;
        std::cout << "Content: " << *content << std::endl;
    }

    std::string getType() const override {
        return "Document";
    }

    void appendContent(const std::string& text) {

```

```

        *content += text;
    }

    void setTitle(const std::string& t) { title = t; }
};

int main() {
    std::cout << "=== Prototype Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: Basic Prototype with Registry
    std::cout << "--- Basic Prototype with Registry ---" << std::endl;

    PrototypeRegistry registry;

    // Register prototypes
    registry.registerPrototype("red_rectangle",
        std::make_unique<Rectangle>(10, 5, "red"));
    registry.registerPrototype("blue_circle",
        std::make_unique<Circle>(7, "blue"));

    // Clone and modify prototypes
    std::cout << "\nCloning red rectangle:" << std::endl;
    auto rect1 = registry.clone("red_rectangle");
    rect1->display();

    std::cout << "\nCloning and modifying red rectangle:" << std::endl;
    auto rect2 = registry.clone("red_rectangle");
    auto* rect2Ptr = dynamic_cast<Rectangle*>(rect2.get());
    if (rect2Ptr) {
        rect2Ptr->setWidth(20);
        rect2Ptr->setColor("green");
    }
    rect2->display();

    std::cout << "\nCloning blue circle:" << std::endl;
    auto circle1 = registry.clone("blue_circle");
    circle1->display();

    // Example 2: Complex object with deep copy
    std::cout << "\n--- Complex Object with Deep Copy ---" << std::endl;

    Employee originalEmployee("John Doe", "Engineering", 75000);
    originalEmployee.addSkill("C++");
    originalEmployee.addSkill("Python");
    originalEmployee.addSkill("JavaScript");

    std::cout << "\nOriginal employee:" << std::endl;
    originalEmployee.display();

    std::cout << "\nCloned employee:" << std::endl;
    auto clonedEmployee = std::unique_ptr<Employee>(
        dynamic_cast<Employee*>(originalEmployee.clone().release())
    );

```

```

        clonedEmployee->setName("Jane Smith");
        clonedEmployee->setSalary(80000);
        clonedEmployee->addSkill("Go");
        clonedEmployee->display();

        std::cout << "\nOriginal employee (unchanged):" << std::endl;
        originalEmployee.display();

        // Example 3: Document with deep copy
        std::cout << "\n--- Document with Deep Copy ---" << std::endl;

        Document originalDoc("Project Plan", "This document outlines the
project plan.");
        std::cout << "\nOriginal document:" << std::endl;
        originalDoc.display();

        std::cout << "\nCloned document:" << std::endl;
        auto clonedDoc = std::unique_ptr<Document>(
            dynamic_cast<Document*>(originalDoc.clone().release())
        );
        clonedDoc->setTitle("Project Plan - Copy");
        clonedDoc->appendContent(" It includes milestones and deliverables.");
        clonedDoc->display();

        std::cout << "\nOriginal document (unchanged):" << std::endl;
        originalDoc.display();

        return 0;
    }

```

2. Structural Design Patterns

Overview

Structural patterns deal with **how classes and objects are composed** to form larger structures. They simplify the structure by identifying relationships between entities and help ensure that when one part of a system changes, the entire structure does not need to change.

Adapter

Description: Acts as a **bridge between two incompatible interfaces**. It wraps an existing class with a new interface so that it becomes compatible with the client's expectations.

- **Why it is used:** To allow classes with incompatible interfaces to work together without modifying their source code.
- **Where it is commonly used:** Integrating third-party libraries, legacy code integration, data format conversion (e.g., XML to JSON).

Code Example (CPP)

```

/*
 * ADAPTER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Adapter pattern allows objects with incompatible interfaces to
collaborate. It acts as a bridge between two incompatible interfaces.
 *
 * WHY DOES IT EXIST?
 * - When you want to use an existing class whose interface doesn't match
the one you need
 * - When you want to create a reusable class that cooperates with
unrelated classes with incompatible interfaces
 * - When you need to use several existing subclasses, but adapting each to
match your interface would be impractical
 *
 * WHEN TO USE IT?
 * - When you want to use a third-party class but its interface doesn't
match your system's interface
 * - When you need to integrate new components into an existing system
without modifying existing code
 * - When you want to reuse existing classes that have similar
functionality but different interfaces
 * - Common use cases: integrating third-party libraries, legacy code
integration, data format conversion
 *
 * PROS:
 * - Single Responsibility Principle (separates conversion logic from
business logic)
 * - Open/Closed Principle (introduce new adapters without modifying
existing code)
 * - Allows incompatible classes to work together
 * - Improves code reusability
 *
 * CONS:
 * - Increases overall code complexity
 * - Can introduce additional layers of indirection
 * - May not be suitable for simple cases where direct modification is
easier
 * - Can make the code harder to understand due to multiple layers
 *
 * REAL-WORLD ANALOGY:
 * Think of a power adapter when traveling abroad. Your device (client)
expects a certain plug shape (target interface), but the wall outlet
(adaptee) has a different shape. The adapter converts between the two,
allowing your device to work with the foreign outlet without modifying
either the device or the wall.
 */

#include <iostream>
#include <string>
#include <memory>

```

```

// Target Interface - the interface that the client expects
class MediaPlayer {
public:
    virtual ~MediaPlayer() = default;
    virtual void play(const std::string& audioType, const std::string&
fileName) = 0;
};

// Adaptee - the class with an incompatible interface
class AdvancedMediaPlayer {
public:
    virtual ~AdvancedMediaPlayer() = default;

    void playVlc(const std::string& fileName) {
        std::cout << "Playing vlc file: " << fileName << std::endl;
    }

    void playMp4(const std::string& fileName) {
        std::cout << "Playing mp4 file: " << fileName << std::endl;
    }
};

// Adapter - makes the Adaptee's interface compatible with the Target
interface
class MediaAdapter : public MediaPlayer {
private:
    std::unique_ptr<AdvancedMediaPlayer> advancedMusicPlayer;

public:
    MediaAdapter() :
advancedMusicPlayer(std::make_unique<AdvancedMediaPlayer>()) {}

    void play(const std::string& audioType, const std::string& fileName)
override {
        if (audioType == "vlc") {
            advancedMusicPlayer->playVlc(fileName);
        } else if (audioType == "mp4") {
            advancedMusicPlayer->playMp4(fileName);
        } else {
            std::cout << "Invalid media. " << audioType << " format not
supported" << std::endl;
        }
    }
};

// Client - uses the Target interface
class AudioPlayer : public MediaPlayer {
private:
    std::unique_ptr<MediaAdapter> mediaAdapter;

public:
    void play(const std::string& audioType, const std::string& fileName)
override {
        // Built-in support for mp3

```

```

        if (audioType == "mp3") {
            std::cout << "Playing mp3 file: " << fileName << std::endl;
        }
        // MediaAdapter provides support for other formats
        else if (audioType == "vlc" || audioType == "mp4") {
            mediaAdapter = std::make_unique<MediaAdapter>();
            mediaAdapter->play(audioType, fileName);
        } else {
            std::cout << "Invalid media. " << audioType << " format not
supported" << std::endl;
        }
    }
};

// Example 2: Object Adapter vs Class Adapter
// This example shows a different scenario: payment processing

// Target Interface
class PaymentProcessor {
public:
    virtual ~PaymentProcessor() = default;
    virtual void processPayment(double amount) = 0;
};

// Adaptee - incompatible payment system
class StripePaymentSystem {
public:
    void makeStripePayment(double amount) {
        std::cout << "Processing $" << amount << " via Stripe" <<
std::endl;
    }
};

// Adaptee - another incompatible payment system
class PayPalPaymentSystem {
public:
    void sendPayPalRequest(double amount) {
        std::cout << "Processing $" << amount << " via PayPal" <<
std::endl;
    }
};

// Adapter for Stripe
class StripeAdapter : public PaymentProcessor {
private:
    StripePaymentSystem stripeSystem;

public:
    void processPayment(double amount) override {
        stripeSystem.makeStripePayment(amount);
    }
};

// Adapter for PayPal

```

```

class PayPalAdapter : public PaymentProcessor {
private:
    PayPalPaymentSystem payPalSystem;

public:
    void processPayment(double amount) override {
        payPalSystem.sendPayPalRequest(amount);
    }
};

// Example 3: Two-way Adapter
// Allows both interfaces to work with each other

// Interface A
class CelsiusTemperature {
public:
    virtual ~CelsiusTemperature() = default;
    virtual double getTemperatureInCelsius() = 0;
    virtual void setTemperatureInCelsius(double temp) = 0;
};

// Interface B
class FahrenheitTemperature {
public:
    virtual ~FahrenheitTemperature() = default;
    virtual double getTemperatureInFahrenheit() = 0;
    virtual void setTemperatureInFahrenheit(double temp) = 0;
};

// Two-way adapter
class TemperatureAdapter : public CelsiusTemperature, public
FahrenheitTemperature {
private:
    double temperatureInCelsius;

public:
    TemperatureAdapter(double temp = 0.0) : temperatureInCelsius(temp) {}

    // Celsius interface implementation
    double getTemperatureInCelsius() override {
        return temperatureInCelsius;
    }

    void setTemperatureInCelsius(double temp) override {
        temperatureInCelsius = temp;
    }

    // Fahrenheit interface implementation
    double getTemperatureInFahrenheit() override {
        return (temperatureInCelsius * 9.0 / 5.0) + 32.0;
    }

    void setTemperatureInFahrenheit(double temp) override {
        temperatureInCelsius = (temp - 32.0) * 5.0 / 9.0;
    }
}

```

```
    }  
};  
  
int main() {  
    std::cout << "=== Adapter Pattern Demo ===" << std::endl << std::endl;  
  
    // Example 1: Media Player Adapter  
    std::cout << "--- Media Player Adapter ---" << std::endl;  
  
    AudioPlayer audioPlayer;  
  
    audioPlayer.play("mp3", "beyond the horizon.mp3");  
    audioPlayer.play("mp4", "alone again.mp4");  
    audioPlayer.play("vlc", "far far away.vlc");  
    audioPlayer.play("avi", "mind me.avi");  
  
    std::cout << std::endl;  
  
    // Example 2: Payment Processing Adapter  
    std::cout << "--- Payment Processing Adapter ---" << std::endl;  
  
    std::unique_ptr<PaymentProcessor> stripeAdapter =  
std::make_unique<StripeAdapter>();  
    stripeAdapter->processPayment(100.50);  
  
    std::unique_ptr<PaymentProcessor> payPalAdapter =  
std::make_unique<PayPalAdapter>();  
    payPalAdapter->processPayment(75.25);  
  
    std::cout << std::endl;  
  
    // Example 3: Two-way Temperature Adapter  
    std::cout << "--- Two-way Temperature Adapter ---" << std::endl;  
  
    TemperatureAdapter tempAdapter;  
  
    // Set using Celsius interface  
    tempAdapter.setTemperatureInCelsius(25.0);  
    std::cout << "Temperature in Celsius: " <<  
tempAdapter.getTemperatureInCelsius() << "°C" << std::endl;  
    std::cout << "Temperature in Fahrenheit: " <<  
tempAdapter.getTemperatureInFahrenheit() << "°F" << std::endl;  
  
    // Set using Fahrenheit interface  
    tempAdapter.setTemperatureInFahrenheit(77.0);  
    std::cout << "\nTemperature in Celsius: " <<  
tempAdapter.getTemperatureInCelsius() << "°C" << std::endl;  
    std::cout << "Temperature in Fahrenheit: " <<  
tempAdapter.getTemperatureInFahrenheit() << "°F" << std::endl;  
  
    return 0;  
}
```

Bridge

Description: Decouples an abstraction from its implementation so that the two can vary independently.

- **Why it is used:** To avoid a permanent binding between an abstraction and its implementation and to allow both to be extended independently.
 - **Where it is commonly used:** Device driver systems, rendering engines (OpenGL vs. DirectX), platform-independent UI frameworks.
-

Code Example (CPP)

```
/*
 * BRIDGE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Bridge pattern separates abstraction from implementation so that
both can vary independently.
 * It uses composition instead of inheritance to decouple the abstraction
from its implementation.
 *
 * WHY DOES IT EXIST?
 * - When you want to avoid a permanent binding between an abstraction and
its implementation
 * - When both the abstraction and the implementation should be extensible
by subclassing
 * - When changes in the implementation should not affect clients
 * - When you have a class hierarchy that explodes due to combining
multiple dimensions
 *
 * WHEN TO USE IT?
 * - When you want to split a monolithic class that has several variants of
some functionality
 * - When you need to extend a class in several independent dimensions
 * - When you want to be able to switch implementations at runtime
 * - Common use cases: UI frameworks (different controls on different
platforms), shape rendering, device drivers
 *
 * PROS:
 * - Separates abstraction from implementation
 * - Improves extensibility (can extend abstractions and implementations
independently)
 * - Hides implementation details from clients
 * - Reduces the number of subclasses needed (avoids combinatorial
explosion)
 *
 * CONS:
 * - Increases code complexity
 * - May require additional indirection
 * - Can make the code harder to understand initially
 * - Might be overkill for simple scenarios
 */
```

```

* REAL-WORLD ANALOGY:
* Think of a TV remote control (abstraction) and the TV itself
(implementation). The remote control doesn't need to know the internal
workings of the TV. You can use the same remote with different TV brands,
and different remotes with the same TV. The remote (abstraction) is bridged
to the TV (implementation) through a standard interface (infrared signals).
*/

#include <iostream>
#include <string>
#include <memory>

// Implementation Interface - defines the operations for all concrete
implementations
class DrawingAPI {
public:
    virtual ~DrawingAPI() = default;
    virtual void drawCircle(double x, double y, double radius) = 0;
    virtual void drawRectangle(double x, double y, double width, double
height) = 0;
};

// Concrete Implementation 1: Red Circle API
class RedCircleAPI : public DrawingAPI {
public:
    void drawCircle(double x, double y, double radius) override {
        std::cout << "Drawing Circle [color: red, center: (" << x << ", "
<< y
                << "), radius: " << radius << "]" << std::endl;
    }

    void drawRectangle(double x, double y, double width, double height)
override {
        std::cout << "Drawing Rectangle [color: red, top-left: (" << x <<
", " << y
                << "), width: " << width << ", height: " << height << "]"
<< std::endl;
    }
};

// Concrete Implementation 2: Green Circle API
class GreenCircleAPI : public DrawingAPI {
public:
    void drawCircle(double x, double y, double radius) override {
        std::cout << "Drawing Circle [color: green, center: (" << x << ", "
<< y
                << "), radius: " << radius << "]" << std::endl;
    }

    void drawRectangle(double x, double y, double width, double height)
override {
        std::cout << "Drawing Rectangle [color: green, top-left: (" << x <<
", " << y
                << "), width: " << width << ", height: " << height << "]"

```

```

<< std::endl;
    }
};

// Abstraction - defines the abstraction's interface and maintains a
reference to an implementation
class Shape {
protected:
    std::unique_ptr<DrawingAPI> drawingAPI;

public:
    Shape(std::unique_ptr<DrawingAPI> api) : drawingAPI(std::move(api)) {}
    virtual ~Shape() = default;
    virtual void draw() = 0;
    virtual void resize(double percentage) = 0;
};

// Refined Abstraction 1: Circle
class Circle : public Shape {
private:
    double x, y, radius;

public:
    Circle(double x, double y, double radius, std::unique_ptr<DrawingAPI>
api)
        : Shape(std::move(api)), x(x), y(y), radius(radius) {}

    void draw() override {
        drawingAPI->drawCircle(x, y, radius);
    }

    void resize(double percentage) override {
        radius *= percentage / 100.0;
        std::cout << "Resizing circle to " << radius << " radius" <<
std::endl;
    }
};

// Refined Abstraction 2: Rectangle
class Rectangle : public Shape {
private:
    double x, y, width, height;

public:
    Rectangle(double x, double y, double width, double height,
std::unique_ptr<DrawingAPI> api)
        : Shape(std::move(api)), x(x), y(y), width(width), height(height)
{}

    void draw() override {
        drawingAPI->drawRectangle(x, y, width, height);
    }

    void resize(double percentage) override {

```

```
        width *= percentage / 100.0;
        height *= percentage / 100.0;
        std::cout << "Resizing rectangle to " << width << "x" << height <<
std::endl;
    }
};

// Example 2: Device and Remote Control Bridge
// This shows how the Bridge pattern can be used for device control

// Implementation Interface
class Device {
public:
    virtual ~Device() = default;
    virtual void turnOn() = 0;
    virtual void turnOff() = 0;
    virtual void setVolume(int volume) = 0;
    virtual int getVolume() = 0;
};

// Concrete Implementation 1: TV
class Television : public Device {
private:
    bool on = false;
    int volume = 10;

public:
    void turnOn() override {
        on = true;
        std::cout << "TV is now ON" << std::endl;
    }

    void turnOff() override {
        on = false;
        std::cout << "TV is now OFF" << std::endl;
    }

    void setVolume(int volume) override {
        if (on) {
            this->volume = volume;
            std::cout << "TV volume set to " << volume << std::endl;
        } else {
            std::cout << "Cannot set volume: TV is OFF" << std::endl;
        }
    }

    int getVolume() override {
        return volume;
    }
};

// Concrete Implementation 2: Radio
class Radio : public Device {
private:
```

```

    bool on = false;
    int volume = 5;

public:
    void turnOn() override {
        on = true;
        std::cout << "Radio is now ON" << std::endl;
    }

    void turnOff() override {
        on = false;
        std::cout << "Radio is now OFF" << std::endl;
    }

    void setVolume(int volume) override {
        if (on) {
            this->volume = volume;
            std::cout << "Radio volume set to " << volume << std::endl;
        } else {
            std::cout << "Cannot set volume: Radio is OFF" << std::endl;
        }
    }

    int getVolume() override {
        return volume;
    }
};

// Abstraction: Remote Control
class RemoteControl {
protected:
    std::unique_ptr<Device> device;

public:
    RemoteControl(std::unique_ptr<Device> device) :
    device(std::move(device)) {}
    virtual ~RemoteControl() = default;

    virtual void togglePower() {
        if (device->getVolume() >= 0) { // Simple check if device is "on"
            device->turnOff();
        } else {
            device->turnOn();
        }
    }

    virtual void volumeUp() {
        int currentVolume = device->getVolume();
        device->setVolume(currentVolume + 1);
    }

    virtual void volumeDown() {
        int currentVolume = device->getVolume();
        device->setVolume(currentVolume - 1);
    }
};

```

```

    }
};

// Refined Abstraction: Advanced Remote Control
class AdvancedRemoteControl : public RemoteControl {
public:
    AdvancedRemoteControl(std::unique_ptr<Device> device) :
    RemoteControl(std::move(device)) {}

    void mute() {
        std::cout << "Muting device" << std::endl;
        device->setVolume(0);
    }
};

int main() {
    std::cout << "=== Bridge Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Shape Drawing Bridge
    std::cout << "--- Shape Drawing Bridge ---" << std::endl;

    auto redCircle = std::make_unique<Circle>(100, 100, 10,
std::make_unique<RedCircleAPI>());
    auto greenCircle = std::make_unique<Circle>(100, 100, 10,
std::make_unique<GreenCircleAPI>());

    redCircle->draw();
    greenCircle->draw();

    auto redRectangle = std::make_unique<Rectangle>(50, 50, 20, 30,
std::make_unique<RedCircleAPI>());
    auto greenRectangle = std::make_unique<Rectangle>(50, 50, 20, 30,
std::make_unique<GreenCircleAPI>());

    redRectangle->draw();
    greenRectangle->draw();

    std::cout << "\nResizing shapes:" << std::endl;
    redCircle->resize(150);
    redCircle->draw();

    std::cout << std::endl;

    // Example 2: Device and Remote Control Bridge
    std::cout << "--- Device and Remote Control Bridge ---" << std::endl;

    auto tv = std::make_unique<Television>();
    auto tvRemote = std::make_unique<RemoteControl>(std::move(tv));

    std::cout << "\nControlling TV with basic remote:" << std::endl;
    tvRemote->togglePower();
    tvRemote->volumeUp();
    tvRemote->volumeUp();
    tvRemote->volumeDown();

```

```

    tvRemote->togglePower();

    auto radio = std::make_unique<Radio>();
    auto advancedRemote = std::make_unique<AdvancedRemoteControl>
(std::move(radio));

    std::cout << "\nControlling Radio with advanced remote:" << std::endl;
    advancedRemote->togglePower();
    advancedRemote->volumeUp();
    advancedRemote->mute();
    advancedRemote->togglePower();

    return 0;
}

```

Composite

Description: Composes objects into **tree structures to represent part-whole hierarchies**. It lets clients treat individual objects and compositions of objects uniformly.

- **Why it is used:** To simplify client code that deals with tree structures by treating individual items and groups in the same way.
- **Where it is commonly used:** File system structures (files and folders), UI component trees, organization hierarchies.

Code Example (CPP)

```

/*
 * COMPOSITE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Composite pattern lets you compose objects into tree structures and
work with these structures as if they were individual objects.
 * It treats individual objects and compositions of objects uniformly.
 *
 * WHY DOES IT EXIST?
 * - When you want to represent part-whole hierarchies of objects
 * - When you want clients to ignore the difference between compositions of
objects and individual objects
 * - When you want to treat individual objects and compositions uniformly
 *
 * WHEN TO USE IT?
 * - When you have a tree structure of objects (e.g., file systems,
organization charts, UI component hierarchies)
 * - When you want to simplify client code by treating individual and
composite objects the same way
 * - When the structure of your objects forms a recursive hierarchy
 * - Common use cases: file systems, graphics scenes, UI component trees,
organization structures
 *
 */

```

```

* PROS:
* - Simplifies client code (can treat complex and simple objects
uniformly)
* - Open/Closed Principle (easy to add new kinds of components)
* - Flexible tree structure (can build complex structures from simple
parts)
* - Makes it easier to add new types of components
*
* CONS:
* - Can make the design overly general (hard to restrict components)
* - May be difficult to restrict the types of components in a composite
* - Can make the code harder to understand due to recursion
* - Performance overhead for traversing large trees
*
* REAL-WORLD ANALOGY:
* Think of a file system. A file is a simple object (leaf), while a folder
is a composite that can contain both files and other folders. When you
delete a folder, you want to delete everything inside it recursively. The
file system treats files and folders uniformly in many operations (like
copying, moving, or calculating size).
*/

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <algorithm>

// Component Interface - declares the interface for objects in the
composition
class FileSystemComponent {
public:
    virtual ~FileSystemComponent() = default;
    virtual void display(const std::string& indent = "") = 0;
    virtual int getSize() = 0;
    virtual void add(std::shared_ptr<FileSystemComponent> component) {
        // Default implementation for leaf nodes
        std::cout << "Cannot add to a file" << std::endl;
    }
    virtual void remove(std::shared_ptr<FileSystemComponent> component) {
        // Default implementation for leaf nodes
        std::cout << "Cannot remove from a file" << std::endl;
    }
    virtual std::shared_ptr<FileSystemComponent> getChild(int index) {
        // Default implementation for leaf nodes
        return nullptr;
    }
};

// Leaf - represents individual objects in the composition
class File : public FileSystemComponent {
private:
    std::string name;
    int size;

```

```

public:
    File(const std::string& name, int size) : name(name), size(size) {}

    void display(const std::string& indent = "") override {
        std::cout << indent << "File: " << name << " (" << size << "
bytes)" << std::endl;
    }

    int getSize() override {
        return size;
    }
};

// Composite - represents groups of objects that can contain other
composites
class Directory : public FileSystemComponent {
private:
    std::string name;
    std::vector<std::shared_ptr<FileSystemComponent>> children;

public:
    Directory(const std::string& name) : name(name) {}

    void add(std::shared_ptr<FileSystemComponent> component) override {
        children.push_back(component);
    }

    void remove(std::shared_ptr<FileSystemComponent> component) override {
        children.erase(
            std::remove(children.begin(), children.end(), component),
            children.end()
        );
    }

    std::shared_ptr<FileSystemComponent> getChild(int index) override {
        if (index >= 0 && index < children.size()) {
            return children[index];
        }
        return nullptr;
    }

    void display(const std::string& indent = "") override {
        std::cout << indent << "Directory: " << name << " (" << getSize()
<< " bytes)" << std::endl;
        for (const auto& child : children) {
            child->display(indent + "  ");
        }
    }

    int getSize() override {
        int totalSize = 0;
        for (const auto& child : children) {
            totalSize += child->getSize();
        }
    }
};

```

```

        }
        return totalSize;
    }
};

// Example 2: Organization Structure
// Shows how Composite can be used for organizational hierarchies

class Employee {
public:
    virtual ~Employee() = default;
    virtual void print(const std::string& indent = "") = 0;
    virtual void add(std::shared_ptr<Employee> employee) {
        std::cout << "Cannot add to a regular employee" << std::endl;
    }
    virtual void remove(std::shared_ptr<Employee> employee) {
        std::cout << "Cannot remove from a regular employee" << std::endl;
    }
};

class Worker : public Employee {
private:
    std::string name;
    std::string position;

public:
    Worker(const std::string& name, const std::string& position)
        : name(name), position(position) {}

    void print(const std::string& indent = "") override {
        std::cout << indent << "Employee: " << name << " - " << position <<
std::endl;
    }
};

class Manager : public Employee {
private:
    std::string name;
    std::string title;
    std::vector<std::shared_ptr<Employee>> subordinates;

public:
    Manager(const std::string& name, const std::string& title)
        : name(name), title(title) {}

    void add(std::shared_ptr<Employee> employee) override {
        subordinates.push_back(employee);
    }

    void remove(std::shared_ptr<Employee> employee) override {
        subordinates.erase(
            std::remove(subordinates.begin(), subordinates.end(),
employee),
            subordinates.end()

```

```

        );
    }

    void print(const std::string& indent = "") override {
        std::cout << indent << "Manager: " << name << " - " << title <<
std::endl;
        for (const auto& subordinate : subordinates) {
            subordinate->print(indent + "  ");
        }
    }
};

// Example 3: Graphic Shapes Composite
// Shows how Composite can be used for graphics

class Graphic {
public:
    virtual ~Graphic() = default;
    virtual void draw(const std::string& indent = "") = 0;
};

class Circle : public Graphic {
private:
    std::string name;

public:
    Circle(const std::string& name) : name(name) {}

    void draw(const std::string& indent = "") override {
        std::cout << indent << "Drawing Circle: " << name << std::endl;
    }
};

class Square : public Graphic {
private:
    std::string name;

public:
    Square(const std::string& name) : name(name) {}

    void draw(const std::string& indent = "") override {
        std::cout << indent << "Drawing Square: " << name << std::endl;
    }
};

class GraphicGroup : public Graphic {
private:
    std::string name;
    std::vector<std::shared_ptr<Graphic>> graphics;

public:
    GraphicGroup(const std::string& name) : name(name) {}

    void add(std::shared_ptr<Graphic> graphic) {

```

```

        graphics.push_back(graphic);
    }

    void remove(std::shared_ptr<Graphic> graphic) {
        graphics.erase(
            std::remove(graphics.begin(), graphics.end(), graphic),
            graphics.end()
        );
    }

    void draw(const std::string& indent = "") override {
        std::cout << indent << "Group: " << name << std::endl;
        for (const auto& graphic : graphics) {
            graphic->draw(indent + "  ");
        }
    }
};

int main() {
    std::cout << "=== Composite Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: File System
    std::cout << "--- File System Example ---" << std::endl;

    auto file1 = std::make_shared<File>("file1.txt", 100);
    auto file2 = std::make_shared<File>("file2.txt", 200);
    auto file3 = std::make_shared<File>("file3.txt", 150);

    auto dir1 = std::make_shared<Directory>("Documents");
    auto dir2 = std::make_shared<Directory>("Pictures");
    auto rootDir = std::make_shared<Directory>("Root");

    dir1->add(file1);
    dir1->add(file2);

    dir2->add(file3);

    rootDir->add(dir1);
    rootDir->add(dir2);

    std::cout << "\nFile System Structure:" << std::endl;
    rootDir->display();

    std::cout << "\nTotal size of Root directory: " << rootDir->getSize()
<< " bytes" << std::endl;

    std::cout << std::endl;

    // Example 2: Organization Structure
    std::cout << "--- Organization Structure Example ---" << std::endl;

    auto ceo = std::make_shared<Manager>("John Smith", "CEO");
    auto cto = std::make_shared<Manager>("Alice Johnson", "CTO");

```

```

    auto cfo = std::make_shared<Manager>("Bob Williams", "CFO");

    auto dev1 = std::make_shared<Worker>("Developer 1", "Senior Developer");
    auto dev2 = std::make_shared<Worker>("Developer 2", "Junior Developer");
    auto accountant = std::make_shared<Worker>("Accountant", "Senior Accountant");

    cto->add(dev1);
    cto->add(dev2);
    cfo->add(accountant);

    ceo->add(cto);
    ceo->add(cfo);

    std::cout << "\nOrganization Structure:" << std::endl;
    ceo->print();

    std::cout << std::endl;

    // Example 3: Graphic Shapes
    std::cout << "--- Graphic Shapes Example ---" << std::endl;

    auto circle1 = std::make_shared<Circle>("Red Circle");
    auto circle2 = std::make_shared<Circle>("Blue Circle");
    auto square1 = std::make_shared<Square>("Green Square");

    auto group1 = std::make_shared<GraphicGroup>("Shapes Group 1");
    auto group2 = std::make_shared<GraphicGroup>("Shapes Group 2");
    auto mainGroup = std::make_shared<GraphicGroup>("Main Group");

    group1->add(circle1);
    group1->add(square1);

    group2->add(circle2);

    mainGroup->add(group1);
    mainGroup->add(group2);

    std::cout << "\nGraphic Structure:" << std::endl;
    mainGroup->draw();

    return 0;
}

```

Decorator

Description: Adds new behavior or responsibilities to an object dynamically, without altering its class. It wraps the original object and adds functionality on top.

- **Why it is used:** To extend object functionality at runtime without using inheritance, keeping classes flexible and following the Open/Closed Principle.
 - **Where it is commonly used:** I/O streams in Java, middleware in web frameworks, text formatting systems.
-

Code Example (CPP)

```
/*
 * DECORATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Decorator pattern lets you attach new behaviors to objects by
 * placing these objects inside special wrapper objects that contain the
 * behaviors.
 * It provides a flexible alternative to inheritance for extending
 * functionality.
 *
 * WHY DOES IT EXIST?
 * - When you want to add responsibilities to individual objects
 *   dynamically without affecting other objects
 * - When extending a class by subclassing is impractical (too many
 *   subclasses)
 * - When you want to add and remove responsibilities at runtime
 *
 * WHEN TO USE IT?
 * - When you need to add responsibilities to individual objects without
 *   affecting other objects of the same class
 * - When the class definition may be hidden or otherwise unavailable for
 *   subclassing
 * - When you want to add functionality to objects in a transparent way
 *   (clients shouldn't know the difference)
 * - Common use cases: UI components (scrollbars, borders), data streams
 *   (compression, encryption), logging, caching
 *
 * PROS:
 * - Single Responsibility Principle (divides functionality into small
 *   classes)
 * - Open/Closed Principle (can extend behavior without modifying existing
 *   code)
 * - Flexible alternative to subclassing
 * - Can add/remove responsibilities at runtime
 *
 * CONS:
 * - Can result in many small classes (code complexity)
 * - Can make it hard to remove wrapped decorators
 * - Can make debugging difficult due to many layers
 * - Initial setup can be complex
 *
 * REAL-WORLD ANALOGY:
 * Think of ordering coffee. You start with a basic coffee (component),
 * then you can add decorators like milk, sugar, whipped cream, or syrup. Each
```

decorator adds its own behavior (flavor, cost) while still being treated as coffee. You can combine multiple decorators in any order.

```
*/

#include <iostream>
#include <string>
#include <memory>

// Component Interface - defines the interface for objects that can have
responsibilities added
class Coffee {
public:
    virtual ~Coffee() = default;
    virtual double getCost() = 0;
    virtual std::string getDescription() = 0;
};

// Concrete Component - the basic object that can have responsibilities
added
class SimpleCoffee : public Coffee {
public:
    double getCost() override {
        return 2.0;
    }

    std::string getDescription() override {
        return "Simple Coffee";
    }
};

// Decorator - implements the Component interface and wraps a Component
class CoffeeDecorator : public Coffee {
protected:
    std::shared_ptr<Coffee> decoratedCoffee;

public:
    CoffeeDecorator(std::shared_ptr<Coffee> coffee) :
    decoratedCoffee(coffee) {}

    virtual double getCost() override {
        return decoratedCoffee->getCost();
    }

    virtual std::string getDescription() override {
        return decoratedCoffee->getDescription();
    }
};

// Concrete Decorator 1: Milk
class MilkDecorator : public CoffeeDecorator {
public:
    MilkDecorator(std::shared_ptr<Coffee> coffee) : CoffeeDecorator(coffee)
    {}
};
```

```

    double getCost() override {
        return CoffeeDecorator::getCost() + 0.5;
    }

    std::string getDescription() override {
        return CoffeeDecorator::getDescription() + ", Milk";
    }
};

// Concrete Decorator 2: Sugar
class SugarDecorator : public CoffeeDecorator {
public:
    SugarDecorator(std::shared_ptr<Coffee> coffee) :
    CoffeeDecorator(coffee) {}

    double getCost() override {
        return CoffeeDecorator::getCost() + 0.2;
    }

    std::string getDescription() override {
        return CoffeeDecorator::getDescription() + ", Sugar";
    }
};

// Concrete Decorator 3: Whipped Cream
class WhippedCreamDecorator : public CoffeeDecorator {
public:
    WhippedCreamDecorator(std::shared_ptr<Coffee> coffee) :
    CoffeeDecorator(coffee) {}

    double getCost() override {
        return CoffeeDecorator::getCost() + 1.0;
    }

    std::string getDescription() override {
        return CoffeeDecorator::getDescription() + ", Whipped Cream";
    }
};

// Example 2: Text Formatting Decorator
// Shows how decorators can be used for text processing

class TextComponent {
public:
    virtual ~TextComponent() = default;
    virtual std::string getText() = 0;
};

class SimpleText : public TextComponent {
private:
    std::string text;

public:
    SimpleText(const std::string& text) : text(text) {}

```

```
        std::string getText() override {
            return text;
        }
};

class TextDecorator : public TextComponent {
protected:
    std::shared_ptr<TextComponent> wrappedText;

public:
    TextDecorator(std::shared_ptr<TextComponent> text) : wrappedText(text)
    {}

    virtual std::string getText() override {
        return wrappedText->getText();
    }
};

class BoldDecorator : public TextDecorator {
public:
    BoldDecorator(std::shared_ptr<TextComponent> text) :
    TextDecorator(text) {}

    std::string getText() override {
        return "<b>" + TextDecorator::getText() + "</b>";
    }
};

class ItalicDecorator : public TextDecorator {
public:
    ItalicDecorator(std::shared_ptr<TextComponent> text) :
    TextDecorator(text) {}

    std::string getText() override {
        return "<i>" + TextDecorator::getText() + "</i>";
    }
};

class UnderlineDecorator : public TextDecorator {
public:
    UnderlineDecorator(std::shared_ptr<TextComponent> text) :
    TextDecorator(text) {}

    std::string getText() override {
        return "<u>" + TextDecorator::getText() + "</u>";
    }
};

// Example 3: Data Stream Decorator
// Shows how decorators can be used for data processing

class DataSource {
public:
```

```

    virtual ~DataSource() = default;
    virtual void writeData(std::string data) = 0;
    virtual std::string readData() = 0;
};

class FileDataSource : public DataSource {
private:
    std::string data;

public:
    void writeData(std::string data) override {
        this->data = data;
        std::cout << "Writing data to file: " << data << std::endl;
    }

    std::string readData() override {
        std::cout << "Reading data from file: " << data << std::endl;
        return data;
    }
};

class DataSourceDecorator : public DataSource {
protected:
    std::shared_ptr<DataSource> wrappedSource;

public:
    DataSourceDecorator(std::shared_ptr<DataSource> source) :
        wrappedSource(source) {}

    virtual void writeData(std::string data) override {
        wrappedSource->writeData(data);
    }

    virtual std::string readData() override {
        return wrappedSource->readData();
    }
};

class EncryptionDecorator : public DataSourceDecorator {
public:
    EncryptionDecorator(std::shared_ptr<DataSource> source) :
        DataSourceDecorator(source) {}

    void writeData(std::string data) override {
        std::string encrypted = "[ENCRYPTED]" + data + "[ENCRYPTED]";
        std::cout << "Encrypting data..." << std::endl;
        wrappedSource->writeData(encrypted);
    }

    std::string readData() override {
        std::string data = wrappedSource->readData();
        std::cout << "Decrypting data..." << std::endl;
        // Remove encryption markers for demo
        if (data.find("[ENCRYPTED]") == 0) {

```

```

        data = data.substr(11, data.length() - 22);
    }
    return data;
}
};

class CompressionDecorator : public DataSourceDecorator {
public:
    CompressionDecorator(std::shared_ptr<DataSource> source) :
DataSourceDecorator(source) {}

    void writeData(std::string data) override {
        std::string compressed = "[COMPRESSED]" + data + "[COMPRESSED]";
        std::cout << "Compressing data..." << std::endl;
        wrappedSource->writeData(compressed);
    }

    std::string readData() override {
        std::string data = wrappedSource->readData();
        std::cout << "Decompressing data..." << std::endl;
        // Remove compression markers for demo
        if (data.find("[COMPRESSED]") == 0) {
            data = data.substr(12, data.length() - 24);
        }
        return data;
    }
};

int main() {
    std::cout << "=== Decorator Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: Coffee Shop
    std::cout << "--- Coffee Shop Example ---" << std::endl;

    std::shared_ptr<Coffee> coffee = std::make_shared<SimpleCoffee>();
    std::cout << coffee->getDescription() << " $" << coffee->getCost() <<
std::endl;

    coffee = std::make_shared<MilkDecorator>(coffee);
    std::cout << coffee->getDescription() << " $" << coffee->getCost() <<
std::endl;

    coffee = std::make_shared<SugarDecorator>(coffee);
    std::cout << coffee->getDescription() << " $" << coffee->getCost() <<
std::endl;

    coffee = std::make_shared<WhippedCreamDecorator>(coffee);
    std::cout << coffee->getDescription() << " $" << coffee->getCost() <<
std::endl;

    std::cout << std::endl;

    // Example 2: Text Formatting

```

```

std::cout << "--- Text Formatting Example ---" << std::endl;

std::shared_ptr<TextComponent> text = std::make_shared<SimpleText>
("Hello World");
std::cout << "Original: " << text->getText() << std::endl;

text = std::make_shared<BoldDecorator>(text);
std::cout << "Bold: " << text->getText() << std::endl;

text = std::make_shared<ItalicDecorator>(text);
std::cout << "Bold + Italic: " << text->getText() << std::endl;

text = std::make_shared<UnderlineDecorator>(text);
std::cout << "Bold + Italic + Underline: " << text->getText() <<
std::endl;

std::cout << std::endl;

// Example 3: Data Stream Processing
std::cout << "--- Data Stream Processing Example ---" << std::endl;

std::shared_ptr<DataSource> dataSource =
std::make_shared<FileDataSource>();
std::cout << "\nWriting plain data:" << std::endl;
dataSource->writeData("Important Data");
std::cout << "Reading: " << dataSource->readData() << std::endl;

std::cout << "\nWriting encrypted data:" << std::endl;
dataSource = std::make_shared<EncryptionDecorator>(dataSource);
dataSource->writeData("Secret Data");
std::cout << "Reading: " << dataSource->readData() << std::endl;

std::cout << "\nWriting encrypted and compressed data:" << std::endl;
dataSource = std::make_shared<CompressionDecorator>(dataSource);
dataSource->writeData("Very Secret Data");
std::cout << "Reading: " << dataSource->readData() << std::endl;

return 0;
}

```

Facade

Description: Provides a **simplified, unified interface** to a complex subsystem or set of interfaces.

- **Why it is used:** To hide complexity from the client. The client interacts with a simple interface instead of dealing with multiple complex components.
- **Where it is commonly used:** API wrappers, library interfaces, startup/shutdown systems in complex applications.

Code Example (CPP)

```

/*
 * FACADE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Facade pattern provides a simplified interface to a library, a
framework, or any other complex set of classes.
 * It hides the complexities of the larger system and provides a simpler
interface to the client.
 *
 * WHY DOES IT EXIST?
 * - When you need to provide a simple interface to a complex subsystem
 * - When you want to reduce dependencies between clients and the
implementation of an abstraction
 * - When you want to layer your subsystems (use facade to define entry
points)
 *
 * WHEN TO USE IT?
 * - When you have a complex system with many interacting classes
 * - When you want to provide a simple interface for clients to interact
with the system
 * - When you want to decouple the client from the subsystem implementation
 * - Common use cases: API libraries, database operations, complex UI
systems, payment processing
 *
 * PROS:
 * - Simplifies the interface to a complex subsystem
 * - Decouples clients from the subsystem (reduces coupling)
 * - Makes the system easier to use and understand
 * - Promotes loose coupling between subsystems and clients
 *
 * CONS:
 * - Can become a "god object" if not carefully designed
 * - May limit the flexibility of the subsystem
 * - Can hide useful functionality from advanced users
 * - Adds an extra layer of indirection
 *
 * REAL-WORLD ANALOGY:
 * Think of a car dashboard. The dashboard (facade) provides a simple
interface to the complex car systems (engine, transmission, brakes, etc.).
You don't need to know how the engine works to drive the car - you just use
the steering wheel, pedals, and dashboard controls. The dashboard hides the
complexity while giving you access to the essential functions.
 */

#include <iostream>
#include <string>
#include <memory>

// Subsystem Class 1: CPU
class CPU {
public:
    void freeze() {

```

```

        std::cout << "CPU: Freezing..." << std::endl;
    }

    void jump(long position) {
        std::cout << "CPU: Jumping to position " << position << std::endl;
    }

    void execute() {
        std::cout << "CPU: Executing instructions..." << std::endl;
    }
};

// Subsystem Class 2: Memory
class Memory {
public:
    void load(long position, std::string data) {
        std::cout << "Memory: Loading data '" << data << "' at position "
<< position << std::endl;
    }
};

// Subsystem Class 3: Hard Drive
class HardDrive {
public:
    std::string read(long lba, int size) {
        std::cout << "HardDrive: Reading " << size << " bytes from LBA " <<
lba << std::endl;
        return "Boot Data";
    }
};

// Facade - provides a simplified interface to the complex subsystem
class ComputerFacade {
private:
    std::unique_ptr<CPU> cpu;
    std::unique_ptr<Memory> memory;
    std::unique_ptr<HardDrive> hardDrive;

public:
    ComputerFacade()
        : cpu(std::make_unique<CPU>()),
          memory(std::make_unique<Memory>()),
          hardDrive(std::make_unique<HardDrive>()) {}

    void start() {
        std::cout << "=== Starting Computer ===" << std::endl;
        cpu->freeze();
        memory->load(0, hardDrive->read(0, 1024));
        cpu->jump(0);
        cpu->execute();
        std::cout << "=== Computer Started ===" << std::endl;
    }

    void shutdown() {

```

```
        std::cout << "=== Shutting Down Computer ===" << std::endl;
        std::cout << "CPU: Stopping..." << std::endl;
        std::cout << "Memory: Clearing..." << std::endl;
        std::cout << "HardDrive: Spinning down..." << std::endl;
        std::cout << "=== Computer Shutdown ===" << std::endl;
    }
};

// Example 2: Home Theater System
// Shows how Facade can simplify controlling multiple devices

class Amplifier {
public:
    void on() { std::cout << "Amplifier: ON" << std::endl; }
    void off() { std::cout << "Amplifier: OFF" << std::endl; }
    void setVolume(int level) { std::cout << "Amplifier: Volume set to " <<
level << std::endl; }
};

class Tuner {
public:
    void on() { std::cout << "Tuner: ON" << std::endl; }
    void off() { std::cout << "Tuner: OFF" << std::endl; }
    void setFrequency(double freq) { std::cout << "Tuner: Frequency set to
" << freq << " FM" << std::endl; }
};

class DVDPlayer {
public:
    void on() { std::cout << "DVD Player: ON" << std::endl; }
    void off() { std::cout << "DVD Player: OFF" << std::endl; }
    void play(const std::string& movie) { std::cout << "DVD Player: Playing
'" << movie << "'" << std::endl; }
};

class Projector {
public:
    void on() { std::cout << "Projector: ON" << std::endl; }
    void off() { std::cout << "Projector: OFF" << std::endl; }
    void setInput(const std::string& input) { std::cout << "Projector:
Input set to " << input << std::endl; }
};

class TheaterLights {
public:
    void on() { std::cout << "Theater Lights: ON" << std::endl; }
    void off() { std::cout << "Theater Lights: OFF" << std::endl; }
    void dim(int level) { std::cout << "Theater Lights: Dimmed to " <<
level << "%" << std::endl; }
};

class Screen {
public:
    void up() { std::cout << "Screen: UP" << std::endl; }
```

```

    void down() { std::cout << "Screen: DOWN" << std::endl; }
};

class PopcornPopper {
public:
    void on() { std::cout << "Popcorn Popper: ON" << std::endl; }
    void off() { std::cout << "Popcorn Popper: OFF" << std::endl; }
    void pop() { std::cout << "Popcorn Popper: Popping popcorn!" <<
std::endl; }
};

// Facade for Home Theater
class HomeTheaterFacade {
private:
    std::unique_ptr<Amplifier> amp;
    std::unique_ptr<Tuner> tuner;
    std::unique_ptr<DVDPlayer> dvd;
    std::unique_ptr<Projector> projector;
    std::unique_ptr<TheaterLights> lights;
    std::unique_ptr<Screen> screen;
    std::unique_ptr<PopcornPopper> popper;

public:
    HomeTheaterFacade()
        : amp(std::make_unique<Amplifier>()),
          tuner(std::make_unique<Tuner>()),
          dvd(std::make_unique<DVDPlayer>()),
          projector(std::make_unique<Projector>()),
          lights(std::make_unique<TheaterLights>()),
          screen(std::make_unique<Screen>()),
          popper(std::make_unique<PopcornPopper>()) {}

    void watchMovie(const std::string& movie) {
        std::cout << "\n=== Get ready to watch a movie ===" << std::endl;
        popper->on();
        popper->pop();
        lights->dim(10);
        screen->down();
        projector->on();
        projector->setInput("DVD");
        amp->on();
        amp->setVolume(5);
        dvd->on();
        dvd->play(movie);
        std::cout << "=== Movie is playing ===" << std::endl;
    }

    void endMovie() {
        std::cout << "\n=== Shutting movie theater down ===" << std::endl;
        popper->off();
        lights->on();
        screen->up();
        projector->off();
        amp->off();
    }
};

```

```

        dvd->off();
        std::cout << "=== Movie theater shut down ===" << std::endl;
    }

    void listenToRadio(double frequency) {
        std::cout << "\n=== Tuning in to radio ===" << std::endl;
        amp->on();
        amp->setVolume(5);
        tuner->on();
        tuner->setFrequency(frequency);
        std::cout << "=== Radio is playing ===" << std::endl;
    }

    void turnOffRadio() {
        std::cout << "\n=== Shutting down radio ===" << std::endl;
        tuner->off();
        amp->off();
        std::cout << "=== Radio turned off ===" << std::endl;
    }
};

// Example 3: Order Processing System
// Shows how Facade can simplify complex business logic

class InventorySystem {
public:
    bool checkInventory(const std::string& item, int quantity) {
        std::cout << "Inventory: Checking " << quantity << " units of " <<
item << std::endl;
        return true; // Simplified for demo
    }

    void updateInventory(const std::string& item, int quantity) {
        std::cout << "Inventory: Updating " << quantity << " units of " <<
item << std::endl;
    }
};

class PaymentSystem {
public:
    bool processPayment(double amount) {
        std::cout << "Payment: Processing payment of $" << amount <<
std::endl;
        return true; // Simplified for demo
    }

    void refundPayment(double amount) {
        std::cout << "Payment: Refunding $" << amount << std::endl;
    }
};

class ShippingSystem {
public:
    void scheduleShipping(const std::string& address) {

```

```

        std::cout << "Shipping: Scheduling delivery to " << address <<
std::endl;
    }

    void cancelShipping() {
        std::cout << "Shipping: Cancelling delivery" << std::endl;
    }
};

class NotificationSystem {
public:
    void sendConfirmation(const std::string& email) {
        std::cout << "Notification: Sending confirmation to " << email <<
std::endl;
    }

    void sendCancellation(const std::string& email) {
        std::cout << "Notification: Sending cancellation to " << email <<
std::endl;
    }
};

// Facade for Order Processing
class OrderFacade {
private:
    std::unique_ptr<InventorySystem> inventory;
    std::unique_ptr<PaymentSystem> payment;
    std::unique_ptr<ShippingSystem> shipping;
    std::unique_ptr<NotificationSystem> notification;

public:
    OrderFacade()
        : inventory(std::make_unique<InventorySystem>()),
          payment(std::make_unique<PaymentSystem>()),
          shipping(std::make_unique<ShippingSystem>()),
          notification(std::make_unique<NotificationSystem>()) {}

    bool placeOrder(const std::string& item, int quantity, double amount,
                    const std::string& address, const std::string& email) {
        std::cout << "\n=== Processing Order ===" << std::endl;

        if (!inventory->checkInventory(item, quantity)) {
            std::cout << "Order failed: Item not in stock" << std::endl;
            return false;
        }

        if (!payment->processPayment(amount)) {
            std::cout << "Order failed: Payment declined" << std::endl;
            return false;
        }

        inventory->updateInventory(item, -quantity);
        shipping->scheduleShipping(address);
        notification->sendConfirmation(email);
    }
};

```

```

        std::cout << "=== Order Placed Successfully ===" << std::endl;
        return true;
    }

    void cancelOrder(const std::string& item, int quantity, double amount,
                    const std::string& email) {
        std::cout << "\n=== Cancelling Order ===" << std::endl;
        inventory->updateInventory(item, quantity);
        payment->refundPayment(amount);
        shipping->cancelShipping();
        notification->sendCancellation(email);
        std::cout << "=== Order Cancelled ===" << std::endl;
    }
};

int main() {
    std::cout << "=== Facade Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Computer Boot
    std::cout << "--- Computer Boot Example ---" << std::endl;

    ComputerFacade computer;
    computer.start();
    std::cout << std::endl;
    computer.shutdown();

    std::cout << std::endl;

    // Example 2: Home Theater
    std::cout << "--- Home Theater Example ---" << std::endl;

    HomeTheaterFacade homeTheater;
    homeTheater.watchMovie("The Matrix");
    homeTheater.endMovie();
    homeTheater.listenToRadio(98.7);
    homeTheater.turnOffRadio();

    std::cout << std::endl;

    // Example 3: Order Processing
    std::cout << "--- Order Processing Example ---" << std::endl;

    OrderFacade orderSystem;
    orderSystem.placeOrder("Laptop", 1, 999.99, "123 Main St",
"customer@email.com");
    orderSystem.cancelOrder("Laptop", 1, 999.99, "customer@email.com");

    return 0;
}

```

Description: Uses **sharing to efficiently support a large number of fine-grained objects**. It separates intrinsic (shared) state from extrinsic (unique) state.

- **Why it is used:** To reduce memory usage when a large number of similar objects need to be created.
- **Where it is commonly used:** Text editors (reusing character objects), game engines (reusing tile or bullet objects), rendering systems.

Code Example (CPP)

```
/*
 * FLYWEIGHT DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Flyweight pattern lets you fit more objects into the available RAM
by sharing common parts of state between multiple objects.
 * It separates intrinsic state (shared) from extrinsic state (unique).
 *
 * WHY DOES IT EXIST?
 * - When you need to reduce memory usage by sharing objects
 * - When a large number of similar objects should be shared
 * - When most of an object's state can be made extrinsic
 *
 * WHEN TO USE IT?
 * - When your application uses a large number of objects
 * - When storage costs are high because of the quantity of objects
 * - When most of the object's state can be made extrinsic
 * - When many groups of objects can be replaced by relatively few shared
objects
 * - Common use cases: text editors (character formatting), game objects
(trees, soldiers), GUI elements (icons, fonts)
 *
 * PROS:
 * - Reduces memory consumption by sharing objects
 * - Improves performance when many similar objects are needed
 * - Centralized state management
 * - Can be combined with other patterns (Composite, Strategy)
 *
 * CONS:
 * - May increase runtime complexity due to sharing
 * - Requires careful identification of intrinsic vs extrinsic state
 * - Can make code more complex
 * - May sacrifice some encapsulation (extrinsic state passed to flyweight)
 *
 * REAL-WORLD ANALOGY:
 * Think of a coffee shop. They don't make a new cup for every customer -
they reuse cups (intrinsic state: the cup itself). What changes is the
coffee inside (extrinsic state: the type of coffee). The cup is shared
(flyweight), while the coffee content varies per customer.
 */

#include <iostream>
```

```

#include <string>
#include <unordered_map>
#include <memory>
#include <vector>

// Flyweight Interface - defines the operations that can be performed on
flyweight objects
class TreeType {
public:
    virtual ~TreeType() = default;
    virtual void draw(int x, int y) = 0;
    std::string getName() const { return name; }
    std::string getColor() const { return color; }

protected:
    std::string name;
    std::string color;
};

// Concrete Flyweight - implements the flyweight interface and stores
intrinsic state
class ConcreteTreeType : public TreeType {
public:
    ConcreteTreeType(const std::string& name, const std::string& color,
const std::string& texture)
        : name(name), color(color), texture(texture) {}

    void draw(int x, int y) override {
        std::cout << "Drawing " << name << " tree at (" << x << ", " << y
        << ") with color " << color << " and texture " << texture
<< std::endl;
    }

private:
    std::string texture;
};

// Flyweight Factory - creates and manages flyweight objects
class TreeFactory {
private:
    std::unordered_map<std::string, std::shared_ptr<TreeType>> treeTypes;

    // Helper function to create a unique key for the map
    std::string getKey(const std::string& name, const std::string& color,
const std::string& texture) {
        return name + "|" + color + "|" + texture;
    }

public:
    std::shared_ptr<TreeType> getTreeType(const std::string& name, const
std::string& color, const std::string& texture) {
        std::string key = getKey(name, color, texture);

        if (treeTypes.find(key) == treeTypes.end()) {

```

```

        std::cout << "Creating new tree type: " << name << std::endl;
        treeTypes[key] = std::make_shared<ConcreteTreeType>(name,
color, texture);
    } else {
        std::cout << "Reusing existing tree type: " << name <<
std::endl;
    }

    return treeTypes[key];
}

int getTotalTreeTypes() const {
    return treeTypes.size();
}
};

// Context - contains extrinsic state and uses flyweight objects
class Tree {
private:
    int x, y;
    std::shared_ptr<TreeType> treeType;

public:
    Tree(int x, int y, std::shared_ptr<TreeType> treeType)
        : x(x), y(y), treeType(treeType) {}

    void draw() {
        treeType->draw(x, y);
    }
};

// Example 2: Character Formatting in Text Editor
// Shows how Flyweight can be used for text rendering

class CharacterFormat {
public:
    virtual ~CharacterFormat() = default;
    virtual void apply(const std::string& text) = 0;

    std::string getFont() const { return font; }
    int getSize() const { return size; }
    std::string getColor() const { return color; }

protected:
    std::string font;
    int size;
    std::string color;
};

class ConcreteCharacterFormat : public CharacterFormat {
public:
    ConcreteCharacterFormat(const std::string& font, int size, const
std::string& color, bool bold, bool italic)
        : font(font), size(size), color(color), bold(bold), italic(italic)

```

```

{}

void apply(const std::string& text) override {
    std::cout << "Rendering '" << text << "' with font: " << font
                << ", size: " << size << ", color: " << color;
    if (bold) std::cout << ", bold";
    if (italic) std::cout << ", italic";
    std::cout << std::endl;
}

private:
    bool bold;
    bool italic;
};

class CharacterFormatFactory {
private:
    std::unordered_map<std::string, std::shared_ptr<CharacterFormat>>
    formats;

    std::string getKey(const std::string& font, int size, const
std::string& color, bool bold, bool italic) {
        return font + "|" + std::to_string(size) + "|" + color + "|" +
            (bold ? "B" : "") + "|" + (italic ? "I" : "");
    }

public:
    std::shared_ptr<CharacterFormat> getFormat(const std::string& font, int
size,
                                                const std::string& color,
bool bold, bool italic) {
        std::string key = getKey(font, size, color, bold, italic);

        if (formats.find(key) == formats.end()) {
            std::cout << "Creating new format: " << font << " " << size <<
"pt" << std::endl;
            formats[key] = std::make_shared<ConcreteCharacterFormat>(font,
size, color, bold, italic);
        } else {
            std::cout << "Reusing existing format: " << font << " " << size
<< "pt" << std::endl;
        }

        return formats[key];
    }

    int getTotalFormats() const {
        return formats.size();
    }
};

class Character {
private:
    char symbol;

```

```

        std::shared_ptr<CharacterFormat> format;

public:
    Character(char symbol, std::shared_ptr<CharacterFormat> format)
        : symbol(symbol), format(format) {}

    void render() {
        std::string text(1, symbol);
        format->apply(text);
    }
};

// Example 3: Game Units
// Shows how Flyweight can be used for game objects

class SoldierType {
public:
    virtual ~SoldierType() = default;
    virtual void render(int x, int y, const std::string& weapon) = 0;

    std::string getUniform() const { return uniform; }
    int getSpeed() const { return speed; }

protected:
    std::string uniform;
    int speed;
};

class ConcreteSoldierType : public SoldierType {
public:
    ConcreteSoldierType(const std::string& uniform, int speed, const
std::string& rank)
        : uniform(uniform), speed(speed), rank(rank) {}

    void render(int x, int y, const std::string& weapon) override {
        std::cout << "Rendering " << rank << " soldier at (" << x << ", "
<< y
                << ") with " << uniform << " uniform, speed " << speed
                << ", holding " << weapon << std::endl;
    }

private:
    std::string rank;
};

class SoldierTypeFactory {
private:
    std::unordered_map<std::string, std::shared_ptr<SoldierType>>
soldierTypes;

    std::string getKey(const std::string& uniform, int speed, const
std::string& rank) {
        return uniform + "|" + std::to_string(speed) + "|" + rank;
    }
};

```

```

public:
    std::shared_ptr<SoldierType> getSoldierType(const std::string& uniform,
int speed, const std::string& rank) {
        std::string key = getKey(uniform, speed, rank);

        if (soldierTypes.find(key) == soldierTypes.end()) {
            std::cout << "Creating new soldier type: " << rank <<
std::endl;
            soldierTypes[key] = std::make_shared<ConcreteSoldierType>
(uniform, speed, rank);
        } else {
            std::cout << "Reusing existing soldier type: " << rank <<
std::endl;
        }

        return soldierTypes[key];
    }

    int getTotalSoldierTypes() const {
        return soldierTypes.size();
    }
};

class Soldier {
private:
    int x, y;
    std::string weapon;
    std::shared_ptr<SoldierType> soldierType;

public:
    Soldier(int x, int y, const std::string& weapon,
std::shared_ptr<SoldierType> soldierType)
        : x(x), y(y), weapon(weapon), soldierType(soldierType) {}

    void render() {
        soldierType->render(x, y, weapon);
    }

    void move(int newX, int newY) {
        x = newX;
        y = newY;
    }
};

int main() {
    std::cout << "=== Flyweight Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: Forest Trees
    std::cout << "--- Forest Trees Example ---" << std::endl;

    TreeFactory treeFactory;

```

```

// Create trees with shared types
std::vector<Tree> forest;

forest.emplace_back(10, 20, treeFactory.getTreeType("Oak", "Green",
"Rough"));
forest.emplace_back(30, 40, treeFactory.getTreeType("Oak", "Green",
"Rough"));
forest.emplace_back(50, 60, treeFactory.getTreeType("Oak", "Green",
"Rough"));

forest.emplace_back(15, 25, treeFactory.getTreeType("Pine", "Dark
Green", "Smooth"));
forest.emplace_back(35, 45, treeFactory.getTreeType("Pine", "Dark
Green", "Smooth"));

forest.emplace_back(20, 30, treeFactory.getTreeType("Oak", "Green",
"Rough")); // Reusing Oak

std::cout << "\nDrawing forest:" << std::endl;
for (const auto& tree : forest) {
    tree.draw();
}

std::cout << "\nTotal unique tree types: " <<
treeFactory.getTotalTreeTypes() << std::endl;
std::cout << "Total trees in forest: " << forest.size() << std::endl;

std::cout << std::endl;

// Example 2: Text Editor Characters
std::cout << "--- Text Editor Characters Example ---" << std::endl;

CharacterFormatFactory formatFactory;

std::vector<Character> document;

document.emplace_back('H', formatFactory.getFormat("Arial", 12,
"Black", true, false));
document.emplace_back('e', formatFactory.getFormat("Arial", 12,
"Black", false, false));
document.emplace_back('l', formatFactory.getFormat("Arial", 12,
"Black", false, false));
document.emplace_back('l', formatFactory.getFormat("Arial", 12,
"Black", false, false));
document.emplace_back('o', formatFactory.getFormat("Arial", 12,
"Black", false, false));

document.emplace_back(' ', formatFactory.getFormat("Arial", 12,
"Black", false, false));

document.emplace_back('W', formatFactory.getFormat("Times New Roman",
14, "Blue", true, true));
document.emplace_back('o', formatFactory.getFormat("Times New Roman",
14, "Blue", false, true));

```

```

    document.emplace_back('r', formatFactory.getFormat("Times New Roman",
14, "Blue", false, true));
    document.emplace_back('l', formatFactory.getFormat("Times New Roman",
14, "Blue", false, true));
    document.emplace_back('d', formatFactory.getFormat("Times New Roman",
14, "Blue", false, true));

    std::cout << "\nRendering document:" << std::endl;
    for (const auto& character : document) {
        character.render();
    }

    std::cout << "\nTotal unique formats: " <<
formatFactory.getTotalFormats() << std::endl;
    std::cout << "Total characters: " << document.size() << std::endl;

    std::cout << std::endl;

    // Example 3: Game Soldiers
    std::cout << "--- Game Soldiers Example ---" << std::endl;

    SoldierTypeFactory soldierFactory;

    std::vector<Soldier> army;

    army.emplace_back(10, 10, "Rifle",
soldierFactory.getSoldierType("Camouflage", 5, "Private"));
    army.emplace_back(20, 20, "Rifle",
soldierFactory.getSoldierType("Camouflage", 5, "Private"));
    army.emplace_back(30, 30, "Rifle",
soldierFactory.getSoldierType("Camouflage", 5, "Private"));

    army.emplace_back(15, 15, "Sniper",
soldierFactory.getSoldierType("Desert", 4, "Sergeant"));
    army.emplace_back(25, 25, "Sniper",
soldierFactory.getSoldierType("Desert", 4, "Sergeant"));

    army.emplace_back(40, 40, "Machine Gun",
soldierFactory.getSoldierType("Camouflage", 5, "Private")); // Reusing

    std::cout << "\nRendering army:" << std::endl;
    for (const auto& soldier : army) {
        soldier.render();
    }

    std::cout << "\nTotal unique soldier types: " <<
soldierFactory.getTotalSoldierTypes() << std::endl;
    std::cout << "Total soldiers: " << army.size() << std::endl;

    return 0;
}

```

Proxy

Description: Provides a **substitute or placeholder** for another object to control access to it.

- **Why it is used:** To add a layer of control over the original object — for access control, lazy initialization, logging, or caching — without changing the original object.
- **Where it is commonly used:** Virtual proxies (lazy loading images), security proxies (access control), remote proxies (network calls).

Code Example (CPP)

```
/*
 * PROXY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Proxy pattern provides a surrogate or placeholder object to control
access to another object.
 * It acts as an intermediary between the client and the real object.
 *
 * WHY DOES IT EXIST?
 * - When you need to control access to an object
 * - When you want to add additional functionality when accessing an object
 * - When you need to defer the creation or initialization of expensive
objects
 *
 * WHEN TO USE IT?
 * - When you want to add a layer of security to access an object
 * - When you need to cache results of expensive operations
 * - When you want to delay object creation until it's actually needed
(lazy initialization)
 * - When you need to log or monitor access to an object
 * - Common use cases: remote objects, virtual proxies, protection proxies,
smart references
 *
 * PROS:
 * - Can control access to the real object
 * - Can add additional functionality without changing the real object
 * - Can optimize performance (lazy loading, caching)
 * - Can provide security by controlling access
 *
 * CONS:
 * - Adds an extra layer of indirection
 * - Can increase response time due to proxy overhead
 * - Can make the code more complex
 * - May introduce bugs if the proxy doesn't perfectly mimic the real
object
 *
 * REAL-WORLD ANALOGY:
 * Think of a credit card. The credit card (proxy) represents your bank
account (real object). You don't carry your actual bank account around -
you use the card as a proxy. The card controls access to your money, adds
```

```

security (PIN, chip), and logs transactions, all while giving you access to
your funds.
*/

#include <iostream>
#include <string>
#include <memory>
#include <unordered_map>

// Subject Interface - defines the common interface for RealSubject and
Proxy
class Image {
public:
    virtual ~Image() = default;
    virtual void display() = 0;
};

// RealSubject - the actual object that the proxy represents
class RealImage : public Image {
private:
    std::string filename;

    // Helper method to load the image from disk
    void loadFromDisk() {
        std::cout << "Loading " << filename << " from disk..." <<
std::endl;
    }

public:
    RealImage(const std::string& filename) : filename(filename) {
        loadFromDisk();
    }

    void display() override {
        std::cout << "Displaying " << filename << std::endl;
    }
};

// Proxy - controls access to the RealSubject
class ProxyImage : public Image {
private:
    std::string filename;
    std::unique_ptr<RealImage> realImage;

public:
    ProxyImage(const std::string& filename) : filename(filename) {}

    void display() override {
        // Lazy initialization: only create the real image when needed
        if (!realImage) {
            std::cout << "Creating real image on first access..." <<
std::endl;
            realImage = std::make_unique<RealImage>(filename);
        }
    }
};

```

```

        realImage->display();
    }
};

// Example 2: Protection Proxy
// Shows how Proxy can control access based on permissions

class Database {
public:
    virtual ~Database() = default;
    virtual void query(const std::string& sql) = 0;
};

class RealDatabase : public Database {
public:
    void query(const std::string& sql) override {
        std::cout << "Executing query: " << sql << std::endl;
    }
};

class DatabaseProxy : public Database {
private:
    std::unique_ptr<RealDatabase> realDatabase;
    std::string currentUser;
    std::unordered_map<std::string, bool> permissions;

    bool hasPermission() {
        return permissions[currentUser];
    }

public:
    DatabaseProxy(const std::string& user) : currentUser(user) {
        // Set up permissions (in real app, this would come from auth
system)
        permissions["admin"] = true;
        permissions["user"] = false;
        permissions["guest"] = false;
    }

    void query(const std::string& sql) override {
        if (hasPermission()) {
            if (!realDatabase) {
                realDatabase = std::make_unique<RealDatabase>();
            }
            realDatabase->query(sql);
        } else {
            std::cout << "Access denied: User '" << currentUser
                << "' does not have permission to execute queries" <<
std::endl;
        }
    }

    void setUser(const std::string& user) {
        currentUser = user;
    }
};

```

```

    }
};

// Example 3: Caching Proxy
// Shows how Proxy can cache expensive operations

class Video {
public:
    virtual ~Video() = default;
    virtual void play() = 0;
};

class RealVideo : public Video {
private:
    std::string filename;

    void loadVideo() {
        std::cout << "Loading video " << filename << " from server..." <<
std::endl;
    }

public:
    RealVideo(const std::string& filename) : filename(filename) {
        loadVideo();
    }

    void play() override {
        std::cout << "Playing video: " << filename << std::endl;
    }
};

class VideoProxy : public Video {
private:
    std::string filename;
    std::unique_ptr<RealVideo> realVideo;
    bool isLoaded = false;

public:
    VideoProxy(const std::string& filename) : filename(filename) {}

    void play() override {
        if (!isLoaded) {
            std::cout << "Loading video on first play..." << std::endl;
            realVideo = std::make_unique<RealVideo>(filename);
            isLoaded = true;
        } else {
            std::cout << "Using cached video..." << std::endl;
        }
        realVideo->play();
    }
};

// Example 4: Logging Proxy
// Shows how Proxy can add logging functionality

```

```

class Service {
public:
    virtual ~Service() = default;
    virtual void doWork() = 0;
};

class RealService : public Service {
public:
    void doWork() override {
        std::cout << "Service: Performing work..." << std::endl;
    }
};

class LoggingProxy : public Service {
private:
    std::unique_ptr<RealService> realService;

    void log(const std::string& message) {
        std::cout << "[LOG] " << message << std::endl;
    }

public:
    LoggingProxy() : realService(std::make_unique<RealService>()) {}

    void doWork() override {
        log("Before doWork");
        realService->doWork();
        log("After doWork");
    }
};

// Example 5: Remote Proxy (simulated)
// Shows how Proxy can represent remote objects

class RemoteServer {
public:
    virtual ~RemoteServer() = default;
    virtual void request(const std::string& data) = 0;
};

class RealRemoteServer : public RemoteServer {
public:
    void request(const std::string& data) override {
        std::cout << "Remote server processing request: " << data <<
std::endl;
    }
};

class RemoteServerProxy : public RemoteServer {
private:
    std::unique_ptr<RealRemoteServer> remoteServer;
    std::string serverAddress;

```

```

    void connectToServer() {
        std::cout << "Connecting to remote server at " << serverAddress <<
        "... " << std::endl;
    }

public:
    RemoteServerProxy(const std::string& address) : serverAddress(address)
    {}

    void request(const std::string& data) override {
        if (!remoteServer) {
            connectToServer();
            remoteServer = std::make_unique<RealRemoteServer>();
        }
        std::cout << "Proxy forwarding request to remote server..." <<
        std::endl;
        remoteServer->request(data);
    }
};

int main() {
    std::cout << "=== Proxy Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Virtual Proxy (Lazy Loading)
    std::cout << "--- Virtual Proxy (Lazy Loading) Example ---" <<
    std::endl;

    std::shared_ptr<Image> image1 = std::make_shared<ProxyImage>
    ("photo1.jpg");
    std::shared_ptr<Image> image2 = std::make_shared<ProxyImage>
    ("photo2.jpg");
    std::shared_ptr<Image> image3 = std::make_shared<ProxyImage>
    ("photo1.jpg"); // Same as image1

    std::cout << "\nFirst display of image1:" << std::endl;
    image1->display();

    std::cout << "\nSecond display of image1 (should use cached):" <<
    std::endl;
    image1->display();

    std::cout << "\nDisplay of image2:" << std::endl;
    image2->display();

    std::cout << "\nDisplay of image3 (same as image1, should use cached):"
    << std::endl;
    image3->display();

    std::cout << std::endl;

    // Example 2: Protection Proxy
    std::cout << "--- Protection Proxy Example ---" << std::endl;

    DatabaseProxy db("guest");

```

```

std::cout << "\nGuest trying to execute query:" << std::endl;
db.query("SELECT * FROM users");

std::cout << "\nSwitching to admin user:" << std::endl;
db.setUser("admin");
db.query("SELECT * FROM users");

std::cout << "\nSwitching back to regular user:" << std::endl;
db.setUser("user");
db.query("SELECT * FROM users");

std::cout << std::endl;

// Example 3: Caching Proxy
std::cout << "--- Caching Proxy Example ---" << std::endl;

VideoProxy video("movie.mp4");

std::cout << "\nFirst play (will load):" << std::endl;
video.play();

std::cout << "\nSecond play (will use cache):" << std::endl;
video.play();

std::cout << "\nThird play (will use cache):" << std::endl;
video.play();

std::cout << std::endl;

// Example 4: Logging Proxy
std::cout << "--- Logging Proxy Example ---" << std::endl;

LoggingProxy loggingService;
loggingService.doWork();

std::cout << std::endl;

// Example 5: Remote Proxy
std::cout << "--- Remote Proxy Example ---" << std::endl;

RemoteServerProxy remoteProxy("192.168.1.100");

std::cout << "\nFirst request (will connect):" << std::endl;
remoteProxy.request("GET /api/data");

std::cout << "\nSecond request (already connected):" << std::endl;
remoteProxy.request("POST /api/data");

return 0;
}

```

3. Behavioral Design Patterns

Overview

Behavioral patterns deal with **how objects interact and communicate** with each other. They focus on the assignment of responsibilities between objects and define clear communication patterns to make the system more flexible in carrying out behavior.

Chain of Responsibility

Description: Passes a request along a **chain of handlers**, where each handler decides to process the request or pass it to the next handler in the chain.

- **Why it is used:** To decouple the sender of a request from its receivers and give multiple objects the chance to handle the request.
 - **Where it is commonly used:** Middleware pipelines (HTTP request handling), event handling systems, logging with multiple levels.
-

Code Example (CPP)

```
/*
 * CHAIN OF RESPONSIBILITY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Chain of Responsibility pattern lets you pass requests along a chain
of handlers.
 * Each handler decides either to process the request or to pass it to the
next handler in the chain.
 *
 * WHY DOES IT EXIST?
 * - When you want to decouple senders of a request from its receivers
 * - When multiple objects can handle a request without the sender knowing
which one will
 * - When you want to dynamically add or remove handlers at runtime
 *
 * WHEN TO USE IT?
 * - When more than one object can handle a request, and the handler isn't
known a priori
 * - When you want to issue a request to one of several objects without
specifying the receiver explicitly
 * - When the set of objects that can handle a request should be specified
dynamically
 * - Common use cases: event handling systems, logging frameworks, approval
workflows, help systems
 *
 * PROS:
 * - Decouples sender from receiver (sender doesn't know which handler will
process)
 * - Open/Closed Principle (can add new handlers without changing existing
code)
 * - Can control the order of handling
 * - Simplifies object interactions
```

```

*
* CONS:
* - No guarantee that a request will be handled
* - Can lead to deep chains that are hard to debug
* - May affect performance if the chain is long
* - Can be difficult to observe runtime characteristics
*
* REAL-WORLD ANALOGY:
* Think of a customer service call center. When you call, your request
goes through a chain of representatives. If the first representative can't
help, they transfer you to a specialist. If that specialist can't help,
they transfer you to a manager. The call moves up the chain until someone
can help or you reach the end.
*/

#include <iostream>
#include <string>
#include <memory>

// Handler interface - declares the handling method and sets a reference to
the next handler
class Handler {
protected:
    std::shared_ptr<Handler> nextHandler;

public:
    virtual ~Handler() = default;

    void setNext(std::shared_ptr<Handler> handler) {
        nextHandler = handler;
    }

    virtual void handleRequest(const std::string& request) = 0;
};

// Concrete Handler 1: Support Agent
class SupportAgent : public Handler {
public:
    void handleRequest(const std::string& request) override {
        if (request == "technical_issue" || request == "basic_question") {
            std::cout << "SupportAgent: Handling request - " << request <<
std::endl;
        } else if (nextHandler) {
            std::cout << "SupportAgent: Passing to next handler..." <<
std::endl;
            nextHandler->handleRequest(request);
        } else {
            std::cout << "SupportAgent: Cannot handle this request" <<
std::endl;
        }
    }
};

// Concrete Handler 2: Technical Specialist

```

```

class TechnicalSpecialist : public Handler {
public:
    void handleRequest(const std::string& request) override {
        if (request == "complex_technical" || request == "bug_report") {
            std::cout << "TechnicalSpecialist: Handling request - " <<
request << std::endl;
        } else if (nextHandler) {
            std::cout << "TechnicalSpecialist: Passing to next handler..."
<< std::endl;
            nextHandler->handleRequest(request);
        } else {
            std::cout << "TechnicalSpecialist: Cannot handle this request"
<< std::endl;
        }
    }
};

// Concrete Handler 3: Manager
class Manager : public Handler {
public:
    void handleRequest(const std::string& request) override {
        if (request == "refund" || request == "complaint" || request ==
"escalation") {
            std::cout << "Manager: Handling request - " << request <<
std::endl;
        } else if (nextHandler) {
            std::cout << "Manager: Passing to next handler..." <<
std::endl;
            nextHandler->handleRequest(request);
        } else {
            std::cout << "Manager: Cannot handle this request" <<
std::endl;
        }
    }
};

// Example 2: Document Approval Chain
// Shows how Chain of Responsibility can be used for approval workflows

class Approver {
protected:
    std::shared_ptr<Approver> nextApprover;

public:
    virtual ~Approver() = default;

    void setNext(std::shared_ptr<Approver> approver) {
        nextApprover = approver;
    }

    virtual void approveRequest(double amount) = 0;
};

class TeamLead : public Approver {

```

```
public:
    void approveRequest(double amount) override {
        if (amount <= 1000) {
            std::cout << "TeamLead: Approved request of $" << amount <<
std::endl;
        } else if (nextApprover) {
            std::cout << "TeamLead: Amount exceeds limit, passing to
manager..." << std::endl;
            nextApprover->approveRequest(amount);
        } else {
            std::cout << "TeamLead: Cannot approve this amount" <<
std::endl;
        }
    }
};

class Manager : public Approver {
public:
    void approveRequest(double amount) override {
        if (amount <= 5000) {
            std::cout << "Manager: Approved request of $" << amount <<
std::endl;
        } else if (nextApprover) {
            std::cout << "Manager: Amount exceeds limit, passing to
director..." << std::endl;
            nextApprover->approveRequest(amount);
        } else {
            std::cout << "Manager: Cannot approve this amount" <<
std::endl;
        }
    }
};

class Director : public Approver {
public:
    void approveRequest(double amount) override {
        if (amount <= 20000) {
            std::cout << "Director: Approved request of $" << amount <<
std::endl;
        } else if (nextApprover) {
            std::cout << "Director: Amount exceeds limit, passing to
CEO..." << std::endl;
            nextApprover->approveRequest(amount);
        } else {
            std::cout << "Director: Cannot approve this amount" <<
std::endl;
        }
    }
};

class CEO : public Approver {
public:
    void approveRequest(double amount) override {
        std::cout << "CEO: Approved request of $" << amount << " (final
```

```
approver)" << std::endl;
    }
};

// Example 3: Logging Chain
// Shows how Chain of Responsibility can be used for logging

class Logger {
protected:
    std::shared_ptr<Logger> nextLogger;

public:
    virtual ~Logger() = default;

    void setNext(std::shared_ptr<Logger> logger) {
        nextLogger = logger;
    }

    virtual void log(const std::string& message, int severity) = 0;
};

class ConsoleLogger : public Logger {
public:
    void log(const std::string& message, int severity) override {
        if (severity >= 1) {
            std::cout << "[CONSOLE] " << message << std::endl;
        }
        if (nextLogger) {
            nextLogger->log(message, severity);
        }
    }
};

class FileLogger : public Logger {
public:
    void log(const std::string& message, int severity) override {
        if (severity >= 2) {
            std::cout << "[FILE] " << message << std::endl;
        }
        if (nextLogger) {
            nextLogger->log(message, severity);
        }
    }
};

class ErrorLogger : public Logger {
public:
    void log(const std::string& message, int severity) override {
        if (severity >= 3) {
            std::cout << "[ERROR] " << message << std::endl;
        }
        if (nextLogger) {
            nextLogger->log(message, severity);
        }
    }
};
```

```
    }  
};  
  
int main() {  
    std::cout << "=== Chain of Responsibility Pattern Demo ===" <<  
std::endl << std::endl;  
  
    // Example 1: Customer Service Chain  
std::cout << "--- Customer Service Chain ---" << std::endl;  
  
    auto agent = std::make_shared<SupportAgent>();  
    auto specialist = std::make_shared<TechnicalSpecialist>();  
    auto manager = std::make_shared<Manager>();  
  
    agent->setNext(specialist);  
    specialist->setNext(manager);  
  
    std::cout << "\nRequest: basic_question" << std::endl;  
    agent->handleRequest("basic_question");  
  
    std::cout << "\nRequest: complex_technical" << std::endl;  
    agent->handleRequest("complex_technical");  
  
    std::cout << "\nRequest: refund" << std::endl;  
    agent->handleRequest("refund");  
  
    std::cout << "\nRequest: unknown" << std::endl;  
    agent->handleRequest("unknown");  
  
    std::cout << std::endl;  
  
    // Example 2: Document Approval Chain  
std::cout << "--- Document Approval Chain ---" << std::endl;  
  
    auto teamLead = std::make_shared<TeamLead>();  
    auto manager2 = std::make_shared<Manager>();  
    auto director = std::make_shared<Director>();  
    auto ceo = std::make_shared<CEO>();  
  
    teamLead->setNext(manager2);  
    manager2->setNext(director);  
    director->setNext(ceo);  
  
    std::cout << "\nRequest: $500" << std::endl;  
    teamLead->approveRequest(500);  
  
    std::cout << "\nRequest: $3000" << std::endl;  
    teamLead->approveRequest(3000);  
  
    std::cout << "\nRequest: $15000" << std::endl;  
    teamLead->approveRequest(15000);  
  
    std::cout << "\nRequest: $50000" << std::endl;  
    teamLead->approveRequest(50000);  
}
```

```

std::cout << std::endl;

// Example 3: Logging Chain
std::cout << "--- Logging Chain ---" << std::endl;

auto consoleLogger = std::make_shared<ConsoleLogger>();
auto fileLogger = std::make_shared<FileLogger>();
auto errorLogger = std::make_shared<ErrorLogger>();

consoleLogger->setNext(fileLogger);
fileLogger->setNext(errorLogger);

std::cout << "\nLog: Info message (severity 1)" << std::endl;
consoleLogger->log("Info message", 1);

std::cout << "\nLog: Warning message (severity 2)" << std::endl;
consoleLogger->log("Warning message", 2);

std::cout << "\nLog: Error message (severity 3)" << std::endl;
consoleLogger->log("Error message", 3);

return 0;
}

```

Command

Description: Encapsulates a request as an object, allowing you to parameterize clients with different requests, queue operations, and support undoable actions.

- **Why it is used:** To decouple the object that sends a request from the object that executes it, enabling flexible command management.
- **Where it is commonly used:** Undo/redo systems, task schedulers, GUI buttons and menu actions, transactional operations.

Code Example (CPP)

```

/*
 * COMMAND DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Command pattern turns a request into a stand-alone object that
 * contains all information about the request.
 * It lets you parameterize clients with different requests, queue or log
 * requests, and support undoable operations.
 *
 * WHY DOES IT EXIST?
 * - When you want to parameterize objects with operations
 * - When you want to queue operations, schedule their execution, or
 * execute them remotely
 */

```

```

* - When you need to support undo operations
* - When you want to decouple the object that invokes an operation from
the one that knows how to perform it
*
* WHEN TO USE IT?
* - When you want to parameterize objects with an operation to execute
* - When you want to queue operations, schedule their execution, or
execute them remotely
* - When you need to support undo/redo functionality
* - When you want to decouple the invoker from the receiver
* - Common use cases: GUI buttons and menus, multi-level undo, macro
recording, transactional systems
*
* PROS:
* - Decouples the object that invokes the operation from the one that
knows how to perform it
* - Can add new commands without changing existing code (Open/Closed
Principle)
* - Allows you to assemble commands into more complex commands (composite
pattern)
* - Supports undo/redo operations
*
* CONS:
* - Can lead to many small command classes (code complexity)
* - Can complicate the code if the receiver and command are tightly
coupled
* - May introduce additional layers of indirection
* - Can make debugging more difficult
*
* REAL-WORLD ANALOGY:
* Think of a restaurant order. The customer (client) places an order
(command) with the waiter (invoker). The waiter writes down the order and
gives it to the kitchen (receiver). The kitchen executes the command
(prepare the meal). The order can be cancelled (undo) before it's
prepared, and multiple orders can be queued.
*/

```

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <stack>

// Receiver - knows how to perform the operations
class Light {
public:
    void on() {
        std::cout << "Light is ON" << std::endl;
    }

    void off() {
        std::cout << "Light is OFF" << std::endl;
    }
};

```

```
class Stereo {
public:
    void on() {
        std::cout << "Stereo is ON" << std::endl;
    }

    void off() {
        std::cout << "Stereo is OFF" << std::endl;
    }

    void setVolume(int level) {
        std::cout << "Stereo volume set to " << level << std::endl;
    }
};

// Command interface - declares the execute method
class Command {
public:
    virtual ~Command() = default;
    virtual void execute() = 0;
    virtual void undo() = 0;
};

// Concrete Command 1: Light On
class LightOnCommand : public Command {
private:
    Light& light;

public:
    LightOnCommand(Light& light) : light(light) {}

    void execute() override {
        light.on();
    }

    void undo() override {
        light.off();
    }
};

// Concrete Command 2: Light Off
class LightOffCommand : public Command {
private:
    Light& light;

public:
    LightOffCommand(Light& light) : light(light) {}

    void execute() override {
        light.off();
    }

    void undo() override {
```

```

        light.on();
    }
};

// Concrete Command 3: Stereo On with Volume
class StereoOnCommand : public Command {
private:
    Stereo& stereo;
    int previousVolume;

public:
    StereoOnCommand(Stereo& stereo) : stereo(stereo), previousVolume(0) {}

    void execute() override {
        stereo.on();
        stereo.setVolume(10);
        previousVolume = 10;
    }

    void undo() override {
        stereo.setVolume(previousVolume);
        stereo.off();
    }
};

// Concrete Command 4: Stereo Off
class StereoOffCommand : public Command {
private:
    Stereo& stereo;

public:
    StereoOffCommand(Stereo& stereo) : stereo(stereo) {}

    void execute() override {
        stereo.off();
    }

    void undo() override {
        stereo.on();
    }
};

// Invoker - asks the command to carry out the request
class RemoteControl {
private:
    std::vector<std::shared_ptr<Command>> commandHistory;
    std::stack<std::shared_ptr<Command>> undoStack;

public:
    void setCommand(std::shared_ptr<Command> command) {
        commandHistory.push_back(command);
    }

    void buttonPressed(size_t slot) {

```

```

        if (slot < commandHistory.size()) {
            commandHistory[slot]->execute();
            undoStack.push(commandHistory[slot]);
        }
    }

    void undoButtonPressed() {
        if (!undoStack.empty()) {
            undoStack.top()->undo();
            undoStack.pop();
        } else {
            std::cout << "Nothing to undo" << std::endl;
        }
    }
};

// Example 2: Macro Command
// Shows how multiple commands can be combined into one

class MacroCommand : public Command {
private:
    std::vector<std::shared_ptr<Command>> commands;

public:
    void addCommand(std::shared_ptr<Command> command) {
        commands.push_back(command);
    }

    void execute() override {
        for (const auto& command : commands) {
            command->execute();
        }
    }

    void undo() override {
        // Undo in reverse order
        for (auto it = commands.rbegin(); it != commands.rend(); ++it) {
            (*it)->undo();
        }
    }
};

// Example 3: Transaction Command
// Shows how Command can be used for transactional operations

class BankAccount {
private:
    double balance;

public:
    BankAccount(double initialBalance = 0) : balance(initialBalance) {}

    void deposit(double amount) {
        balance += amount;
    }
};

```

```
        std::cout << "Deposited $" << amount << ". New balance: $" <<
balance << std::endl;
    }

    void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
            std::cout << "Withdrew $" << amount << ". New balance: $" <<
balance << std::endl;
        } else {
            std::cout << "Insufficient funds" << std::endl;
        }
    }

    double getBalance() const {
        return balance;
    }
};

class DepositCommand : public Command {
private:
    BankAccount& account;
    double amount;

public:
    DepositCommand(BankAccount& account, double amount)
        : account(account), amount(amount) {}

    void execute() override {
        account.deposit(amount);
    }

    void undo() override {
        account.withdraw(amount);
    }
};

class WithdrawCommand : public Command {
private:
    BankAccount& account;
    double amount;
    bool success;

public:
    WithdrawCommand(BankAccount& account, double amount)
        : account(account), amount(amount), success(false) {}

    void execute() override {
        double oldBalance = account.getBalance();
        account.withdraw(amount);
        success = (account.getBalance() < oldBalance);
    }

    void undo() override {
```

```
        if (success) {
            account.deposit(amount);
        }
    }
};

int main() {
    std::cout << "=== Command Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Remote Control
    std::cout << "--- Remote Control Example ---" << std::endl;

    Light livingRoomLight;
    Stereo livingRoomStereo;

    auto lightOn = std::make_shared<LightOnCommand>(livingRoomLight);
    auto lightOff = std::make_shared<LightOffCommand>(livingRoomLight);
    auto stereoOn = std::make_shared<StereoOnCommand>(livingRoomStereo);
    auto stereoOff = std::make_shared<StereoOffCommand>(livingRoomStereo);

    RemoteControl remote;
    remote.setCommand(lightOn);
    remote.setCommand(lightOff);
    remote.setCommand(stereoOn);
    remote.setCommand(stereoOff);

    std::cout << "\nTurning on light (slot 0):" << std::endl;
    remote.buttonPressed(0);

    std::cout << "\nTurning on stereo (slot 2):" << std::endl;
    remote.buttonPressed(2);

    std::cout << "\nUndo last action:" << std::endl;
    remote.undoButtonPressed();

    std::cout << "\nTurning off light (slot 1):" << std::endl;
    remote.buttonPressed(1);

    std::cout << "\nUndo last action:" << std::endl;
    remote.undoButtonPressed();

    std::cout << std::endl;

    // Example 2: Macro Command
    std::cout << "--- Macro Command Example ---" << std::endl;

    auto partyMode = std::make_shared<MacroCommand>();
    partyMode->addCommand(lightOn);
    partyMode->addCommand(stereoOn);

    auto shutdownMode = std::make_shared<MacroCommand>();
    shutdownMode->addCommand(stereoOff);
    shutdownMode->addCommand(lightOff);
```

```

std::cout << "\nActivating party mode:" << std::endl;
partyMode->execute();

std::cout << "\nActivating shutdown mode:" << std::endl;
shutdownMode->execute();

std::cout << "\nUndo shutdown:" << std::endl;
shutdownMode->undo();

std::cout << std::endl;

// Example 3: Transaction Command
std::cout << "--- Transaction Command Example ---" << std::endl;

BankAccount myAccount(1000);

std::cout << "\nInitial balance: $" << myAccount.getBalance() <<
std::endl;

auto deposit = std::make_shared<DepositCommand>(myAccount, 500);
auto withdraw = std::make_shared<WithdrawCommand>(myAccount, 200);

std::cout << "\nExecuting deposit:" << std::endl;
deposit->execute();

std::cout << "\nExecuting withdrawal:" << std::endl;
withdraw->execute();

std::cout << "\nUndo withdrawal:" << std::endl;
withdraw->undo();

std::cout << "\nFinal balance: $" << myAccount.getBalance() <<
std::endl;

return 0;
}

```

Iterator

Description: Provides a way to **sequentially access elements** of a collection without exposing its underlying representation.

- **Why it is used:** To traverse different types of collections in a uniform way without depending on their internal structure.
- **Where it is commonly used:** Iterating over lists, trees, graphs, and custom data structures in any programming language.

Code Example (CPP)

```

/*
 * ITERATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Iterator pattern provides a way to access the elements of an
 * aggregate object sequentially without exposing its underlying
 * representation.
 * It traverses the elements of a collection without knowing its internal
 * structure.
 *
 * WHY DOES IT EXIST?
 * - When you want to access a collection's contents without exposing its
 * internal representation
 * - When you want to provide multiple ways to traverse a collection
 * - When you want to provide a uniform interface for traversing different
 * data structures
 *
 * WHEN TO USE IT?
 * - When you have a collection of objects and want to traverse it without
 * exposing its internal structure
 * - When you want to provide multiple traversal algorithms for the same
 * collection
 * - When you want to simplify the interface for accessing a complex data
 * structure
 * - Common use cases: collections, tree structures, database result sets,
 * file systems
 *
 * PROS:
 * - Single Responsibility Principle (separates traversal logic from the
 * collection)
 * - Open/Closed Principle (can add new iterators without changing the
 * collection)
 * - Provides a uniform interface for traversing different collections
 * - Can support multiple traversals simultaneously
 *
 * CONS:
 * - Can be overkill for simple collections
 * - May violate encapsulation if the iterator exposes too much about the
 * collection
 * - Can be less efficient than direct access
 * - May complicate the code for simple use cases
 *
 * REAL-WORLD ANALOGY:
 * Think of a TV remote control with channel up/down buttons. You don't
 * need to know how the TV stores channels - you just use the next/previous
 * buttons to navigate. The remote (iterator) provides a simple interface to
 * traverse the channels (collection) without knowing the internal channel
 * storage mechanism.
 */

#include <iostream>
#include <string>

```

```

#include <vector>
#include <memory>

// Iterator interface - declares the operations for traversing
template <typename T>
class Iterator {
public:
    virtual ~Iterator() = default;
    virtual bool hasNext() = 0;
    virtual T next() = 0;
};

// Aggregate interface - declares the method for creating an iterator
template <typename T>
class Aggregate {
public:
    virtual ~Aggregate() = default;
    virtual std::unique_ptr<Iterator<T>> createIterator() = 0;
};

// Concrete Aggregate - implements the aggregate interface
template <typename T>
class BookCollection : public Aggregate<T> {
private:
    std::vector<T> items;

public:
    void add(const T& item) {
        items.push_back(item);
    }

    std::unique_ptr<Iterator<T>> createIterator() override {
        return std::make_unique<BookIterator<T>>(this);
    }

    size_t size() const {
        return items.size();
    }

    const T& get(size_t index) const {
        return items[index];
    }

    // Concrete Iterator (nested class)
    class BookIterator : public Iterator<T> {
private:
        BookCollection* collection;
        size_t position = 0;

public:
        BookIterator(BookCollection* collection) : collection(collection)
        {}

        bool hasNext() override {

```

```

        return position < collection->size();
    }

    T next() override {
        if (hasNext()) {
            return collection->get(position++);
        }
        return T();
    }
};

friend class BookIterator;

// Example 2: Tree Traversal Iterator
// Shows how Iterator can be used for tree structures

template <typename T>
class TreeNode {
public:
    T data;
    std::shared_ptr<TreeNode<T>> left;
    std::shared_ptr<TreeNode<T>> right;

    TreeNode(T data) : data(data) {}
};

template <typename T>
class BinaryTreeIterator : public Iterator<T> {
private:
    std::vector<std::shared_ptr<TreeNode<T>>> stack;
    std::shared_ptr<TreeNode<T>> current;

    void pushLeft(std::shared_ptr<TreeNode<T>> node) {
        while (node) {
            stack.push_back(node);
            node = node->left;
        }
    }

public:
    BinaryTreeIterator(std::shared_ptr<TreeNode<T>> root) {
        pushLeft(root);
    }

    bool hasNext() override {
        return !stack.empty();
    }

    T next() override {
        if (!hasNext()) return T();

        auto node = stack.back();
        stack.pop_back();

```

```

        if (node->right) {
            pushLeft(node->right);
        }

        return node->data;
    }
};

// Example 3: Custom Container with Iterator
// Shows how to implement iterator for a custom data structure

template <typename T>
class CustomList {
private:
    struct Node {
        T data;
        std::shared_ptr<Node> next;
        Node(T data) : data(data), next(nullptr) {}
    };

    std::shared_ptr<Node> head;
    size_t count;

public:
    CustomList() : head(nullptr), count(0) {}

    void add(const T& item) {
        auto newNode = std::make_shared<Node>(item);
        if (!head) {
            head = newNode;
        } else {
            auto current = head;
            while (current->next) {
                current = current->next;
            }
            current->next = newNode;
        }
        count++;
    }

    std::unique_ptr<Iterator<T>> iterator() {
        return std::make_unique<ListIterator>(head);
    }

    class ListIterator : public Iterator<T> {
    private:
        std::shared_ptr<Node> current;

    public:
        ListIterator(std::shared_ptr<Node> head) : current(head) {}

        bool hasNext() override {
            return current != nullptr;
        }
    };
};

```

```

    }

    T next() override {
        if (!hasNext()) return T();
        auto data = current->data;
        current = current->next;
        return data;
    }
};

int main() {
    std::cout << "=== Iterator Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Book Collection
    std::cout << "--- Book Collection Example ---" << std::endl;

    BookCollection<std::string> books;
    books.add("Design Patterns");
    books.add("Clean Code");
    books.add("Refactoring");
    books.add("The Pragmatic Programmer");

    auto bookIterator = books.createIterator();

    std::cout << "\nIterating through books:" << std::endl;
    while (bookIterator->hasNext()) {
        std::cout << "  - " << bookIterator->next() << std::endl;
    }

    std::cout << std::endl;

    // Example 2: Binary Tree Traversal
    std::cout << "--- Binary Tree Traversal Example ---" << std::endl;

    // Build a simple binary tree
    auto root = std::make_shared<TreeNode<int>>(5);
    root->left = std::make_shared<TreeNode<int>>(3);
    root->right = std::make_shared<TreeNode<int>>(7);
    root->left->left = std::make_shared<TreeNode<int>>(1);
    root->left->right = std::make_shared<TreeNode<int>>(4);
    root->right->left = std::make_shared<TreeNode<int>>(6);
    root->right->right = std::make_shared<TreeNode<int>>(9);

    BinaryTreeIterator<int> treeIterator(root);

    std::cout << "\nIn-order traversal:" << std::endl;
    while (treeIterator.hasNext()) {
        std::cout << "  " << treeIterator.next() << std::endl;
    }

    std::cout << std::endl;

    // Example 3: Custom List Iterator

```

```

std::cout << "--- Custom List Iterator Example ---" << std::endl;

CustomList<int> numbers;
numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);
numbers.add(50);

auto listIterator = numbers.iterator();

std::cout << "\nIterating through custom list:" << std::endl;
while (listIterator->hasNext()) {
    std::cout << "  " << listIterator->next() << std::endl;
}

return 0;
}

```

Mediator

Description: Defines an object that **encapsulates how a set of objects interact**. It promotes loose coupling by preventing objects from referring to each other directly.

- **Why it is used:** To reduce the chaotic dependencies between many objects communicating directly. All communication goes through the mediator.
- **Where it is commonly used:** Chat room systems, air traffic control systems, UI dialog form coordination.

Code Example (CPP)

```

/*
 * MEDIATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Mediator pattern defines an object that encapsulates how a set of
 * objects interact.
 * It promotes loose coupling by preventing objects from referring to each
 * other explicitly.
 *
 * WHY DOES IT EXIST?
 * - When you want to reduce the chaotic dependencies between objects
 * - When you want to centralize complex communications and control logic
 * between related objects
 * - When you want to reuse the interaction logic without duplicating it
 * across multiple objects
 *
 * WHEN TO USE IT?
 * - When a set of objects communicate in well-defined but complex ways
 * - When you want to customize a behavior that's distributed between

```

```

several objects without subclassing
* - When you have many objects that need to interact with each other
* - Common use cases: chat rooms, air traffic control, GUI frameworks,
auction systems
*
* PROS:
* - Single Responsibility Principle (extracts communication logic to a
separate class)
* - Open/Closed Principle (can add new colleagues without changing the
mediator)
* - Reduces coupling between colleagues
* - Simplifies object interactions
*
* CONS:
* - The mediator can become a monolith (god object) if not carefully
designed
* - Can make the system harder to understand due to indirect communication
* - May reduce performance due to indirection
* - Can be overkill for simple interactions
*
* REAL-WORLD ANALOGY:
* Think of an air traffic control tower. Planes (colleagues) don't
communicate directly with each other - they all communicate through the
tower (mediator). The tower coordinates takeoffs, landings, and routes,
ensuring planes don't collide. This centralizes the coordination logic and
prevents planes from needing to know about each other.
*/

#include <iostream>
#include <string>
#include <memory>
#include <vector>
#include <map>

// Mediator interface - declares the communication interface
class Mediator {
public:
    virtual ~Mediator() = default;
    virtual void sendMessage(const std::string& message, const std::string&
sender) = 0;
};

// Colleague interface - declares the interface for communicating with the
mediator
class Colleague {
protected:
    std::shared_ptr<Mediator> mediator;
    std::string name;

public:
    Colleague(std::shared_ptr<Mediator> mediator, const std::string& name)
        : mediator(mediator), name(name) {}

    virtual ~Colleague() = default;

```

```

    virtual void send(const std::string& message) {
        mediator->sendMessage(message, name);
    }

    virtual void receive(const std::string& message) = 0;

    std::string getName() const {
        return name;
    }
};

// Concrete Colleague 1
class User : public Colleague {
public:
    User(std::shared_ptr<Mediator> mediator, const std::string& name)
        : Colleague(mediator, name) {}

    void receive(const std::string& message) override {
        std::cout << name << " received: " << message << std::endl;
    }
};

// Concrete Colleague 2
class Bot : public Colleague {
public:
    Bot(std::shared_ptr<Mediator> mediator, const std::string& name)
        : Colleague(mediator, name) {}

    void receive(const std::string& message) override {
        std::cout << name << " received: " << message << std::endl;
        // Bot can auto-reply
        send("Auto-reply: Thanks for your message!");
    }
};

// Concrete Mediator
class ChatRoom : public Mediator {
private:
    std::map<std::string, std::shared_ptr<Colleague>> colleagues;

public:
    void addUser(std::shared_ptr<Colleague> colleague) {
        colleagues[colleague->getName()] = colleague;
    }

    void sendMessage(const std::string& message, const std::string& sender)
    override {
        std::cout << sender << " sends: " << message << std::endl;
        for (const auto& [name, colleague] : colleagues) {
            if (name != sender) {
                colleague->receive(message);
            }
        }
    }
};

```

```

    }
};

// Example 2: Air Traffic Control
// Shows how Mediator can coordinate complex interactions

class Aircraft {
protected:
    std::shared_ptr<Mediator> controlTower;
    std::string flightNumber;
    int altitude;
    int speed;

public:
    Aircraft(std::shared_ptr<Mediator> tower, const std::string&
flightNumber)
        : controlTower(tower), flightNumber(flightNumber), altitude(0),
speed(0) {}

    virtual ~Aircraft() = default;

    void requestTakeoff() {
        std::cout << flightNumber << " requesting takeoff clearance..." <<
std::endl;
        // In real implementation, would send to mediator
    }

    void requestLanding() {
        std::cout << flightNumber << " requesting landing clearance..." <<
std::endl;
        // In real implementation, would send to mediator
    }

    void setAltitude(int alt) {
        altitude = alt;
        std::cout << flightNumber << " at altitude " << altitude << " feet"
<< std::endl;
    }

    std::string getFlightNumber() const {
        return flightNumber;
    }
};

class ControlTower : public Mediator {
private:
    std::map<std::string, std::shared_ptr<Aircraft>> aircrafts;

public:
    void registerAircraft(std::shared_ptr<Aircraft> aircraft) {
        aircrafts[aircraft->getFlightNumber()] = aircraft;
    }

    void sendMessage(const std::string& message, const std::string& sender)

```

```

override {
    // Simplified for demo
    std::cout << "Control Tower: " << message << " from " << sender <<
std::endl;
}

void grantTakeoff(const std::string& flightNumber) {
    std::cout << "Control Tower: Takeoff granted to " << flightNumber
<< std::endl;
    if (aircrafts.find(flightNumber) != aircrafts.end()) {
        aircrafts[flightNumber]->setAltitude(10000);
    }
}

void grantLanding(const std::string& flightNumber) {
    std::cout << "Control Tower: Landing granted to " << flightNumber
<< std::endl;
    if (aircrafts.find(flightNumber) != aircrafts.end()) {
        aircrafts[flightNumber]->setAltitude(0);
    }
}
};

// Example 3: Smart Home System
// Shows how Mediator can coordinate home automation

class SmartDevice {
protected:
    std::shared_ptr<Mediator> homeHub;
    std::string deviceName;
    bool isOn;

public:
    SmartDevice(std::shared_ptr<Mediator> hub, const std::string& name)
        : homeHub(hub), deviceName(name), isOn(false) {}

    virtual ~SmartDevice() = default;

    void turnOn() {
        isOn = true;
        std::cout << deviceName << " turned ON" << std::endl;
        homeHub->sendMessage(deviceName + " turned ON", deviceName);
    }

    void turnOff() {
        isOn = false;
        std::cout << deviceName << " turned OFF" << std::endl;
        homeHub->sendMessage(deviceName + " turned OFF", deviceName);
    }

    std::string getName() const {
        return deviceName;
    }
}

```

```

        bool getStatus() const {
            return isOn;
        }
    };

    class SmartLight : public SmartDevice {
    public:
        SmartLight(std::shared_ptr<Mediator> hub, const std::string& name)
            : SmartDevice(hub, name) {}
    };

    class SmartThermostat : public SmartDevice {
    private:
        int temperature;

    public:
        SmartThermostat(std::shared_ptr<Mediator> hub, const std::string& name)
            : SmartDevice(hub, name), temperature(70) {}

        void setTemperature(int temp) {
            temperature = temp;
            std::cout << getName() << " set to " << temperature << "°F" <<
std::endl;
        }

        int getTemperature() const {
            return temperature;
        }
    };

    class HomeHub : public Mediator {
    private:
        std::map<std::string, std::shared_ptr<SmartDevice>> devices;

    public:
        void addDevice(std::shared_ptr<SmartDevice> device) {
            devices[device->getName()] = device;
        }

        void sendMessage(const std::string& message, const std::string& sender)
override {
            std::cout << "Home Hub: " << message << std::endl;

            // Example logic: if thermostat turns on, turn on lights
            if (sender.find("Thermostat") != std::string::npos &&
message.find("turned ON") != std::string::npos) {
                for (const auto& [name, device] : devices) {
                    if (name.find("Light") != std::string::npos && !device-
>getStatus()) {
                        device->turnOn();
                    }
                }
            }
        }
    };

```

```

void activateAwayMode() {
    std::cout << "Home Hub: Activating away mode..." << std::endl;
    for (const auto& [name, device] : devices) {
        if (device->getStatus()) {
            device->turnOff();
        }
    }
}

void activateHomeMode() {
    std::cout << "Home Hub: Activating home mode..." << std::endl;
    for (const auto& [name, device] : devices) {
        if (name.find("Light") != std::string::npos) {
            device->turnOn();
        }
    }
}

};

int main() {
    std::cout << "=== Mediator Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Chat Room
    std::cout << "--- Chat Room Example ---" << std::endl;

    auto chatRoom = std::make_shared<ChatRoom>();

    auto user1 = std::make_shared<User>(chatRoom, "Alice");
    auto user2 = std::make_shared<User>(chatRoom, "Bob");
    auto bot = std::make_shared<Bot>(chatRoom, "HelperBot");

    chatRoom->addUser(user1);
    chatRoom->addUser(user2);
    chatRoom->addUser(bot);

    std::cout << "\nAlice sends a message:" << std::endl;
    user1->send("Hello everyone!");

    std::cout << "\nBob sends a message:" << std::endl;
    user2->send("Hi Alice!");

    std::cout << std::endl;

    // Example 2: Air Traffic Control
    std::cout << "--- Air Traffic Control Example ---" << std::endl;

    auto controlTower = std::make_shared<ControlTower>();

    auto flight1 = std::make_shared<Aircraft>(controlTower, "AA123");
    auto flight2 = std::make_shared<Aircraft>(controlTower, "UA456");

    controlTower->registerAircraft(flight1);
    controlTower->registerAircraft(flight2);

```

```

std::cout << "\nFlight AA123 requests takeoff:" << std::endl;
controlTower->grantTakeoff("AA123");

std::cout << "\nFlight UA456 requests landing:" << std::endl;
controlTower->grantLanding("UA456");

std::cout << std::endl;

// Example 3: Smart Home System
std::cout << "--- Smart Home System Example ---" << std::endl;

auto homeHub = std::make_shared<HomeHub>();

auto livingRoomLight = std::make_shared<SmartLight>(homeHub, "Living
Room Light");
auto bedroomLight = std::make_shared<SmartLight>(homeHub, "Bedroom
Light");
auto thermostat = std::make_shared<SmartThermostat>(homeHub, "Main
Thermostat");

homeHub->addDevice(livingRoomLight);
homeHub->addDevice(bedroomLight);
homeHub->addDevice(thermostat);

std::cout << "\nTurning on thermostat (should trigger lights):" <<
std::endl;
thermostat->turnOn();

std::cout << "\nActivating away mode:" << std::endl;
homeHub->activateAwayMode();

std::cout << "\nActivating home mode:" << std::endl;
homeHub->activateHomeMode();

return 0;
}

```

Memento

Description: Captures and externalizes an object's **internal state** so it can be restored later, without violating encapsulation.

- **Why it is used:** To implement undo/redo functionality by saving snapshots of an object's state.
- **Where it is commonly used:** Text editors (undo history), game save states, transactional systems.

Code Example (CPP)

```

/*
 * MEMENTO DESIGN PATTERN

```

```

*
* WHAT IS IT?
* The Memento pattern lets you save and restore the previous state of an
object without revealing the details of its implementation.
* It provides the ability to restore an object to its previous state
(undo).
*
* WHY DOES IT EXIST?
* - When you need to implement undo/redo functionality
* - When you want to create snapshots of an object's state
* - When direct access to the object's internal representation would
violate encapsulation
*
* WHEN TO USE IT?
* - When you need to save and restore the state of an object without
exposing its internal structure
* - When you want to implement undo/redo operations
* - When you need to maintain a history of states
* - Common use cases: text editors (undo/redo), game states, database
transactions, configuration management
*
* PROS:
* - Preserves encapsulation (doesn't expose internal state)
* - Simplifies the originator (doesn't need to manage state history)
* - Can provide snapshots at different times
* - Supports undo/redo functionality
*
* CONS:
* - Can be expensive if the originator has large state
* - May consume significant memory if many mementos are stored
* - Caretaker must track the originator's lifecycle
* - Can be complex to implement for complex objects
*
* REAL-WORLD ANALOGY:
* Think of a game save system. When you save a game, the game creates a
snapshot of your current state (position, health, inventory). This snapshot
(memento) is stored separately. Later, you can restore the game to that
exact state by loading the snapshot. The game doesn't need to know how the
snapshot is stored, just that it can be restored.
*/

#include <iostream>
#include <string>
#include <memory>
#include <vector>
#include <ctime>

// Memento - stores the internal state of the Originator
class TextEditorMemento {
private:
    std::string content;
    std::string timestamp;

public:

```

```

    TextEditorMemento(const std::string& content)
        : content(content), timestamp(getCurrentTime()) {}

    std::string getContent() const {
        return content;
    }

    std::string getTimestamp() const {
        return timestamp;
    }

private:
    static std::string getCurrentTime() {
        time_t now = time(0);
        char* dt = ctime(&now);
        std::string timeStr(dt);
        timeStr.pop_back(); // Remove newline
        return timeStr;
    }
};

// Originator - creates a memento containing a snapshot of its current
state
class TextEditor {
private:
    std::string content;

public:
    TextEditor() : content("") {}

    void write(const std::string& text) {
        content += text;
        std::cout << "Text: \"" << text << "\" added" << std::endl;
    }

    void setContent(const std::string& newContent) {
        content = newContent;
    }

    std::string getContent() const {
        return content;
    }

    std::unique_ptr<TextEditorMemento> save() {
        std::cout << "Saving current state..." << std::endl;
        return std::make_unique<TextEditorMemento>(content);
    }

    void restore(const TextEditorMemento& memento) {
        content = memento.getContent();
        std::cout << "Restored state from " << memento.getTimestamp() <<
std::endl;
        std::cout << "Current content: \"" << content << "\"" << std::endl;
    }
}

```

```
};

// Caretaker - manages mementos, never operates on their contents
class History {
private:
    std::vector<std::unique_ptr<TextEditorMemento>> mementos;

public:
    void push(std::unique_ptr<TextEditorMemento> memento) {
        mementos.push_back(std::move(memento));
    }

    std::unique_ptr<TextEditorMemento> pop() {
        if (mementos.empty()) {
            return nullptr;
        }
        auto memento = std::move(mementos.back());
        mementos.pop_back();
        return memento;
    }

    size_t size() const {
        return mementos.size();
    }
};

// Example 2: Game State Memento
// Shows how Memento can be used for game saves

class GameStateMemento {
private:
    int level;
    int health;
    int score;
    std::string position;

public:
    GameStateMemento(int level, int health, int score, const std::string&
position)
        : level(level), health(health), score(score), position(position) {}

    int getLevel() const { return level; }
    int getHealth() const { return health; }
    int getScore() const { return score; }
    std::string getPosition() const { return position; }
};

class GameCharacter {
private:
    int level;
    int health;
    int score;
    std::string position;
```

```
public:
    GameCharacter() : level(1), health(100), score(0), position("Start") {}

    void moveTo(const std::string& newPosition) {
        position = newPosition;
        std::cout << "Moved to " << position << std::endl;
    }

    void takeDamage(int damage) {
        health -= damage;
        if (health < 0) health = 0;
        std::cout << "Took " << damage << " damage. Health: " << health <<
std::endl;
    }

    void gainScore(int points) {
        score += points;
        std::cout << "Gained " << points << " points. Score: " << score <<
std::endl;
    }

    void levelUp() {
        level++;
        health = 100;
        std::cout << "Level up! Now at level " << level << std::endl;
    }

    void displayStatus() const {
        std::cout << "=== Character Status ===" << std::endl;
        std::cout << "Level: " << level << std::endl;
        std::cout << "Health: " << health << std::endl;
        std::cout << "Score: " << score << std::endl;
        std::cout << "Position: " << position << std::endl;
        std::cout << "===== " << std::endl;
    }

    std::unique_ptr<GameStateMemento> saveState() {
        std::cout << "Saving game state..." << std::endl;
        return std::make_unique<GameStateMemento>(level, health, score,
position);
    }

    void restoreState(const GameStateMemento& memento) {
        level = memento.getLevel();
        health = memento.getHealth();
        score = memento.getScore();
        position = memento.getPosition();
        std::cout << "Game state restored!" << std::endl;
        displayStatus();
    }
};

class GameSaveManager {
private:
```

```

        std::vector<std::unique_ptr<GameStateMemento>> saves;

public:
    void createSave(std::unique_ptr<GameStateMemento> save) {
        saves.push_back(std::move(save));
        std::cout << "Save created. Total saves: " << saves.size() <<
std::endl;
    }

    std::unique_ptr<GameStateMemento> loadSave(int index) {
        if (index >= 0 && index < saves.size()) {
            std::cout << "Loading save " << (index + 1) << "..." <<
std::endl;
            return std::move(saves[index]);
        }
        return nullptr;
    }

    int getSaveCount() const {
        return saves.size();
    }
};

// Example 3: Configuration Memento
// Shows how Memento can be used for configuration management

class ConfigurationMemento {
private:
    std::string theme;
    int fontSize;
    bool notificationsEnabled;

public:
    ConfigurationMemento(const std::string& theme, int fontSize, bool
notifications)
        : theme(theme), fontSize(fontSize),
notificationsEnabled(notifications) {}

    std::string getTheme() const { return theme; }
    int getFontSize() const { return fontSize; }
    bool getNotificationsEnabled() const { return notificationsEnabled; }
};

class ApplicationSettings {
private:
    std::string theme;
    int fontSize;
    bool notificationsEnabled;

public:
    ApplicationSettings() : theme("light"), fontSize(12),
notificationsEnabled(true) {}

    void setTheme(const std::string& newTheme) {

```

```

        theme = newTheme;
        std::cout << "Theme changed to " << theme << std::endl;
    }

    void setFontSize(int size) {
        fontSize = size;
        std::cout << "Font size set to " << fontSize << std::endl;
    }

    void setNotificationsEnabled(bool enabled) {
        notificationsEnabled = enabled;
        std::cout << "Notifications " << (enabled ? "enabled" : "disabled")
<< std::endl;
    }

    void displaySettings() const {
        std::cout << "=== Current Settings ===" << std::endl;
        std::cout << "Theme: " << theme << std::endl;
        std::cout << "Font Size: " << fontSize << std::endl;
        std::cout << "Notifications: " << (notificationsEnabled ? "Enabled"
: "Disabled") << std::endl;
        std::cout << "===== " << std::endl;
    }

    std::unique_ptr<ConfigurationMemento> saveConfiguration() {
        return std::make_unique<ConfigurationMemento>(theme, fontSize,
notificationsEnabled);
    }

    void restoreConfiguration(const ConfigurationMemento& memento) {
        theme = memento.getTheme();
        fontSize = memento.getFontSize();
        notificationsEnabled = memento.getNotificationsEnabled();
        std::cout << "Configuration restored!" << std::endl;
        displaySettings();
    }
};

int main() {
    std::cout << "=== Memento Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Text Editor with Undo
    std::cout << "--- Text Editor with Undo ---" << std::endl;

    TextEditor editor;
    History history;

    editor.write("Hello ");
    history.push(editor.save());

    editor.write("World ");
    history.push(editor.save());

    editor.write("!");

```

```
history.push(editor.save());

std::cout << "\nCurrent content: \"" << editor.getContent() << "\"" <<
std::endl;
std::cout << "History size: " << history.size() << std::endl;

std::cout << "\nPerforming undo..." << std::endl;
if (auto memento = history.pop()) {
    editor.restore(*memento);
}

std::cout << "\nPerforming undo..." << std::endl;
if (auto memento = history.pop()) {
    editor.restore(*memento);
}

std::cout << std::endl;

// Example 2: Game Save System
std::cout << "--- Game Save System ---" << std::endl;

GameCharacter player;
GameSaveManager saveManager;

std::cout << "\nInitial state:" << std::endl;
player.displayStatus();

std::cout << "\nPlaying game..." << std::endl;
player.moveTo("Forest");
player.gainScore(100);
player.takeDamage(10);

std::cout << "\nCreating save 1..." << std::endl;
saveManager.createSave(player.saveState());

std::cout << "\nContinuing game..." << std::endl;
player.moveTo("Castle");
player.levelUp();
player.gainScore(500);
player.takeDamage(30);

std::cout << "\nCreating save 2..." << std::endl;
saveManager.createSave(player.saveState());

std::cout << "\nCurrent state:" << std::endl;
player.displayStatus();

std::cout << "\nLoading save 1..." << std::endl;
if (auto save = saveManager.loadSave(0)) {
    player.restoreState(*save);
}

std::cout << std::endl;
```

```
// Example 3: Configuration Management
std::cout << "--- Configuration Management ---" << std::endl;

ApplicationSettings settings;

std::cout << "\nDefault settings:" << std::endl;
settings.displaySettings();

auto originalConfig = settings.saveConfiguration();

std::cout << "\nChanging settings..." << std::endl;
settings.setTheme("dark");
settings.setFontSize(14);
settings.setNotificationsEnabled(false);

std::cout << "\nNew settings:" << std::endl;
settings.displaySettings();

std::cout << "\nRestoring original settings..." << std::endl;
settings.restoreConfiguration(*originalConfig);

return 0;
}
```

Observer

Description: Defines a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.

- **Why it is used:** To implement event-driven systems where multiple parts of an application need to react to changes in a shared object.
- **Where it is commonly used:** Event listeners, pub/sub systems, real-time dashboards, MVC architecture (model notifying views).

Code Example (CPP)

```
/*
 * OBSERVER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Observer pattern defines a subscription mechanism to notify multiple
 * objects about any events that happen to the object they are observing.
 * It establishes a one-to-many dependency between objects so that when one
 * object changes state, all its dependents are notified.
 *
 * WHY DOES IT EXIST?
 * - When you need to notify multiple objects about changes to a single
 * object
 * - When you want to decouple the subject from its observers
 * - When you want to broadcast changes to multiple subscribers dynamically
 */
```

```

*
* WHEN TO USE IT?
* - When changes to one object require changing other objects
* - When the set of objects that need to be notified is unknown or dynamic
* - When you want to implement a publish-subscribe model
* - Common use cases: GUI event handling, news feeds, social media
notifications, stock price updates
*
* PROS:
* - Open/Closed Principle (can add new observers without changing the
subject)
* - Establishes loose coupling between subject and observers
* - Can broadcast communications to multiple objects
* - Supports dynamic subscription/unsubscription
*
* CONS:
* - Can lead to unexpected updates if observers are not careful
* - Can cause performance issues with many observers
* - Debugging can be difficult due to indirect calls
* - May cause memory leaks if observers don't unsubscribe
*
* REAL-WORLD ANALOGY:
* Think of a YouTube channel (subject) and subscribers (observers). When
the channel uploads a new video, all subscribers get notified. Subscribers
can subscribe or unsubscribe at any time. The channel doesn't need to know
who is subscribed - it just broadcasts notifications to all current
subscribers.
*/

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <functional>

// Observer interface - declares the update method
class Observer {
public:
    virtual ~Observer() = default;
    virtual void update(const std::string& message) = 0;
};

// Subject interface - declares methods for attaching, detaching, and
notifying observers
class Subject {
protected:
    std::vector<std::shared_ptr<Observer>> observers;

public:
    virtual ~Subject() = default;

    void attach(std::shared_ptr<Observer> observer) {
        observers.push_back(observer);
        std::cout << "Attached an observer" << std::endl;
    }
};

```

```

    }

    void detach(std::shared_ptr<Observer> observer) {
        auto it = std::find(observers.begin(), observers.end(), observer);
        if (it != observers.end()) {
            observers.erase(it);
            std::cout << "Detached an observer" << std::endl;
        }
    }

    void notify(const std::string& message) {
        std::cout << "Notifying observers..." << std::endl;
        for (const auto& observer : observers) {
            observer->update(message);
        }
    }
};

// Concrete Subject
class YouTubeChannel : public Subject {
private:
    std::string channelName;

public:
    YouTubeChannel(const std::string& name) : channelName(name) {}

    void uploadVideo(const std::string& videoTitle) {
        std::cout << "\n" << channelName << " uploaded: " << videoTitle <<
std::endl;
        notify("New video: " + videoTitle);
    }

    void goLive(const std::string& streamTitle) {
        std::cout << "\n" << channelName << " is live: " << streamTitle <<
std::endl;
        notify("Live stream started: " + streamTitle);
    }
};

// Concrete Observer 1
class Subscriber : public Observer {
private:
    std::string name;

public:
    Subscriber(const std::string& name) : name(name) {}

    void update(const std::string& message) override {
        std::cout << "  " << name << " received notification: " << message
<< std::endl;
    }
};

// Concrete Observer 2

```

```

class NotificationService : public Observer {
public:
    void update(const std::string& message) override {
        std::cout << " [NotificationService] Push notification sent: " <<
message << std::endl;
    }
};

// Example 2: Stock Market
// Shows how Observer can be used for real-time updates

class StockObserver {
public:
    virtual ~StockObserver() = default;
    virtual void onStockPriceChange(const std::string& symbol, double
price) = 0;
};

class StockMarket {
private:
    std::vector<std::shared_ptr<StockObserver>> observers;
    std::map<std::string, double> stockPrices;

public:
    void addObserver(std::shared_ptr<StockObserver> observer) {
        observers.push_back(observer);
    }

    void removeObserver(std::shared_ptr<StockObserver> observer) {
        auto it = std::find(observers.begin(), observers.end(), observer);
        if (it != observers.end()) {
            observers.erase(it);
        }
    }

    void updateStockPrice(const std::string& symbol, double price) {
        stockPrices[symbol] = price;
        std::cout << "\n" << symbol << " price updated to $" << price <<
std::endl;
        notifyObservers(symbol, price);
    }

private:
    void notifyObservers(const std::string& symbol, double price) {
        for (const auto& observer : observers) {
            observer->onStockPriceChange(symbol, price);
        }
    }
};

class Investor : public StockObserver {
private:
    std::string name;
    std::map<std::string, int> portfolio;

```

```

public:
    Investor(const std::string& name) : name(name) {}

    void buyStock(const std::string& symbol, int shares) {
        portfolio[symbol] += shares;
        std::cout << name << " bought " << shares << " shares of " <<
symbol << std::endl;
    }

    void onStockPriceChange(const std::string& symbol, double price)
override {
        if (portfolio.find(symbol) != portfolio.end()) {
            std::cout << " " << name << " owns " << portfolio[symbol]
                << " shares of " << symbol << " (current price: $" <<
price << ")" << std::endl;
        }
    }
};

class TradingBot : public StockObserver {
public:
    void onStockPriceChange(const std::string& symbol, double price)
override {
        std::cout << " [TradingBot] Analyzing " << symbol << " at $" <<
price << std::endl;
        if (price > 100) {
            std::cout << " [TradingBot] Recommendation: SELL" <<
std::endl;
        } else if (price < 50) {
            std::cout << " [TradingBot] Recommendation: BUY" << std::endl;
        }
    }
};

// Example 3: Weather Station
// Shows how Observer can be used for sensor data

class WeatherObserver {
public:
    virtual ~WeatherObserver() = default;
    virtual void onWeatherUpdate(float temperature, float humidity, float
pressure) = 0;
};

class WeatherStation {
private:
    std::vector<std::shared_ptr<WeatherObserver>> observers;
    float temperature;
    float humidity;
    float pressure;

public:
    WeatherStation() : temperature(0), humidity(0), pressure(0) {}

```

```

void addObserver(std::shared_ptr<WeatherObserver> observer) {
    observers.push_back(observer);
}

void removeObserver(std::shared_ptr<WeatherObserver> observer) {
    auto it = std::find(observers.begin(), observers.end(), observer);
    if (it != observers.end()) {
        observers.erase(it);
    }
}

void setMeasurements(float temp, float hum, float press) {
    temperature = temp;
    humidity = hum;
    pressure = press;
    std::cout << "\nWeather updated: " << temperature << "°C, "
                << humidity << "% humidity, " << pressure << " hPa" <<
std::endl;
    notifyObservers();
}

private:
void notifyObservers() {
    for (const auto& observer : observers) {
        observer->onWeatherUpdate(temperature, humidity, pressure);
    }
}

};

class DisplayDevice : public WeatherObserver {
private:
    std::string deviceName;

public:
    DisplayDevice(const std::string& name) : deviceName(name) {}

    void onWeatherUpdate(float temperature, float humidity, float pressure)
    override {
        std::cout << " [" << deviceName << "] Displaying: " << temperature
        << "°C, "
                << humidity << "%, " << pressure << " hPa" << std::endl;
    }
};

class AlertSystem : public WeatherObserver {
public:
    void onWeatherUpdate(float temperature, float humidity, float pressure)
    override {
        std::cout << " [AlertSystem] Checking conditions..." << std::endl;
        if (temperature > 35) {
            std::cout << " [AlertSystem] WARNING: High temperature!" <<
std::endl;
        }
    }
}

```

```
        if (humidity > 80) {
            std::cout << "    [AlertSystem] WARNING: High humidity!" <<
std::endl;
        }
        if (pressure < 980) {
            std::cout << "    [AlertSystem] WARNING: Low pressure (storm
coming)!" << std::endl;
        }
    }
};

int main() {
    std::cout << "=== Observer Pattern Demo ===" << std::endl << std::endl;

    // Example 1: YouTube Channel
    std::cout << "--- YouTube Channel Example ---" << std::endl;

    auto channel = std::make_shared<YouTubeChannel>("Tech Channel");

    auto subscriber1 = std::make_shared<Subscriber>("Alice");
    auto subscriber2 = std::make_shared<Subscriber>("Bob");
    auto notificationService = std::make_shared<NotificationService>();

    channel->attach(subscriber1);
    channel->attach(subscriber2);
    channel->attach(notificationService);

    channel->uploadVideo("Design Patterns Tutorial");

    channel->detach(subscriber2);

    channel->goLive("Live Q&A Session");

    std::cout << std::endl;

    // Example 2: Stock Market
    std::cout << "--- Stock Market Example ---" << std::endl;

    StockMarket market;

    auto investor1 = std::make_shared<Investor>("John");
    auto investor2 = std::make_shared<Investor>("Jane");
    auto tradingBot = std::make_shared<TradingBot>();

    market.addObserver(investor1);
    market.addObserver(investor2);
    market.addObserver(tradingBot);

    investor1->buyStock("AAPL", 100);
    investor2->buyStock("GOOGL", 50);

    market.updateStockPrice("AAPL", 150.50);
    market.updateStockPrice("GOOGL", 45.00);
    market.updateStockPrice("TSLA", 200.00);
```

```

std::cout << std::endl;

// Example 3: Weather Station
std::cout << "--- Weather Station Example ---" << std::endl;

WeatherStation station;

auto phoneDisplay = std::make_shared<DisplayDevice>("Phone");
auto watchDisplay = std::make_shared<DisplayDevice>("Smart Watch");
auto alertSystem = std::make_shared<AlertSystem>();

station.addObserver(phoneDisplay);
station.addObserver(watchDisplay);
station.addObserver(alertSystem);

station.setMeasurements(25.0, 60.0, 1013.0);
station.setMeasurements(38.0, 85.0, 975.0);
station.setMeasurements(20.0, 50.0, 1020.0);

return 0;
}

```

State

Description: Allows an object to **alter its behavior when its internal state changes**. The object will appear to change its class.

- **Why it is used:** To replace complex conditional statements (*if/switch*) that depend on the object's state with dedicated state classes.
- **Where it is commonly used:** Traffic light systems, vending machines, order status workflows, game character states.

Code Example (CPP)

```

/*
 * STATE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The State pattern allows an object to alter its behavior when its
 * internal state changes.
 * The object appears to change its class by delegating state-specific
 * behavior to different state objects.
 *
 * WHY DOES IT EXIST?
 * - When an object's behavior depends on its state and must change at
 * runtime
 * - When you have large conditional statements that switch between states
 * - When you want to avoid monolithic conditional logic
 */

```

```

* WHEN TO USE IT?
* - When an object has many states and its behavior changes based on those
states
* - When you have large conditional operators that switch between states
* - When you want to eliminate conditional logic from the main class
* - Common use cases: game characters, vending machines, audio/video
players, document editors
*
* PROS:
* - Single Responsibility Principle (each state is a separate class)
* - Open/Closed Principle (can add new states without changing existing
code)
* - Eliminates conditional logic from the context
* - Makes state transitions explicit
*
* CONS:
* - Can increase the number of classes in the system
* - State transitions can be complex to manage
* - May make the code more complex for simple scenarios
* - Can lead to tight coupling between states
*
* REAL-WORLD ANALOGY:
* Think of a vending machine. It has different states: "No Coin", "Has
Coin", "Dispensing", "Out of Stock". When you insert a coin, it transitions
from "No Coin" to "Has Coin". When you press a button, it transitions to
"Dispensing". The machine's behavior changes based on its current state,
but the machine itself doesn't change - it just delegates to the
appropriate state object.
*/

#include <iostream>
#include <string>
#include <memory>

// State interface - declares the state-specific methods
class State {
public:
    virtual ~State() = default;
    virtual void insertCoin() = 0;
    virtual void ejectCoin() = 0;
    virtual void pressButton() = 0;
    virtual void dispense() = 0;
};

// Context - maintains a reference to the current state
class VendingMachine {
private:
    std::unique_ptr<State> currentState;
    int coinCount;
    int stockCount;

public:
    VendingMachine(int initialStock) : coinCount(0),
stockCount(initialStock) {

```

```

        // Initial state depends on stock
        if (initialStock > 0) {
            currentState = std::make_unique<NoCoinState>(this);
        } else {
            currentState = std::make_unique<SoldOutState>(this);
        }
    }

    void setState(std::unique_ptr<State> state) {
        currentState = std::move(state);
    }

    void insertCoin() {
        currentState->insertCoin();
    }

    void ejectCoin() {
        currentState->ejectCoin();
    }

    void pressButton() {
        currentState->pressButton();
    }

    void dispense() {
        currentState->dispense();
    }

    void addCoin() {
        coinCount++;
        std::cout << "Coin inserted. Total coins: " << coinCount <<
std::endl;
    }

    void returnCoin() {
        if (coinCount > 0) {
            coinCount--;
            std::cout << "Coin returned. Total coins: " << coinCount <<
std::endl;
        }
    }

    void releaseProduct() {
        if (stockCount > 0) {
            stockCount--;
            coinCount--;
            std::cout << "Product dispensed. Stock: " << stockCount << ",
Coins: " << coinCount << std::endl;
        }
    }

    int getCoinCount() const {
        return coinCount;
    }

```

```
    int getStockCount() const {
        return stockCount;
    }
};

// Concrete State 1: No Coin
class NoCoinState : public State {
private:
    VendingMachine* machine;

public:
    NoCoinState(VendingMachine* machine) : machine(machine) {}

    void insertCoin() override {
        std::cout << "Coin inserted..." << std::endl;
        machine->addCoin();
        machine->setState(std::make_unique<HasCoinState>(machine));
    }

    void ejectCoin() override {
        std::cout << "No coin to eject" << std::endl;
    }

    void pressButton() override {
        std::cout << "Please insert a coin first" << std::endl;
    }

    void dispense() override {
        std::cout << "Please insert a coin first" << std::endl;
    }
};

// Concrete State 2: Has Coin
class HasCoinState : public State {
private:
    VendingMachine* machine;

public:
    HasCoinState(VendingMachine* machine) : machine(machine) {}

    void insertCoin() override {
        std::cout << "Coin already inserted" << std::endl;
    }

    void ejectCoin() override {
        std::cout << "Coin ejected" << std::endl;
        machine->returnCoin();
        machine->setState(std::make_unique<NoCoinState>(machine));
    }

    void pressButton() override {
        std::cout << "Button pressed..." << std::endl;
        if (machine->getStockCount() > 0) {
```

```
        machine->setState(std::make_unique<DispensingState>(machine));
        machine->dispense();
    } else {
        std::cout << "Out of stock" << std::endl;
        machine->setState(std::make_unique<SoldOutState>(machine));
    }
}

void dispense() override {
    std::cout << "Press button to dispense" << std::endl;
}

};

// Concrete State 3: Dispensing
class DispensingState : public State {
private:
    VendingMachine* machine;

public:
    DispensingState(VendingMachine* machine) : machine(machine) {}

    void insertCoin() override {
        std::cout << "Please wait, dispensing..." << std::endl;
    }

    void ejectCoin() override {
        std::cout << "Cannot eject during dispensing" << std::endl;
    }

    void pressButton() override {
        std::cout << "Already dispensing..." << std::endl;
    }

    void dispense() override {
        machine->releaseProduct();
        if (machine->getStockCount() > 0) {
            machine->setState(std::make_unique<NoCoinState>(machine));
        } else {
            machine->setState(std::make_unique<SoldOutState>(machine));
        }
    }
};

// Concrete State 4: Sold Out
class SoldOutState : public State {
private:
    VendingMachine* machine;

public:
    SoldOutState(VendingMachine* machine) : machine(machine) {}

    void insertCoin() override {
        std::cout << "Cannot insert coin, machine is sold out" <<
std::endl;
```

```
}

void ejectCoin() override {
    std::cout << "Cannot eject, no coin inserted" << std::endl;
}

void pressButton() override {
    std::cout << "Machine is sold out" << std::endl;
}

void dispense() override {
    std::cout << "Machine is sold out" << std::endl;
}
};

// Example 2: Audio Player States
// Shows how State can be used for media players

class AudioPlayerState {
public:
    virtual ~AudioPlayerState() = default;
    virtual void clickPlay() = 0;
    virtual void clickStop() = 0;
    virtual void clickNext() = 0;
    virtual void clickPrevious() = 0;
};

class AudioPlayer {
private:
    std::unique_ptr<AudioPlayerState> currentState;
    bool isPlaying;
    int currentTrack;
    int totalTracks;

public:
    AudioPlayer(int tracks) : isPlaying(false), currentTrack(1),
totalTracks(tracks) {
        currentState = std::make_unique<StoppedState>(this);
    }

    void setState(std::unique_ptr<AudioPlayerState> state) {
        currentState = std::move(state);
    }

    void clickPlay() {
        currentState->clickPlay();
    }

    void clickStop() {
        currentState->clickStop();
    }

    void clickNext() {
        currentState->clickNext();
    }
}
```

```
}

void clickPrevious() {
    currentState->clickPrevious();
}

void startPlaying() {
    isPlaying = true;
    std::cout << "Started playing track " << currentTrack << std::endl;
}

void stopPlaying() {
    isPlaying = false;
    std::cout << "Stopped playing" << std::endl;
}

void nextTrack() {
    currentTrack = (currentTrack % totalTracks) + 1;
    std::cout << "Moved to track " << currentTrack << std::endl;
}

void previousTrack() {
    currentTrack = (currentTrack == 1) ? totalTracks : currentTrack -
1;
    std::cout << "Moved to track " << currentTrack << std::endl;
}

bool getIsPlaying() const {
    return isPlaying;
}

};

class PlayingState : public AudioPlayerState {
private:
    AudioPlayer* player;

public:
    PlayingState(AudioPlayer* player) : player(player) {}

    void clickPlay() override {
        std::cout << "Already playing" << std::endl;
    }

    void clickStop() override {
        std::cout << "Stopping playback..." << std::endl;
        player->stopPlaying();
        player->setState(std::make_unique<StoppedState>(player));
    }

    void clickNext() override {
        std::cout << "Skipping to next track..." << std::endl;
        player->nextTrack();
    }
}
```

```
void clickPrevious() override {
    std::cout << "Going to previous track..." << std::endl;
    player->previousTrack();
}

};

class StoppedState : public AudioPlayerState {
private:
    AudioPlayer* player;

public:
    StoppedState(AudioPlayer* player) : player(player) {}

    void clickPlay() override {
        std::cout << "Starting playback..." << std::endl;
        player->startPlaying();
        player->setState(std::make_unique<PlayingState>(player));
    }

    void clickStop() override {
        std::cout << "Already stopped" << std::endl;
    }

    void clickNext() override {
        std::cout << "Skipping to next track..." << std::endl;
        player->nextTrack();
    }

    void clickPrevious() override {
        std::cout << "Going to previous track..." << std::endl;
        player->previousTrack();
    }
};

int main() {
    std::cout << "=== State Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Vending Machine
    std::cout << "--- Vending Machine Example ---" << std::endl;

    VendingMachine machine(5); // 5 items in stock

    std::cout << "\nAttempt to press button without coin:" << std::endl;
    machine.pressButton();

    std::cout << "\nInsert coin:" << std::endl;
    machine.insertCoin();

    std::cout << "\nPress button:" << std::endl;
    machine.pressButton();

    std::cout << "\nInsert another coin:" << std::endl;
    machine.insertCoin();
}
```

```

std::cout << "\nPress button:" << std::endl;
machine.pressButton();

std::cout << "\nEject coin:" << std::endl;
machine.ejectCoin();

std::cout << std::endl;

// Example 2: Audio Player
std::cout << "--- Audio Player Example ---" << std::endl;

AudioPlayer player(10); // 10 tracks

std::cout << "\nClick play (stopped state):" << std::endl;
player.clickPlay();

std::cout << "\nClick next:" << std::endl;
player.clickNext();

std::cout << "\nClick stop:" << std::endl;
player.clickStop();

std::cout << "\nClick previous:" << std::endl;
player.clickPrevious();

std::cout << "\nClick play again:" << std::endl;
player.clickPlay();

return 0;
}

```

Strategy

Description: Defines a **family of algorithms**, encapsulates each one, and makes them interchangeable. The strategy lets the algorithm vary independently from clients that use it.

- **Why it is used:** To select behavior at runtime without changing the client that uses it. Promotes the Open/Closed Principle.
- **Where it is commonly used:** Sorting algorithms (choose bubble sort vs. quicksort at runtime), payment processing systems, data compression strategies.

Code Example (CPP)

```

/*
 * STRATEGY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Strategy pattern defines a family of algorithms, encapsulates each
 * one, and makes them interchangeable.
 * It lets the algorithm vary independently from clients that use it.
 */

```

```

*
* WHY DOES IT EXIST?
* - When you want to define a family of algorithms and make them
interchangeable
* - When you want to avoid conditional statements for selecting algorithms
* - When you want to separate algorithm implementation from the code that
uses it
*
* WHEN TO USE IT?
* - When you have multiple ways to do an operation and want to switch
between them at runtime
* - When you want to isolate the algorithm implementation from the client
code
* - When you have complex conditional logic that selects different
algorithms
* - Common use cases: sorting algorithms, payment processing, compression,
route planning, validation
*
* PROS:
* - Open/Closed Principle (can add new strategies without changing
existing code)
* - Separates algorithm implementation from the code that uses it
* - Eliminates conditional statements for selecting algorithms
* - Allows runtime selection of algorithms
*
* CONS:
* - Clients must be aware of the differences between strategies
* - Can increase the number of classes in the system
* - May lead to performance overhead if strategies are complex
* - Can make the code more complex for simple scenarios
*
* REAL-WORLD ANALOGY:
* Think of a GPS navigation app. It can use different route calculation
strategies: fastest route, shortest distance, avoid highways, scenic route.
You can switch between these strategies at runtime without changing the app
itself. The app (context) delegates the route calculation to the selected
strategy.
*/

```

```

#include <iostream>
#include <string>
#include <memory>
#include <vector>
#include <algorithm>

// Strategy interface - declares the algorithm method
class PaymentStrategy {
public:
    virtual ~PaymentStrategy() = default;
    virtual void pay(double amount) = 0;
};

// Concrete Strategy 1: Credit Card
class CreditCardPayment : public PaymentStrategy {

```

```

private:
    std::string cardNumber;
    std::string expiryDate;
    std::string cvv;

public:
    CreditCardPayment(const std::string& card, const std::string& expiry,
const std::string& cvv)
        : cardNumber(card), expiryDate(expiry), cvv(cvv) {}

    void pay(double amount) override {
        std::cout << "Paying $" << amount << " with Credit Card ending in "
            << cardNumber.substr(cardNumber.length() - 4) <<
std::endl;
        std::cout << "   Card: ****-****-****-" <<
cardNumber.substr(cardNumber.length() - 4) << std::endl;
        std::cout << "   Expiry: " << expiryDate << std::endl;
    }
};

// Concrete Strategy 2: PayPal
class PayPalPayment : public PaymentStrategy {
private:
    std::string email;
    std::string password;

public:
    PayPalPayment(const std::string& email, const std::string& password)
        : email(email), password(password) {}

    void pay(double amount) override {
        std::cout << "Paying $" << amount << " with PayPal" << std::endl;
        std::cout << "   Account: " << email << std::endl;
    }
};

// Concrete Strategy 3: Crypto
class CryptoPayment : public PaymentStrategy {
private:
    std::string walletAddress;
    std::string privateKey;

public:
    CryptoPayment(const std::string& wallet, const std::string& key)
        : walletAddress(wallet), privateKey(key) {}

    void pay(double amount) override {
        std::cout << "Paying $" << amount << " with Cryptocurrency" <<
std::endl;
        std::cout << "   Wallet: " << walletAddress.substr(0, 10) << "..."
<< std::endl;
    }
};

```

```
// Context - uses the strategy
class ShoppingCart {
private:
    std::unique_ptr<PaymentStrategy> paymentStrategy;
    std::vector<std::string> items;
    std::vector<double> prices;

public:
    void setPaymentStrategy(std::unique_ptr<PaymentStrategy> strategy) {
        paymentStrategy = std::move(strategy);
    }

    void addItem(const std::string& item, double price) {
        items.push_back(item);
        prices.push_back(price);
    }

    double getTotal() const {
        double total = 0;
        for (double price : prices) {
            total += price;
        }
        return total;
    }

    void checkout() {
        std::cout << "\n=== Checkout ===" << std::endl;
        std::cout << "Items:" << std::endl;
        for (size_t i = 0; i < items.size(); i++) {
            std::cout << "  - " << items[i] << ": $" << prices[i] <<
std::endl;
        }
        std::cout << "Total: $" << getTotal() << std::endl;

        if (paymentStrategy) {
            paymentStrategy->pay(getTotal());
        } else {
            std::cout << "No payment method selected" << std::endl;
        }
    }
};

// Example 2: Sorting Strategies
// Shows how Strategy can be used for different algorithms

class SortingStrategy {
public:
    virtual ~SortingStrategy() = default;
    virtual void sort(std::vector<int>& data) = 0;
};

class BubbleSort : public SortingStrategy {
public:
    void sort(std::vector<int>& data) override {
```

```

        std::cout << "Using Bubble Sort..." << std::endl;
        int n = data.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (data[j] > data[j + 1]) {
                    std::swap(data[j], data[j + 1]);
                }
            }
        }
    }
};

class QuickSort : public SortingStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "Using Quick Sort..." << std::endl;
        quickSort(data, 0, data.size() - 1);
    }

private:
    void quickSort(std::vector<int>& data, int low, int high) {
        if (low < high) {
            int pi = partition(data, low, high);
            quickSort(data, low, pi - 1);
            quickSort(data, pi + 1, high);
        }
    }

    int partition(std::vector<int>& data, int low, int high) {
        int pivot = data[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (data[j] < pivot) {
                i++;
                std::swap(data[i], data[j]);
            }
        }
        std::swap(data[i + 1], data[high]);
        return i + 1;
    }
};

class MergeSort : public SortingStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "Using Merge Sort..." << std::endl;
        mergeSort(data, 0, data.size() - 1);
    }

private:
    void mergeSort(std::vector<int>& data, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(data, left, mid);

```

```

        mergeSort(data, mid + 1, right);
        merge(data, left, mid, right);
    }
}

void merge(std::vector<int>& data, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = data[left + i];
    for (int j = 0; j < n2; j++) R[j] = data[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            data[k++] = L[i++];
        } else {
            data[k++] = R[j++];
        }
    }

    while (i < n1) data[k++] = L[i++];
    while (j < n2) data[k++] = R[j++];
}

};

class Sorter {
private:
    std::unique_ptr<SortingStrategy> strategy;

public:
    void setSortingStrategy(std::unique_ptr<SortingStrategy> strat) {
        strategy = std::move(strat);
    }

    void sortData(std::vector<int>& data) {
        std::cout << "\nOriginal data: ";
        for (int num : data) std::cout << num << " ";
        std::cout << std::endl;

        if (strategy) {
            strategy->sort(data);
        } else {
            std::cout << "No sorting strategy selected" << std::endl;
        }

        std::cout << "Sorted data: ";
        for (int num : data) std::cout << num << " ";
        std::cout << std::endl;
    }
};

```

```
// Example 3: Route Planning Strategies
// Shows how Strategy can be used for navigation

class RouteStrategy {
public:
    virtual ~RouteStrategy() = default;
    virtual void calculateRoute(const std::string& from, const std::string&
to) = 0;
};

class FastestRoute : public RouteStrategy {
public:
    void calculateRoute(const std::string& from, const std::string& to)
override {
        std::cout << "Calculating fastest route from " << from << " to " <<
to << std::endl;
        std::cout << "    Using highways and main roads to minimize time" <<
std::endl;
        std::cout << "    Estimated time: 25 minutes" << std::endl;
    }
};

class ShortestRoute : public RouteStrategy {
public:
    void calculateRoute(const std::string& from, const std::string& to)
override {
        std::cout << "Calculating shortest route from " << from << " to "
<< to << std::endl;
        std::cout << "    Using direct path to minimize distance" <<
std::endl;
        std::cout << "    Estimated distance: 15 km" << std::endl;
    }
};

class ScenicRoute : public RouteStrategy {
public:
    void calculateRoute(const std::string& from, const std::string& to)
override {
        std::cout << "Calculating scenic route from " << from << " to " <<
to << std::endl;
        std::cout << "    Using parks and landmarks for best views" <<
std::endl;
        std::cout << "    Estimated time: 45 minutes" << std::endl;
    }
};

class NavigationSystem {
private:
    std::unique_ptr<RouteStrategy> strategy;

public:
    void setRouteStrategy(std::unique_ptr<RouteStrategy> strat) {
        strategy = std::move(strat);
    }
}
```

```
void navigate(const std::string& from, const std::string& to) {
    std::cout << "\n=== Navigation ===" << std::endl;
    std::cout << "From: " << from << std::endl;
    std::cout << "To: " << to << std::endl;

    if (strategy) {
        strategy->calculateRoute(from, to);
    } else {
        std::cout << "No route strategy selected" << std::endl;
    }
}

};

int main() {
    std::cout << "=== Strategy Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Payment Processing
    std::cout << "--- Payment Processing Example ---" << std::endl;

    ShoppingCart cart;
    cart.addItem("Laptop", 999.99);
    cart.addItem("Mouse", 29.99);
    cart.addItem("Keyboard", 79.99);

    std::cout << "\nPaying with Credit Card:" << std::endl;
    cart.setPaymentStrategy(std::make_unique<CreditCardPayment>
("4111111111111111", "12/25", "123"));
    cart.checkout();

    std::cout << "\nPaying with PayPal:" << std::endl;
    cart.setPaymentStrategy(std::make_unique<PayPalPayment>
("user@example.com", "password"));
    cart.checkout();

    std::cout << "\nPaying with Crypto:" << std::endl;
    cart.setPaymentStrategy(std::make_unique<CryptoPayment>
("0x1234567890abcdef", "private_key"));
    cart.checkout();

    // Example 2: Sorting Algorithms
    std::cout << "\n--- Sorting Algorithms Example ---" << std::endl;

    Sorter sorter;
    std::vector<int> data = {64, 34, 25, 12, 22, 11, 90};

    sorter.setSortingStrategy(std::make_unique<BubbleSort>());
    sorter.sortData(data);

    std::vector<int> data2 = {64, 34, 25, 12, 22, 11, 90};
    sorter.setSortingStrategy(std::make_unique<QuickSort>());
    sorter.sortData(data2);

    std::vector<int> data3 = {64, 34, 25, 12, 22, 11, 90};
```

```

    sorter.setSortingStrategy(std::make_unique<MergeSort>());
    sorter.sortData(data3);

    // Example 3: Route Planning
    std::cout << "\n--- Route Planning Example ---" << std::endl;

    NavigationSystem nav;

    std::cout << "\nFastest route:" << std::endl;
    nav.setRouteStrategy(std::make_unique<FastestRoute>());
    nav.navigate("Home", "Office");

    std::cout << "\nShortest route:" << std::endl;
    nav.setRouteStrategy(std::make_unique<ShortestRoute>());
    nav.navigate("Home", "Office");

    std::cout << "\nScenic route:" << std::endl;
    nav.setRouteStrategy(std::make_unique<ScenicRoute>());
    nav.navigate("Home", "Office");

    return 0;
}

```

Template Method

Description: Defines the **skeleton of an algorithm** in a base class, but lets subclasses override specific steps without changing the algorithm's overall structure.

- **Why it is used:** To avoid code duplication when multiple classes share the same algorithm structure but differ in specific steps.
- **Where it is commonly used:** Data parsing pipelines, report generation systems, game AI turn sequences.

Code Example (CPP)

```

/*
 * TEMPLATE METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Template Method pattern defines the skeleton of an algorithm in the
 * base class but lets subclasses override specific steps without changing the
 * algorithm's structure.
 * It's a behavioral pattern that uses inheritance to define parts of an
 * algorithm.
 *
 * WHY DOES IT EXIST?
 * - When you have an algorithm with invariant parts and variable parts
 * - When you want to avoid code duplication by extracting common behavior
 * - When you want to control the extension points of an algorithm
 */

```

```

* WHEN TO USE IT?
* - When you have several classes that contain almost identical algorithms
  with minor differences
* - When you want to let subclasses extend only certain parts of a large
  algorithm
* - When you want to enforce the overall structure of an algorithm while
  allowing customization
* - Common use cases: data processing pipelines, frameworks, build
  systems, report generation
*
* PROS:
* - Reuses common code in the base class
* - Follows Open/Closed Principle (extend behavior without modifying
  existing code)
* - Provides control over the extension points
* - Reduces code duplication
*
* CONS:
* - Can be difficult to maintain if the template method is complex
* - May limit flexibility due to rigid structure
* - Can lead to class explosion if many variations are needed
* - Subclasses may need to implement methods they don't use
*
* REAL-WORLD ANALOGY:
* Think of cooking a recipe. The recipe (template method) defines the
  steps: prepare ingredients, cook, serve. The specific ingredients and
  cooking methods can vary (subclass implementations), but the overall
  structure remains the same. You can make pasta, curry, or stir-fry using
  the same recipe structure with different implementations.
*/

#include <iostream>
#include <string>
#include <vector>

// Abstract Class - defines the template method and abstract primitive
operations
class DataProcessor {
public:
    virtual ~DataProcessor() = default;

    // Template method - defines the algorithm structure
    void processData() {
        std::cout << "\n=== Starting Data Processing ===" << std::endl;
        readData();
        validateData();
        transformData();
        saveData();
        std::cout << "=== Data Processing Complete ===" << std::endl;
    }

protected:
    // Primitive operations - to be implemented by subclasses
    virtual void readData() = 0;

```

```

    virtual void validateData() = 0;
    virtual void transformData() = 0;
    virtual void saveData() = 0;

    // Hook method - optional override point
    virtual void logProcessing() {
        std::cout << "Processing logged" << std::endl;
    }
};

// Concrete Class 1: CSV Data Processor
class CSVDataProcessor : public DataProcessor {
private:
    std::string data;

protected:
    void readData() override {
        std::cout << "Reading CSV file..." << std::endl;
        data = "name,age,city\nJohn,30,NYC\nJane,25,LA";
    }

    void validateData() override {
        std::cout << "Validating CSV format..." << std::endl;
        std::cout << "CSV format is valid" << std::endl;
    }

    void transformData() override {
        std::cout << "Transforming CSV to JSON..." << std::endl;
        // Simulate transformation
        data = "[{\"name\":\"John\",\"age\":30,\"city\":\"NYC\"},\n\
        {\"name\":\"Jane\",\"age\":25,\"city\":\"LA\"}]";
    }

    void saveData() override {
        std::cout << "Saving JSON file..." << std::endl;
        std::cout << "Data saved: " << data << std::endl;
    }
};

// Concrete Class 2: XML Data Processor
class XMLDataProcessor : public DataProcessor {
private:
    std::string data;

protected:
    void readData() override {
        std::cout << "Reading XML file..." << std::endl;
        data = "<users><user><name>John</name><age>30</age></user>\n\
        </users>";
    }

    void validateData() override {
        std::cout << "Validating XML structure..." << std::endl;
        std::cout << "XML structure is valid" << std::endl;
    }
};

```

```
}

void transformData() override {
    std::cout << "Transforming XML to CSV..." << std::endl;
    // Simulate transformation
    data = "name,age\nJohn,30";
}

void saveData() override {
    std::cout << "Saving CSV file..." << std::endl;
    std::cout << "Data saved: " << data << std::endl;
}

};

// Example 2: Report Generation
// Shows how Template Method can be used for report generation

class ReportGenerator {
public:
    virtual ~ReportGenerator() = default;

    // Template method
    void generateReport() {
        std::cout << "\n=== Generating Report ===" << std::endl;
        collectData();
        analyzeData();
        formatReport();
        exportReport();
        std::cout << "=== Report Generated ===" << std::endl;
    }

protected:
    virtual void collectData() = 0;
    virtual void analyzeData() = 0;
    virtual void formatReport() = 0;
    virtual void exportReport() = 0;
};

class SalesReport : public ReportGenerator {
protected:
    void collectData() override {
        std::cout << "Collecting sales data from database..." << std::endl;
    }

    void analyzeData() override {
        std::cout << "Analyzing sales trends..." << std::endl;
        std::cout << "Sales increased by 15% this quarter" << std::endl;
    }

    void formatReport() override {
        std::cout << "Formatting sales report with charts..." << std::endl;
    }

    void exportReport() override {
```

```

        std::cout << "Exporting to PDF..." << std::endl;
    }
};

class InventoryReport : public ReportGenerator {
protected:
    void collectData() override {
        std::cout << "Collecting inventory data..." << std::endl;
    }

    void analyzeData() override {
        std::cout << "Analyzing stock levels..." << std::endl;
        std::cout << "3 items need restocking" << std::endl;
    }

    void formatReport() override {
        std::cout << "Formatting inventory report with tables..." <<
std::endl;
    }

    void exportReport() override {
        std::cout << "Exporting to Excel..." << std::endl;
    }
};

// Example 3: Game Character Creation
// Shows how Template Method can be used for character creation

class GameCharacter {
public:
    virtual ~GameCharacter() = default;

    // Template method
    void createCharacter() {
        std::cout << "\n=== Creating Character ===" << std::endl;
        chooseName();
        selectRace();
        selectClass();
        assignStats();
        equipStartingItems();
        std::cout << "=== Character Created ===" << std::endl;
    }

protected:
    virtual void chooseName() = 0;
    virtual void selectRace() = 0;
    virtual void selectClass() = 0;
    virtual void assignStats() = 0;
    virtual void equipStartingItems() = 0;

    // Hook method
    virtual void displayCharacter() {
        std::cout << "Character ready for adventure!" << std::endl;
    }
}

```

```
};

class Warrior : public GameCharacter {
protected:
    void chooseName() override {
        std::cout << "Name chosen: Thorin" << std::endl;
    }

    void selectRace() override {
        std::cout << "Race selected: Dwarf" << std::endl;
    }

    void selectClass() override {
        std::cout << "Class selected: Warrior" << std::endl;
    }

    void assignStats() override {
        std::cout << "Stats assigned: STR: 18, DEX: 12, CON: 16, INT: 8,
WIS: 10" << std::endl;
    }

    void equipStartingItems() override {
        std::cout << "Equipped: Iron Sword, Steel Shield, Chain Mail" <<
std::endl;
    }

    void displayCharacter() override {
        GameCharacter::displayCharacter();
        std::cout << "Special ability: Battle Rage" << std::endl;
    }
};

class Mage : public GameCharacter {
protected:
    void chooseName() override {
        std::cout << "Name chosen: Gandalf" << std::endl;
    }

    void selectRace() override {
        std::cout << "Race selected: Human" << std::endl;
    }

    void selectClass() override {
        std::cout << "Class selected: Mage" << std::endl;
    }

    void assignStats() override {
        std::cout << "Stats assigned: STR: 8, DEX: 14, CON: 10, INT: 18,
WIS: 16" << std::endl;
    }

    void equipStartingItems() override {
        std::cout << "Equipped: Wooden Staff, Robe, Spellbook" <<
std::endl;
    }
};
```

```
}

void displayCharacter() override {
    GameCharacter::displayCharacter();
    std::cout << "Special ability: Fireball" << std::endl;
}

};

class Rogue : public GameCharacter {
protected:
    void chooseName() override {
        std::cout << "Name chosen: Legolas" << std::endl;
    }

    void selectRace() override {
        std::cout << "Race selected: Elf" << std::endl;
    }

    void selectClass() override {
        std::cout << "Class selected: Rogue" << std::endl;
    }

    void assignStats() override {
        std::cout << "Stats assigned: STR: 12, DEX: 18, CON: 12, INT: 14,
WIS: 12" << std::endl;
    }

    void equipStartingItems() override {
        std::cout << "Equipped: Dagger, Leather Armor, Lockpicks" <<
std::endl;
    }

    void displayCharacter() override {
        GameCharacter::displayCharacter();
        std::cout << "Special ability: Stealth" << std::endl;
    }
};

int main() {
    std::cout << "=== Template Method Pattern Demo ===" << std::endl <<
std::endl;

    // Example 1: Data Processing
    std::cout << "--- Data Processing Example ---" << std::endl;

    CSVDataProcessor csvProcessor;
    csvProcessor.processData();

    XMLDataProcessor xmlProcessor;
    xmlProcessor.processData();

    // Example 2: Report Generation
    std::cout << "\n--- Report Generation Example ---" << std::endl;
```

```
    SalesReport salesReport;
    salesReport.generateReport();

    InventoryReport inventoryReport;
    inventoryReport.generateReport();

    // Example 3: Game Character Creation
    std::cout << "\n--- Game Character Creation Example ---" << std::endl;

    Warrior warrior;
    warrior.createCharacter();

    Mage mage;
    mage.createCharacter();

    Rogue rogue;
    rogue.createCharacter();

    return 0;
}
```

Visitor

Description: Lets you **add new operations to existing object structures** without modifying the objects. A visitor object carries the operation to be performed.

- **Why it is used:** To separate an algorithm from the object structure it operates on, making it easy to add new operations without touching existing classes.
- **Where it is commonly used:** Compilers (AST traversal), document export systems (export to PDF, HTML, XML), tax/discount calculation systems.

Summary Table

Category	Focus	Goal
Creational	Object creation	Control how objects are instantiated
Structural	Object composition	Build flexible and efficient structures
Behavioral	Object interaction	Define clear communication between objects

Tip for Beginners: Don't try to memorize all patterns at once. Start with the most commonly used ones — **Singleton**, **Factory Method**, **Observer**, **Strategy**, and **Decorator** — and learn the rest as you encounter real-world problems they solve.

Code Example (CPP)

```
/*
 * VISITOR DESIGN PATTERN
```

```

*
* WHAT IS IT?
* The Visitor pattern lets you define a new operation without changing the
classes of the elements on which it operates.
* It separates the algorithm from the object structure it operates on.
*
* WHY DOES IT EXIST?
* - When you need to add operations to a complex object structure without
changing the structure
* - When you want to perform operations across different object types
* - When you want to centralize related operations in a single class
*
* WHEN TO USE IT?
* - When you have a complex object structure with different types of
elements
* - When you need to perform many unrelated operations on these elements
* - When the object structure is stable but you want to add new operations
frequently
* - Common use cases: document processing, compiler AST traversal, file
system operations, shopping cart calculations
*
* PROS:
* - Open/Closed Principle (can add new operations without changing element
classes)
* - Single Responsibility Principle (moves related operations to a single
visitor)
* - Can accumulate state while traversing the object structure
* - Works well with Composite pattern
*
* CONS:
* - Adding new element classes requires updating all visitors
* - Can violate encapsulation by exposing element internals
* - Can lead to complex visitor hierarchies
* - May make the code harder to understand
*
* REAL-WORLD ANALOGY:
* Think of a tax inspector visiting different types of businesses. The
inspector (visitor) visits a restaurant, a retail store, and a factory
(elements). Each business type calculates taxes differently. The inspector
knows how to calculate taxes for each type but doesn't change how the
businesses operate. You can add new types of inspections (health inspector,
fire inspector) without changing the businesses.
*/

```

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <map>

```

```

// Visitor interface - declares visit methods for each concrete element
class Visitor {
public:
    virtual ~Visitor() = default;

```

```
        virtual void visit(class Book* book) = 0;
        virtual void visit(class Fruit* fruit) = 0;
        virtual void visit(class Electronic* electronic) = 0;
};

// Element interface - declares the accept method
class Item {
public:
    virtual ~Item() = default;
    virtual void accept(Visitor* visitor) = 0;
    virtual double getPrice() const = 0;
};

// Concrete Element 1: Book
class Book : public Item {
private:
    std::string title;
    std::string author;
    double price;

public:
    Book(const std::string& title, const std::string& author, double price)
        : title(title), author(author), price(price) {}

    void accept(Visitor* visitor) override {
        visitor->visit(this);
    }

    double getPrice() const override {
        return price;
    }

    std::string getTitle() const { return title; }
    std::string getAuthor() const { return author; }
};

// Concrete Element 2: Fruit
class Fruit : public Item {
private:
    std::string name;
    double price;
    double weight;

public:
    Fruit(const std::string& name, double price, double weight)
        : name(name), price(price), weight(weight) {}

    void accept(Visitor* visitor) override {
        visitor->visit(this);
    }

    double getPrice() const override {
        return price;
    }
}
```

```

        std::string getName() const { return name; }
        double getWeight() const { return weight; }
};

// Concrete Element 3: Electronic
class Electronic : public Item {
private:
    std::string model;
    std::string brand;
    double price;

public:
    Electronic(const std::string& brand, const std::string& model, double
price)
        : model(model), brand(brand), price(price) {}

    void accept(Visitor* visitor) override {
        visitor->visit(this);
    }

    double getPrice() const override {
        return price;
    }

    std::string getModel() const { return model; }
    std::string getBrand() const { return brand; }
};

// Concrete Visitor 1: Price Calculator
class PriceCalculator : public Visitor {
private:
    double totalPrice;

public:
    PriceCalculator() : totalPrice(0) {}

    void visit(Book* book) override {
        totalPrice += book->getPrice();
        std::cout << "Book: " << book->getTitle() << " - $" << book-
>getPrice() << std::endl;
    }

    void visit(Fruit* fruit) override {
        totalPrice += fruit->getPrice();
        std::cout << "Fruit: " << fruit->getName() << " - $" << fruit-
>getPrice() << std::endl;
    }

    void visit(Electronic* electronic) override {
        totalPrice += electronic->getPrice();
        std::cout << "Electronic: " << electronic->getBrand() << " " <<
electronic->getModel()
            << " - $" << electronic->getPrice() << std::endl;
    }
};

```

```

    }

    double getTotalPrice() const {
        return totalPrice;
    }
};

// Concrete Visitor 2: Description Generator
class DescriptionGenerator : public Visitor {
public:
    void visit(Book* book) override {
        std::cout << "Book: \"" << book->getTitle() << "\" by " << book->getAuthor()
                    << " - A great read!" << std::endl;
    }

    void visit(Fruit* fruit) override {
        std::cout << "Fruit: " << fruit->getName() << " (" << fruit->getWeight() << "kg) - Fresh and healthy!" << std::endl;
    }

    void visit(Electronic* electronic) override {
        std::cout << "Electronic: " << electronic->getBrand() << " " <<
        electronic->getModel()
                    << " - High-tech gadget!" << std::endl;
    }
};

// Concrete Visitor 3: Tax Calculator
class TaxCalculator : public Visitor {
private:
    double totalTax;

public:
    TaxCalculator() : totalTax(0) {}

    void visit(Book* book) override {
        double tax = book->getPrice() * 0.05; // 5% tax on books
        totalTax += tax;
        std::cout << "Book tax: $" << tax << std::endl;
    }

    void visit(Fruit* fruit) override {
        double tax = fruit->getPrice() * 0.02; // 2% tax on food
        totalTax += tax;
        std::cout << "Fruit tax: $" << tax << std::endl;
    }

    void visit(Electronic* electronic) override {
        double tax = electronic->getPrice() * 0.10; // 10% tax on
        electronics
        totalTax += tax;
        std::cout << "Electronic tax: $" << tax << std::endl;
    }
}

```

```

        double getTotalTax() const {
            return totalTax;
        }
};

// Object Structure - manages the elements
class ShoppingCart {
private:
    std::vector<std::unique_ptr<Item>> items;

public:
    void addItem(std::unique_ptr<Item> item) {
        items.push_back(std::move(item));
    }

    void acceptVisitor(Visitor* visitor) {
        for (const auto& item : items) {
            item->accept(visitor);
        }
    }
};

// Example 2: File System Visitor
// Shows how Visitor can be used for file system operations

class File;
class Directory;

class FileSystemVisitor {
public:
    virtual ~FileSystemVisitor() = default;
    virtual void visit(File* file) = 0;
    virtual void visit(Directory* directory) = 0;
};

class FileSystemNode {
public:
    virtual ~FileSystemNode() = default;
    virtual void accept(FileSystemVisitor* visitor) = 0;
    virtual std::string getName() const = 0;
    virtual int getSize() const = 0;
};

class File : public FileSystemNode {
private:
    std::string name;
    int size;

public:
    File(const std::string& name, int size) : name(name), size(size) {}

    void accept(FileSystemVisitor* visitor) override {
        visitor->visit(this);
    }
};

```

```

    }

    std::string getName() const override { return name; }
    int getSize() const override { return size; }
};

class Directory : public FileSystemNode {
private:
    std::string name;
    std::vector<std::shared_ptr<FileSystemNode>> children;

public:
    Directory(const std::string& name) : name(name) {}

    void addChild(std::shared_ptr<FileSystemNode> child) {
        children.push_back(child);
    }

    void accept(FileSystemVisitor* visitor) override {
        visitor->visit(this);
    }

    std::string getName() const override { return name; }
    int getSize() const override {
        int total = 0;
        for (const auto& child : children) {
            total += child->getSize();
        }
        return total;
    }
};

class SizeCalculator : public FileSystemVisitor {
private:
    int totalSize;

public:
    SizeCalculator() : totalSize(0) {}

    void visit(File* file) override {
        totalSize += file->getSize();
        std::cout << "File: " << file->getName() << " - " << file-
>getSize() << " bytes" << std::endl;
    }

    void visit(Directory* directory) override {
        std::cout << "Directory: " << directory->getName() << " - " <<
directory->getSize() << " bytes" << std::endl;
        totalSize += directory->getSize();
    }

    int getTotalSize() const {
        return totalSize;
    }
}

```

```

};

class XMLExporter : public FileSystemVisitor {
public:
    void visit(File* file) override {
        std::cout << "    <file name=\"" << file->getName() << "\"" size=\""
<< file->getSize() << "\"/>" << std::endl;
    }

    void visit(Directory* directory) override {
        std::cout << "    <directory name=\"" << directory->getName() << "\""
size=\"" << directory->getSize() << "\">" << std::endl;
    }
};

// Example 3: Employee Salary Calculator
// Shows how Visitor can be used for salary calculations

class Employee;
class Manager;
class Developer;
class Designer;

class SalaryVisitor {
public:
    virtual ~SalaryVisitor() = default;
    virtual void visit(Employee* employee) = 0;
    virtual void visit(Manager* manager) = 0;
    virtual void visit(Developer* developer) = 0;
    virtual void visit(Designer* designer) = 0;
};

class Employee {
public:
    virtual ~Employee() = default;
    virtual void accept(SalaryVisitor* visitor) = 0;
    virtual std::string getName() const = 0;
    virtual double getBaseSalary() const = 0;
};

class Manager : public Employee {
private:
    std::string name;
    double baseSalary;
    double bonus;

public:
    Manager(const std::string& name, double salary, double bonus)
        : name(name), baseSalary(salary), bonus(bonus) {}

    void accept(SalaryVisitor* visitor) override {
        visitor->visit(this);
    }
}

```

```
std::string getName() const override { return name; }
double getBaseSalary() const override { return baseSalary; }
double getBonus() const { return bonus; }
};

class Developer : public Employee {
private:
    std::string name;
    double baseSalary;
    int overtimeHours;

public:
    Developer(const std::string& name, double salary, int overtime)
        : name(name), baseSalary(salary), overtimeHours(overtime) {}

    void accept(SalaryVisitor* visitor) override {
        visitor->visit(this);
    }

    std::string getName() const override { return name; }
    double getBaseSalary() const override { return baseSalary; }
    int getOvertimeHours() const { return overtimeHours; }
};

class Designer : public Employee {
private:
    std::string name;
    double baseSalary;
    int projectsCompleted;

public:
    Designer(const std::string& name, double salary, int projects)
        : name(name), baseSalary(salary), projectsCompleted(projects) {}

    void accept(SalaryVisitor* visitor) override {
        visitor->visit(this);
    }

    std::string getName() const override { return name; }
    double getBaseSalary() const override { return baseSalary; }
    int getProjectsCompleted() const { return projectsCompleted; }
};

class SalaryCalculator : public SalaryVisitor {
private:
    double totalSalary;

public:
    SalaryCalculator() : totalSalary(0) {}

    void visit(Employee* employee) override {
        double salary = employee->getBaseSalary();
        totalSalary += salary;
        std::cout << "Employee: " << employee->getName() << " - $" <<
```

```

salary << std::endl;
}

void visit(Manager* manager) override {
    double salary = manager->getBaseSalary() + manager->getBonus();
    totalSalary += salary;
    std::cout << "Manager: " << manager->getName() << " - $" << salary
<< " (base: $"
                << manager->getBaseSalary() << " + bonus: $" << manager-
>getBonus() << ")" << std::endl;
}

void visit(Developer* developer) override {
    double overtimePay = developer->getOvertimeHours() * 50.0;
    double salary = developer->getBaseSalary() + overtimePay;
    totalSalary += salary;
    std::cout << "Developer: " << developer->getName() << " - $" <<
salary
                << " (base: $" << developer->getBaseSalary() << " +
overtime: $" << overtimePay << ")" << std::endl;
}

void visit(Designer* designer) override {
    double projectBonus = designer->getProjectsCompleted() * 500.0;
    double salary = designer->getBaseSalary() + projectBonus;
    totalSalary += salary;
    std::cout << "Designer: " << designer->getName() << " - $" <<
salary
                << " (base: $" << designer->getBaseSalary() << " +
project bonus: $" << projectBonus << ")" << std::endl;
}

double getTotalSalary() const {
    return totalSalary;
}

};

int main() {
    std::cout << "=== Visitor Pattern Demo ===" << std::endl << std::endl;

    // Example 1: Shopping Cart
    std::cout << "--- Shopping Cart Example ---" << std::endl;

    ShoppingCart cart;
    cart.addItem(std::make_unique<Book>("Design Patterns", "Erich Gamma",
49.99));
    cart.addItem(std::make_unique<Fruit>("Apple", 1.99, 0.5));
    cart.addItem(std::make_unique<Electronic>("Apple", "iPhone 15",
999.99));

    std::cout << "\nCalculating total price:" << std::endl;
    PriceCalculator priceCalc;
    cart.acceptVisitor(&priceCalc);
    std::cout << "Total: $" << priceCalc.getTotalPrice() << std::endl;
}

```

```

std::cout << "\nGenerating descriptions:" << std::endl;
DescriptionGenerator descGen;
cart.acceptVisitor(&descGen);

std::cout << "\nCalculating tax:" << std::endl;
TaxCalculator taxCalc;
cart.acceptVisitor(&taxCalc);
std::cout << "Total tax: $" << taxCalc.getTotalTax() << std::endl;

// Example 2: File System
std::cout << "\n--- File System Example ---" << std::endl;

auto root = std::make_shared<Directory>("root");
auto file1 = std::make_shared<File>("document.txt", 1024);
auto file2 = std::make_shared<File>("image.jpg", 2048);
auto subdir = std::make_shared<Directory>("subdir");
auto file3 = std::make_shared<File>("data.csv", 512);

subdir->addChild(file3);
root->addChild(file1);
root->addChild(file2);
root->addChild(subdir);

std::cout << "\nCalculating sizes:" << std::endl;
SizeCalculator sizeCalc;
root->accept(&sizeCalc);
std::cout << "Total size: " << sizeCalc.getTotalSize() << " bytes" <<
std::endl;

std::cout << "\nExporting to XML:" << std::endl;
std::cout << "<filesystem>" << std::endl;
XMLExporter xmlExporter;
root->accept(&xmlExporter);
std::cout << "</filesystem>" << std::endl;

// Example 3: Employee Salary
std::cout << "\n--- Employee Salary Example ---" << std::endl;

std::vector<std::unique_ptr<Employee>> employees;
employees.push_back(std::make_unique<Manager>("Alice", 80000, 20000));
employees.push_back(std::make_unique<Developer>("Bob", 70000, 10));
employees.push_back(std::make_unique<Designer>("Charlie", 65000, 5));

std::cout << "\nCalculating total salary:" << std::endl;
SalaryCalculator salaryCalc;
for (const auto& emp : employees) {
    emp->accept(&salaryCalc);
}
std::cout << "Total salary: $" << salaryCalc.getTotalSalary() <<
std::endl;

```

```
    return 0;  
}
```